

Web Scrapping + Database dengan Open Source LLM: Pipeline Data Modern

Onno W. Purbo
onno@indo.net.id
onno@itts.ac.id
Twitter onnowpurbo

**Institut Teknologi Tangerang Selatan (ITTS)
Dinas KOMINFO Tangerang Selatan
OnnoCenter
2026**

Lisensi: Karya ini dilisensikan di bawah **Creative Commons Attribution (CC BY) 4.0 International**. Anda bebas menggunakan, membagikan, dan mengadaptasi karya ini untuk tujuan apa pun, termasuk komersial, dengan tetap mencantumkan atribusi yang layak kepada penulis.

Disclaimer: Buku ini disusun untuk tujuan edukasi, dokumentasi, dan refleksi publik berdasarkan sumber-sumber yang dianggap kredibel pada saat penulisan. Penulis telah berupaya menjaga akurasi, namun sangat mungkin masih terdapat kekurangan atau kekeliruan; karena itu, buku ini tidak dimaksudkan sebagai satu-satunya rujukan yang final, dan penulis terbuka terhadap koreksi serta masukan yang membangun

Data yang baik bukan sekedar berhasil diambil dari web, tetapi harus legal, bersih, terstruktur, dapat diaudit, dan siap dipakai untuk keputusan.

Kata Pengantar

Buku **Web Scraping + Database dengan Open Source LLM: Pipeline Data Modern, 2026** disusun untuk membantu mahasiswa memahami keterampilan data modern secara utuh: mulai dari mengambil data web secara bertanggung jawab, membersihkan dan memvalidasi data, menyimpannya ke database open source, hingga memanfaatkan LLM lokal untuk ekstraksi, ringkasan, dan struktur JSON.

Buku ini tidak menempatkan *web scraping* sebagai sekadar teknik mengambil data, tetapi sebagai bagian dari **pipeline data yang legal, etis, bersih, terstruktur, dan dapat diaudit**. Karena itu, pembahasan tidak berhenti pada script, tetapi juga mencakup kualitas data, database, keamanan credential, logging, dokumentasi, dan portofolio profesional.

Seluruh contoh diarahkan menggunakan **Python dan tools open source** seperti SQLite, PostgreSQL, MongoDB, Scrapy, Playwright, Crawl4AI, Ollama, dan Pydantic. Harapannya, mahasiswa tidak hanya mampu mengikuti praktik, tetapi juga memahami cara berpikir seorang *data engineer* dan *AI engineer* yang bertanggung jawab.

Semoga buku ini menjadi bekal awal untuk membangun proyek data yang bermanfaat, reproducible, dan siap dikembangkan menjadi portofolio nyata. Data yang baik bukan hanya data yang berhasil diambil, tetapi data yang **dapat dipercaya, dijelaskan, dan dipertanggungjawabkan**.

Jakarta, Mei 2026

Onno W. Purbo

Executive Summary

Buku **Web Scraping + Database dengan Open Source LLM: Pipeline Data Modern, 2026** dirancang sebagai panduan strategis bagi mahasiswa untuk memahami dan membangun **pipeline data modern** secara utuh: mulai dari mengambil data web secara bertanggung jawab, membersihkan dan memvalidasi data, menyimpannya ke database, hingga memanfaatkan *open source LLM* untuk ekstraksi informasi, ringkasan, klasifikasi, dan interaksi cerdas. Buku ini tidak memposisikan *web scraping* sebagai keterampilan teknis yang berdiri sendiri, melainkan sebagai bagian dari **rekayasa data end-to-end** yang menuntut etika, kualitas, auditabilitas, dan kemampuan reproduksi. Dengan fondasi Python, Docker, SQLite, PostgreSQL, MongoDB, Scrapy, Playwright, Crawl4AI, Ollama, dan Pydantic, mahasiswa diarahkan untuk membangun proyek yang bukan hanya berjalan, tetapi juga rapi, terstruktur, aman, dan layak menjadi portofolio profesional. Scrapy diperkenalkan sebagai tulang punggung crawling skala menengah karena mampu mengelola banyak halaman, item, pipeline, dan ekspor data secara sistematis. BeautifulSoup tetap digunakan sebagai pintu masuk yang sederhana untuk memahami struktur HTML dan proses parsing. Playwright menjadi penting karena banyak website modern memuat konten melalui JavaScript, sehingga membutuhkan otomasi browser untuk membaca hasil render secara akurat. Crawl4AI memperkuat pipeline karena menghasilkan konten yang lebih siap diproses oleh LLM, sementara Ollama memberi mahasiswa kemampuan menjalankan model lokal tanpa bergantung pada layanan tertutup. Revisi buku ini mempertajam arah belajar dari sekadar “mengambil data” menjadi **membangun sistem data yang legal, bersih, tervalidasi, dapat diaudit, dan siap digunakan untuk analisis maupun AI**. Pesan utama buku ini jelas: masa depan mahasiswa data bukan hanya ditentukan oleh kemampuan menulis script, tetapi oleh kemampuan merancang pipeline yang bertanggung jawab, reproducible, dan bernilai nyata.

Daftar Isi

Kata Pengantar.....	4
Executive Summary.....	5
Daftar Isi.....	6
Bab 1 — Web Scraping, Database, dan LLM: Skill Data Modern untuk Mahasiswa.....	18
1.1 Mengapa Mahasiswa Perlu Skill Pipeline Data Modern.....	18
1.2 Data Web sebagai Bahan Baku Riset, Bisnis, dan AI.....	19
1.3 Perbedaan Web Scraping, Crawling, API Access, dan Data Ingestion.....	20
1. Web Scraping.....	20
2. Crawling.....	20
3. API Access.....	20
4. Data Ingestion.....	21
1.4 Mengapa Database Penting dalam Pipeline Scraping.....	22
1.5 Peran Open Source LLM dalam Pipeline Data.....	23
1.6 Risiko Web Scraping yang Wajib Dipahami.....	24
Risiko utama.....	24
Prinsip aman.....	24
1.7 Gambaran Pipeline Buku.....	25
Bagian Praktik.....	26
1.8 Instalasi Python.....	26
1.9 Membuat Folder Proyek.....	27
1.10 Membuat Virtual Environment.....	28
1.11 Instalasi Paket Awal.....	29
1.12 Instalasi Editor Kode Open Source.....	29
1.13 Membuat File .env.....	30
1.14 Script Pertama: “Hello Pipeline”.....	31
1.15 Penjelasan Script per Bagian.....	32
1.16 Checklist Praktik Bab 1.....	33
1.17 Kesalahan Umum di Bab 1.....	34
1. Virtual environment belum aktif.....	34
2. File .env salah lokasi.....	34
3. Salah menjalankan path script.....	34
4. Paket belum tercatat.....	34
1.18 Ringkasan Bab.....	35
Bab 2 — Dasar HTTP, HTML, dan Parsing Web dengan Python.....	36
2.1 Mengapa HTTP dan HTML Wajib Dipahami.....	36
2.2 Cara Kerja HTTP Request dan Response.....	37
2.3 Metode HTTP yang Paling Sering Ditemui.....	38
1. GET.....	38
2. POST.....	38
3. HEAD.....	38
4. OPTIONS.....	38
2.4 Status Code yang Harus Dipahami.....	39

200 — OK.....	39
301 — Moved Permanently.....	39
403 — Forbidden.....	39
404 — Not Found.....	39
429 — Too Many Requests.....	39
500 — Internal Server Error.....	40
2.5 Header, User-Agent, Content-Type, dan Encoding.....	41
User-Agent.....	41
Content-Type.....	41
Encoding.....	42
2.6 Struktur HTML: Tag, Class, ID, Link, Table, dan Form.....	43
Tag.....	43
Class.....	43
ID.....	43
Link.....	44
Table.....	44
Form.....	44
2.7 HTML Statis dan Konten Dinamis.....	45
HTML Statis.....	45
Konten Dinamis.....	45
2.8 Dasar Selector: CSS Selector dan XPath.....	46
CSS Selector.....	46
XPath.....	46
2.9 Etika Dasar Scraping.....	47
Bagian Praktik.....	48
2.10 Persiapan Folder Bab 2.....	48
2.11 Instalasi Paket Praktik.....	48
2.12 Praktik 1 — Mengambil Halaman Web dengan Requests.....	49
2.13 Praktik 2 — Menambahkan Header Sederhana.....	50
2.14 Praktik 3 — Menyimpan HTML Mentah.....	51
2.15 Praktik 4 — Parsing HTML dengan BeautifulSoup.....	52
2.16 Praktik 5 — Mengambil Semua Link.....	53
2.17 Praktik 6 — Mengambil Paragraf.....	54
2.18 Praktik 7 — Menyimpan Hasil ke JSON.....	55
2.19 Praktik 8 — Menyimpan Hasil ke CSV.....	57
2.20 Praktik 9 — Membaca Tabel Sederhana dari HTML.....	59
2.21 Praktik 10 — Scraper Mini untuk Halaman Statis.....	61
2.22 Penjelasan Struktur Scraper Mini.....	64
fetch_page().....	64
parse_page().....	64
save_json().....	64
save_csv().....	64
main().....	64
2.23 Catatan Struktur HTML Target.....	65

2.24 Checklist Praktik Bab 2.....	66
2.25 Kesalahan Umum Bab 2.....	67
1. Langsung parsing tanpa cek status code.....	67
2. Data kosong karena halaman dinamis.....	67
3. Selector terlalu rapuh.....	67
4. Tidak menyimpan URL sumber.....	67
5. Tidak menyimpan raw HTML.....	68
6. Terlalu banyak request.....	68
2.26 Ringkasan Bab.....	69
Bab 3 — Scraping Terstruktur dengan BeautifulSoup, lxml, dan Pandas.....	70
3.1 Dari Scraping Mentah ke Scraping Terstruktur.....	70
3.2 Strategi Memilih Elemen HTML yang Stabil.....	71
Prinsip memilih selector.....	71
3.3 Parsing Cepat dengan lxml.....	72
3.4 Ekstraksi Tabel HTML.....	73
3.5 Normalisasi Teks.....	74
3.6 Struktur Data: List of Dict, JSON Lines, dan CSV.....	75
1. List of Dict.....	75
2. JSON Lines.....	75
3. CSV.....	76
3.7 Kesalahan Umum Scraping Terstruktur.....	77
1. Selector rapuh.....	77
2. Data kosong.....	77
3. Duplikasi.....	77
4. Perubahan layout.....	78
5. Tidak ada data sumber.....	78
Bagian Praktik.....	79
3.8 Persiapan Folder Bab 3.....	79
3.9 Praktik 1 — Membuat HTML Latihan Daftar Artikel.....	80
3.10 Praktik 2 — Parsing Daftar Artikel.....	82
3.11 Penjelasan Script Parsing Artikel.....	84
3.12 Praktik 3 — Menyimpan ke JSON, JSON Lines, dan CSV.....	85
3.13 Praktik 4 — Membaca CSV dengan Pandas.....	88
3.14 Praktik 5 — Membersihkan Dataset dengan Pandas.....	89
3.15 Praktik 6 — Membuat Log Error Sederhana.....	90
3.16 Praktik 7 — Scraping Tabel Publik dengan Pandas.....	94
3.17 Praktik 8 — Fungsi Reusable: fetch_page(), parse_items(), save_json().....	96
3.18 Praktik 9 — Deduplication Berdasarkan URL.....	99
3.19 Praktik 10 — Membuat Ringkasan Kualitas Dataset.....	100
3.20 Struktur Akhir Folder Bab 3.....	102
3.21 Checklist Praktik Bab 3.....	103
3.22 Kesalahan Umum Bab 3.....	104
1. Mengambil semua judul, semua tanggal, semua author secara terpisah.....	104
2. Tidak membersihkan teks.....	104

3. Tidak menangani field kosong.....	104
4. Tidak membuat log.....	105
5. Tidak menghapus duplikasi.....	105
6. Langsung percaya hasil scraping.....	105
3.23 Ringkasan Bab.....	106
Bab 4 — Scrapy Framework untuk Crawling yang Lebih Serius.....	107
4.1 Mengapa Perlu Scrapy?.....	107
4.2 Kapan Memakai BeautifulSoup, Kapan Memakai Scrapy?.....	108
Gunakan BeautifulSoup jika:.....	108
Gunakan Scrapy jika:.....	108
Ringkasnya.....	108
4.3 Arsitektur Scrapy.....	109
4.4 Konsep Spider.....	110
4.5 Konsep Item.....	111
4.6 Konsep Pipeline.....	112
4.7 Pagination dan Follow Link.....	113
4.8 Delay, Retry, Timeout, dan Throttling.....	114
Bagian Praktik.....	115
4.9 Persiapan Folder Bab 4.....	115
4.10 Instalasi Scrapy.....	116
4.11 Membuat Project Scrapy.....	117
4.12 Membuat Halaman Latihan Lokal.....	118
4.13 Membuat Spider Pertama.....	120
4.14 Penjelasan Spider Pertama.....	122
4.15 Export ke JSON Lines.....	123
4.16 Export ke JSON dan CSV.....	124
4.17 Menggunakan Item.....	125
4.18 Membuat Pipeline Pembersihan Awal.....	127
4.19 Mengatur Delay, Retry, Timeout, dan AutoThrottle.....	129
4.20 Menyimpan Log Scrapy.....	130
4.21 Membaca Hasil Scrapy dengan Pandas.....	131
4.22 Struktur Akhir Project Bab 4.....	132
4.23 Checklist Praktik Bab 4.....	133
4.24 Kesalahan Umum Bab 4.....	134
1. Menjalankan spider dari folder yang salah.....	134
2. Nama spider tidak cocok.....	134
3. Selector salah.....	134
4. Lupa mengaktifkan pipeline.....	135
5. Output lama tercampur.....	135
6. Crawling terlalu agresif.....	135
4.25 Praktik Scraping yang Bertanggung Jawab.....	136
4.26 Ringkasan Bab.....	137
Bab 5 — Database Open Source untuk Data Scraping: SQLite, PostgreSQL, dan MongoDB.....	138

5.1 Mengapa Data Scraping Perlu Database?	138
5.2 CSV, SQLite, PostgreSQL, dan MongoDB: Apa Bedanya?	139
5.3 Konsep Dasar Database Relasional	140
Tabel	140
Kolom	140
Primary Key	141
Index	141
Relasi	141
5.4 Konsep Document Database	142
5.5 Desain Schema untuk Data Scraping	143
5.6 Risiko Data Duplikat dan Data Kotor	144
Bagian Praktik	145
5.7 Persiapan Folder Bab 5	145
5.8 Menyiapkan Dataset Latihan	146
5.9 Praktik 1 — Membaca JSON Lines dengan Pandas	147
5.10 Praktik 2 — Membuat Content Hash	148
SQLite	149
5.11 Instalasi dan Pemeriksaan SQLite	149
5.12 Praktik 3 — Membuat Schema SQLite	150
5.13 Praktik 4 — Membuat Database SQLite	151
5.14 Praktik 5 — Insert Data ke SQLite	152
5.15 Praktik 6 — Query Dasar SQLite	153
PostgreSQL	154
5.16 Mengapa PostgreSQL?	154
5.17 Menjalankan PostgreSQL dengan Docker Compose	155
5.18 Praktik 7 — Membuat File .env untuk Database	157
5.19 Praktik 8 — Membuat Tabel PostgreSQL	158
5.20 Praktik 9 — Insert Data ke PostgreSQL	160
5.21 Praktik 10 — Query Dasar PostgreSQL	162
MongoDB	164
5.22 Mengapa MongoDB?	164
5.23 Praktik 11 — Insert Dokumen JSON ke MongoDB	165
5.24 Praktik 12 — Query Dasar MongoDB	166
5.25 Membandingkan Query SQLite, PostgreSQL, dan MongoDB	167
SQLite	167
PostgreSQL	167
MongoDB	167
5.26 Praktik 13 — Membuat Index MongoDB	168
5.27 Praktik 14 — Membuat Ringkasan Kualitas Data dari Database	169
5.28 Struktur Akhir Folder Bab 5	171
5.29 Checklist Praktik Bab 5	172
5.30 Kesalahan Umum Bab 5	173
1. Menganggap CSV sudah cukup untuk semua kasus	173
2. Tidak membuat field unik	173

3. Menyimpan data tanpa source_url.....	173
4. Memasukkan data kosong ke database.....	174
5. Credential ditulis langsung di kode.....	174
6. Container hidup, tetapi koneksi gagal.....	174
7. MongoDB terlalu bebas.....	175
5.31 Ringkasan Bab.....	175
Bab 6 — Cleaning, Validation, dan Data Quality sebelum Masuk Database.....	176
6.1 Mengapa Cleaning dan Validasi Wajib?.....	176
6.2 Masalah Umum Data Scraping.....	177
1. Data kosong.....	177
2. Duplikasi.....	177
3. Tanggal tidak seragam.....	177
4. HTML tersisa.....	177
5. Encoding rusak.....	177
6. Field tidak konsisten.....	177
6.3 Cleaning dengan Python dan Pandas.....	178
6.4 Validasi Field Wajib.....	179
6.5 Validasi URL, Tanggal, Angka, dan Kategori.....	180
URL.....	180
Tanggal.....	180
Angka.....	180
Kategori.....	180
6.6 Deduplication Berdasarkan URL dan Hash.....	181
6.7 Logging Data yang Gagal Diproses.....	182
6.8 Konsep Data Lineage.....	183
Bagian Praktik.....	184
6.9 Persiapan Folder Bab 6.....	184
6.10 Membuat Dataset Kotor untuk Latihan.....	185
6.11 Praktik 1 — Membaca dan Mengecek Dataset Kotor.....	186
6.12 Praktik 2 — Membersihkan Teks dengan Python.....	187
6.13 Praktik 3 — Membuat Content Hash.....	188
6.14 Praktik 4 — Deduplication Berdasarkan URL dan Hash.....	189
6.15 Praktik 5 — Membuat Schema Pydantic.....	190
6.16 Penjelasan Schema Pydantic.....	192
6.17 Praktik 6 — Mengubah Data Valid ke CSV Siap Database.....	193
6.18 Praktik 7 — Membuat Ringkasan Kualitas Data.....	194
6.19 Praktik 8 — Membuat Script Cleaning Terpadu.....	196
6.20 Praktik 9 — Validasi Sebelum Insert ke SQLite.....	200
6.21 Struktur Akhir Folder Bab 6.....	202
6.22 Checklist Praktik Bab 6.....	203
6.23 Kesalahan Umum Bab 6.....	204
1. Membersihkan data setelah masuk database.....	204
2. Tidak menyimpan rejected data.....	204
3. Hanya cek URL, tidak cek isi.....	204

4. Menganggap data kosong sebagai normal.....	204
5. Tidak membuat laporan kualitas.....	204
6. Tidak menyimpan lineage.....	205
6.24 Ringkasan Bab.....	205
Bab 7 — Open Source LLM untuk Ekstraksi, Ringkasan, dan Struktur JSON.....	206
7.1 Mengapa LLM Masuk ke Pipeline Scraping?.....	206
7.2 Scraping Biasa vs LLM-Assisted Extraction.....	207
Scraping biasa.....	207
LLM-assisted extraction.....	208
7.3 Kapan LLM Berguna, Kapan Tidak Perlu?.....	209
LLM berguna ketika:.....	209
LLM tidak perlu ketika:.....	209
7.4 Memilih Model Lokal Ringan untuk Mahasiswa.....	210
7.5 Risiko Halusinasi dan Cara Mengurangnya.....	211
7.6 JSON sebagai Format Utama Output LLM.....	212
Bagian Praktik.....	213
7.7 Persiapan Folder Bab 7.....	213
7.8 Instalasi Ollama.....	214
7.9 Konfigurasi .env.....	215
7.10 Membuat Data Artikel untuk Latihan.....	216
7.11 Praktik 1 — Memanggil Ollama dari Python.....	217
7.12 Praktik 2 — Prompt Ringkasan Artikel.....	218
7.13 Praktik 3 — Prompt Ekstraksi Entitas ke JSON.....	220
7.14 Praktik 4 — Membersihkan Output JSON dari LLM.....	222
7.15 Praktik 5 — Validasi JSON LLM dengan Pydantic.....	224
7.16 Praktik 6 — Membuat Tabel PostgreSQL untuk Hasil LLM.....	226
7.17 Praktik 7 — Membuat Tabel dari Python.....	227
7.18 Praktik 8 — Insert Hasil LLM ke PostgreSQL.....	228
7.19 Praktik 9 — Query Hasil LLM.....	230
7.20 Praktik 10 — Contoh Anti-Halusinasi Sederhana.....	232
7.21 Crawl4AI untuk Konten yang Lebih Bersih.....	234
7.22 Struktur Akhir Folder Bab 7.....	235
7.23 Checklist Praktik Bab 7.....	236
7.24 Kesalahan Umum Bab 7.....	237
1. Menganggap LLM selalu benar.....	237
2. Tidak meminta JSON valid.....	237
3. Tidak menyimpan output mentah.....	237
4. Tidak memakai schema.....	237
5. Memakai LLM untuk semua hal.....	238
6. Tidak menyimpan nama model.....	238
7. Tidak melakukan upsert.....	238
7.25 Ringkasan Bab.....	239
Bab 8 — Playwright dan Scraping Website Dinamis.....	240
8.1 Mengapa Website JavaScript-Heavy Sulit di-Scrape?.....	240

8.2 HTTP Scraping vs Browser Automation.....	242
HTTP scraping.....	242
Browser automation.....	243
8.3 Apa Itu Headless Browser?.....	244
8.4 Waiting Strategy: Jangan Asal Sleep.....	245
8.5 Screenshot dan Debugging.....	246
8.6 Risiko Scraping Login Page dan Data Privat.....	247
Bagian Praktik.....	248
8.7 Persiapan Folder Bab 8.....	248
8.8 Membuat Halaman Dinamis Lokal untuk Latihan.....	249
8.9 Praktik 1 — Menjalankan Browser Headless.....	252
8.10 Praktik 2 — Menunggu Konten JavaScript Selesai.....	253
8.11 Praktik 3 — Mengambil Data Hasil Render.....	254
8.12 Praktik 4 — Klik Tombol Load More.....	256
8.13 Praktik 5 — Mengisi Form Pencarian.....	258
8.14 Praktik 6 — Menyimpan Data ke CSV.....	260
8.15 Praktik 7 — Membuat Content Hash dan Timestamp.....	261
8.16 Praktik 8 — Simpan Data ke SQLite.....	262
8.17 Praktik 9 — Logging Screenshot Jika Gagal.....	264
8.18 Praktik 10 — Pipeline Playwright → Clean → Database.....	266
8.19 Struktur Akhir Folder Bab 8.....	269
8.20 Checklist Praktik Bab 8.....	270
8.21 Kesalahan Umum Bab 8.....	271
1. Memakai Playwright untuk semua scraping.....	271
2. Mengambil data sebelum render selesai.....	271
3. Terlalu banyak memakai fixed timeout.....	271
4. Tidak menyimpan screenshot.....	271
5. Scraping login page tanpa izin.....	272
6. Tidak menyimpan source_url.....	272
7. Tidak membersihkan data sebelum database.....	272
8.22 Ringkasan Bab.....	273
Bab 9 — Pipeline End-to-End: Web → Scraper → LLM → Database → Query.....	274
9.1 Mengapa Bab Ini Menjadi Jantung Buku?.....	274
9.2 Arsitektur Pipeline End-to-End.....	276
Jalur 1 — Raw Data.....	276
Jalur 2 — Processed Data.....	276
9.3 Folder Proyek Produksi Sederhana.....	277
9.4 Konfigurasi dengan .env.....	278
9.5 Docker Compose untuk Database.....	279
9.6 Schema PostgreSQL.....	280
9.7 Instalasi Paket Python.....	282
Bagian Praktik Proyek Utama.....	283
9.8 Membuat Halaman Artikel Legal untuk Latihan.....	283
9.9 Membuat Script Pipeline Utama.....	285

9.10 Penjelasan Script run_pipeline.py.....	295
fetch_html().....	295
parse_articles().....	295
save_raw_to_mongo().....	295
validate_and_deduplicate().....	296
extract_with_llm().....	296
save_processed_to_postgres().....	296
save_pipeline_run().....	296
9.11 Error Handling dan Retry.....	297
9.12 Logging Pipeline.....	299
9.13 Query PostgreSQL untuk Analisis.....	300
9.14 Query MongoDB untuk Dokumen Mentah.....	302
9.15 Scheduling dengan Cron.....	303
9.16 Audit Trail.....	304
9.17 Reproducibility.....	305
9.18 Membuat README Proyek.....	306
Setup.....	307
Jalankan Database.....	307
Jalankan Pipeline.....	307
Query PostgreSQL.....	307
Query MongoDB.....	307
Output.....	307
Etika.....	307
9.20 Struktur Akhir Proyek Bab 9.....	309
9.21 Checklist Praktik Bab 9.....	310
9.22 Kesalahan Umum Bab 9.....	311
1. Semua kode ditaruh di satu blok tanpa fungsi.....	311
2. Tidak menyimpan raw data.....	311
3. Tidak mencatat run ID.....	311
4. Tidak menyimpan nama model.....	311
5. Tidak ada logging.....	312
6. Tidak menangani duplikasi.....	312
7. README tidak jelas.....	312
9.23 Ringkasan Bab.....	313
Bab 10 — Etika, Keamanan, Portofolio, dan Masa Depan Pipeline Data AI.....	314
10.1 Mengapa Etika Menjadi Bab Penutup?.....	314
10.2 Prinsip Responsible Scraping.....	315
10.3 Menghormati robots.txt, Rate Limit, dan Terms of Service.....	316
1. Cek robots.txt.....	316
2. Cek terms of service.....	316
3. Gunakan rate limit.....	316
4. Jangan scraping area sensitif.....	316
10.4 Jangan Mengambil Data Pribadi Tanpa Dasar yang Sah.....	317
10.5 Keamanan Credential Database dan API.....	318

10.6 Struktur Repository Portofolio yang Profesional.....	319
10.7 Dokumentasi Proyek.....	320
README.....	320
Architecture.....	320
Project Report.....	320
10.8 Cara Menjelaskan Proyek Saat Wawancara.....	321
10.9 Tren 2026: Masa Depan Pipeline Data AI.....	322
1. AI-ready crawling.....	322
2. RAG pipeline.....	322
3. Agentic data pipeline.....	322
4. Local LLM.....	322
5. Data governance.....	322
Bagian Praktik.....	323
10.10 Praktik 1 — Checklist Legal dan Etika Scraping.....	323
10.11 Praktik 2 — Menambahkan .gitignore.....	325
10.12 Praktik 3 — Menyimpan Credential di .env.....	326
10.13 Praktik 4 — Membuat README Final.....	327
Tools.....	328
Setup.....	328
Jalankan Database.....	328
Jalankan Pipeline.....	328
Query.....	328
Output.....	328
Etika.....	328
Batasan.....	329
Lisensi.....	329
Audit Trail.....	330
10.16 Praktik 7 — Menulis Laporan Singkat Proyek Akhir.....	333
10.17 Praktik 8 — Cek Repository Sebelum Dipublikasikan.....	335
10.18 Praktik 9 — Cara Menjelaskan Project dalam 60 Detik.....	336
10.19 Praktik 10 — Final Audit Command.....	337
10.20 Struktur Akhir Repository Siap Publik.....	339
10.21 Checklist Bab 10.....	340
10.22 Kesalahan Umum Bab 10.....	341
1. Menganggap data publik selalu bebas diambil.....	341
2. Mengunggah .env.....	341
3. README terlalu pendek.....	341
4. Tidak punya laporan proyek.....	341
5. Tidak menyebut batasan proyek.....	342
6. Tidak bisa menjelaskan proyek.....	342
10.23 Ringkasan Bab.....	343
Lampiran A — Instalasi Tools Open Source.....	344
A.1 Python, pip, dan Virtual Environment.....	344
A.2 Docker dan Docker Compose.....	345

A.3 PostgreSQL.....	345
A.4 MongoDB.....	346
A.5 SQLite.....	346
A.6 Ollama.....	347
A.7 Playwright.....	347
A.8 Scrapy.....	348
A.9 Crawl4AI.....	348
Lampiran B — Template Struktur Folder Proyek.....	349
B.1 Fungsi Folder.....	349
B.2 Template .env.example.....	350
B.3 Template .gitignore.....	350
Lampiran C — Template Schema Database.....	351
C.1 Tabel raw_pages.....	351
C.2 Tabel scraped_items.....	352
C.3 Tabel clean_items.....	353
C.4 Tabel llm_extractions.....	354
C.5 Tabel pipeline_logs.....	355
C.6 Field Penting.....	356
Lampiran D — 30 Prompt LLM untuk Data Extraction.....	357
D.1 Ekstraksi Judul dan Ringkasan.....	357
D.2 Ekstraksi Entitas.....	358
D.3 Ekstraksi Topik.....	359
D.4 Klasifikasi Artikel.....	360
D.5 Konversi Teks ke JSON.....	361
D.6 Validasi JSON.....	363
D.7 Deteksi Data Kosong.....	364
D.8 Deteksi Informasi Sensitif.....	365
D.9 Ringkasan Eksekutif.....	366
D.10 Pembuatan Metadata.....	367
Lampiran E — Checklist Responsible Scraping.....	368
E.1 Legal dan Aturan Situs.....	368
E.2 Etika Teknis.....	368
E.3 Audit Data.....	368
E.4 Keamanan.....	368
E.5 LLM.....	369
E.6 Keputusan Akhir.....	369
Lampiran F — Troubleshooting Praktis.....	370
F.1 Error 403 — Forbidden.....	370
F.2 Error 429 — Too Many Requests.....	371
F.3 Selector Berubah.....	372
F.4 Data Kosong.....	373
F.5 Encoding Rusak.....	374
F.6 Docker Database Gagal Start.....	375
F.7 Koneksi PostgreSQL Gagal.....	376

F.8 MongoDB Tidak Bisa Connect.....	377
F.9 Output LLM Bukan JSON.....	378
F.10 Playwright Timeout.....	379
F.11 Ringkasan Troubleshooting Cepat.....	380
Glossary.....	381
Referensi.....	386
Tentang Penulis.....	388

Bab 1 — Web Scraping, Database, dan LLM: Skill Data Modern untuk Mahasiswa

Data yang baik bukan sekadar berhasil diambil, tetapi harus legal, bersih, terstruktur, dapat diaudit, dan siap dipakai untuk keputusan.

Bab 1 sebagai fondasi awal: mahasiswa perlu memahami hubungan antara *web scraping*, database, dan *open source LLM* sebagai satu alur kerja data modern, bukan sebagai keterampilan yang berdiri sendiri-sendiri.

1.1 Mengapa Mahasiswa Perlu Skill Pipeline Data Modern

Mahasiswa sering belajar Python, database, dan AI secara terpisah. Python dipakai untuk menulis kode. Database dipakai untuk menyimpan data. AI dipakai untuk menjawab pertanyaan atau membuat ringkasan. Masalahnya, dunia nyata tidak bekerja sesederhana itu.

Dalam riset, bisnis, jurnalisme data, otomasi, dan pengembangan AI, data harus melewati proses lengkap:

Web → Scraper → Cleaner → Validator → Database → LLM → Dashboard/API

Artinya, mahasiswa tidak cukup hanya bisa menulis script. Mahasiswa perlu bisa membangun **pipeline data** yang utuh:

- **Mengambil data** dari sumber yang legal.
- **Membersihkan data** agar tidak kacau.
- **Memvalidasi data** agar formatnya konsisten.
- **Menyimpan data** ke database agar mudah dicari.
- **Menggunakan LLM lokal** untuk ekstraksi, klasifikasi, ringkasan, dan validasi.
- **Membuat output** yang bisa dipakai untuk analisis, laporan, aplikasi, atau portofolio.

Skill modern bukan hanya coding. Skill modern adalah kemampuan mengubah data mentah menjadi pengetahuan yang bisa dipertanggungjawabkan.

1.2 Data Web sebagai Bahan Baku Riset, Bisnis, dan AI

Web adalah salah satu sumber data terbesar. Di dalamnya ada artikel, katalog produk, pengumuman kampus, data publik, berita, dokumen, forum, repositori, dan halaman organisasi. Bagi mahasiswa, data web dapat menjadi bahan untuk:

- **Riset akademik**, misalnya analisis tren topik berita.
- **Jurnalisme data**, misalnya mengamati perubahan nilai atau isu publik.
- **Bisnis digital**, misalnya analisis produk, ulasan, atau kompetitor.
- **AI dan NLP**, misalnya membuat dataset teks untuk klasifikasi.
- **Portofolio data engineering**, misalnya membangun pipeline dari web ke database.
- **Otomasi laporan**, misalnya mengambil data berkala lalu membuat ringkasan.

Namun, data web tidak otomatis siap pakai. Data web sering kotor, tidak konsisten, berubah struktur, memiliki duplikasi, atau mengandung informasi yang tidak boleh diambil.

Karena itu, mahasiswa perlu memahami prinsip penting:

Web scraping bukan sekadar mengambil data. Web scraping adalah proses rekayasa data yang harus etis, rapi, dan bisa diaudit.

1.3 Perbedaan Web Scraping, Crawling, API Access, dan Data Ingestion

Empat istilah ini sering tercampur. Padahal masing-masing berbeda.

1. Web Scraping

Web scraping adalah proses mengambil data dari halaman web. Biasanya yang diambil adalah teks, link, tabel, judul, tanggal, nilai, atau isi artikel.

Contoh sederhana:

- Ambil judul artikel dari halaman berita publik.
- Ambil daftar produk dari halaman katalog publik.
- Ambil tabel data dari halaman statistik publik.

Fokusnya adalah **ekstraksi data dari halaman web**.

2. Crawling

Crawling adalah proses menjelajahi banyak halaman secara otomatis. Biasanya dimulai dari satu URL, lalu program mengikuti link ke halaman lain.

Contoh:

- Ambil semua artikel dari halaman indeks.
- Ikuti tombol “next page”.
- Ambil data dari banyak halaman kategori.

Scrapy, misalnya, didefinisikan dalam dokumentasi resminya sebagai framework tingkat tinggi untuk *web crawling* dan *web scraping* yang digunakan untuk mengambil data terstruktur dari halaman web. ([Scrapy](#))

3. API Access

API access adalah pengambilan data melalui jalur resmi yang disediakan aplikasi atau organisasi. Biasanya output-nya sudah berbentuk JSON atau format terstruktur lain.

Contoh:

- Mengambil data cuaca dari API publik.
- Mengambil data katalog dari API resmi.
- Mengambil data statistik dari endpoint terbuka.

Jika API resmi tersedia, **API biasanya lebih baik daripada scraping HTML**, karena lebih stabil, lebih jelas formatnya, dan lebih mudah diproses.

4. Data Ingestion

Data ingestion adalah proses memasukkan data dari berbagai sumber ke sistem penyimpanan atau pipeline.

Sumbernya bisa berupa:

- Halaman web.
- API.
- CSV.
- JSON.
- Database lama.
- File log.
- Dokumen teks.

Dengan kata lain, *scraping* dan *API access* adalah cara mengambil data, sedangkan *data ingestion* adalah proses memasukkan data itu ke pipeline yang lebih besar.

1.4 Mengapa Database Penting dalam Pipeline Scraping

Banyak pemula berhenti di file CSV. Untuk latihan awal, CSV cukup. Tetapi untuk proyek yang lebih serius, data perlu disimpan ke database.

Database membantu mahasiswa menjaga data agar:

- **Rapi**, karena setiap field punya tempat.
- **Konsisten**, karena format data bisa dikontrol.
- **Mudah dicari**, karena bisa memakai query.
- **Tidak mudah duplikat**, karena bisa memakai *primary key* atau *unique constraint*.
- **Bisa diaudit**, karena asal data dan waktu scraping bisa disimpan.
- **Siap dianalisis**, karena data tersusun dalam tabel atau dokumen.

Contoh field penting dalam database scraping:

- `id`
- `source_url`
- `title`
- `raw_text`
- `clean_text`
- `content_hash`
- `scraped_at`
- `validation_status`

Database bukan hanya tempat menyimpan data. Database adalah **alat kontrol kualitas**.

Tanpa database, data mudah tercecer. Dengan database, pipeline menjadi lebih profesional.

1.5 Peran Open Source LLM dalam Pipeline Data

Large Language Model atau LLM dapat membantu membuat data web lebih bermakna. Tetapi LLM bukan pengganti scraper.

Scraper mengambil data. LLM menafsirkan data.

LLM lokal dapat digunakan untuk:

- **Meringkas artikel.**
- **Mengekstrak entitas**, seperti nama orang, organisasi, lokasi, topik.
- **Mengklasifikasi teks**, misalnya berita ekonomi, teknologi, pendidikan.
- **Mengubah teks menjadi JSON.**
- **Membantu validasi kualitas teks.**
- **Membuat metadata** untuk dataset.
- **Membantu membaca data mentah yang panjang.**

Namun, LLM memiliki risiko. Output LLM bisa salah, tidak konsisten, atau menghasilkan informasi yang tidak ada di teks asli. Karena itu, hasil LLM harus divalidasi dengan schema, dicek ulang, dan disimpan bersama sumber aslinya.

Prinsipnya sederhana:

LLM boleh membantu membaca data, tetapi sumber data asli tetap harus disimpan.

1.6 Risiko Web Scraping yang Wajib Dipahami

Mahasiswa perlu paham bahwa scraping bukan aktivitas bebas tanpa batas. Ada resiko teknis, etika, dan legal.

Risiko utama

- **Legalitas**
Tidak semua halaman boleh diambil datanya. Cek aturan situs, lisensi, dan batas penggunaan.
- **Privasi**
Jangan mengambil data pribadi tanpa dasar yang sah. Hindari data sensitif.
- **Terms of Service**
Beberapa situs melarang scraping dalam ketentuan layanan.
- **Beban server**
Scraper yang terlalu agresif bisa membebani server orang lain.
- **Kualitas data**
Data web bisa salah, duplikat, tidak lengkap, atau berubah format.
- **Keamanan credential**
Jangan menaruh password, token, atau konfigurasi rahasia langsung di kode.
- **Halusinasi LLM**
LLM dapat membuat ringkasan atau struktur JSON yang tampak benar tetapi tidak sesuai sumber.

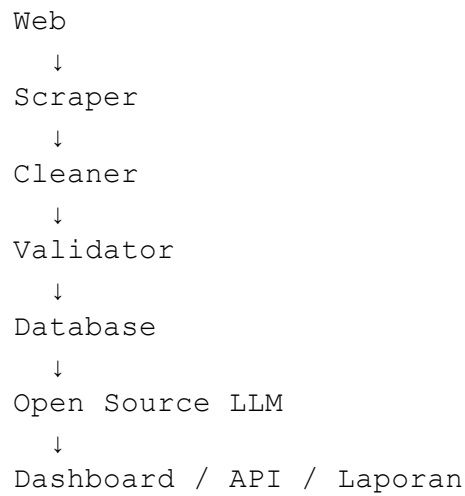
Prinsip aman

- Gunakan data publik yang memang aman untuk latihan.
- Mulai dari jumlah request kecil.
- Beri jeda antar request.
- Simpan URL sumber.
- Simpan waktu pengambilan data.
- Jangan mengambil data login, data privat, atau data yang dibatasi.
- Validasi hasil sebelum masuk database.

Scraping yang baik bukan yang paling banyak mengambil data, tetapi yang paling bertanggung jawab.

1.7 Gambaran Pipeline Buku

Buku ini memakai satu alur besar yang berkembang dari bab ke bab.



Penjelasan tiap tahap:

- **Web**
Sumber data publik yang akan diambil.
- **Scraper**
Script Python untuk mengambil halaman atau data.
- **Cleaner**
Proses membersihkan teks, spasi, karakter aneh, HTML sisa, dan data kosong.
- **Validator**
Proses memastikan data punya format yang benar.
- **Database**
Tempat menyimpan data mentah dan data bersih.
- **Open Source LLM**
Alat untuk ringkasan, ekstraksi, klasifikasi, dan konversi teks ke JSON.
- **Dashboard/API/Laporan**
Output akhir yang bisa dipakai manusia atau aplikasi.

Target akhir buku adalah mahasiswa memiliki proyek portofolio:

Pipeline Data Artikel Publik dengan Open Source LLM

Bagian Praktik

1.8 Instalasi Python

Python adalah bahasa utama dalam buku ini. Kita memakai Python karena ekosistemnya kuat untuk scraping, cleaning, validasi, database, dan integrasi LLM.

Di sistem GNU/Linux berbasis Debian/Ubuntu, cek Python dengan:

```
python3 --version
```

Penjelasan:

- `python3` memanggil interpreter Python versi 3.
- `--version` menampilkan versi Python yang terpasang.

Jika Python belum tersedia, instal dengan:

```
sudo apt update  
sudo apt install python3 python3-pip python3-venv
```

Penjelasan:

- `sudo` menjalankan perintah dengan hak administrator.
- `apt update` memperbarui daftar paket.
- `apt install` memasang paket.
- `python3` adalah interpreter Python.
- `python3-pip` adalah pengelola paket Python.
- `python3-venv` dipakai untuk membuat *virtual environment*.

Modul `venv` adalah bagian resmi Python untuk membuat lingkungan virtual yang ringan dan terisolasi dari paket Python global. ([Python documentation](#))

1.9 Membuat Folder Proyek

Buat folder proyek utama:

```
mkdir webscraping-llm-database
cd webscraping-llm-database
```

Penjelasan:

- **mkdir** membuat folder baru.
- **webscraping-llm-database** adalah nama folder proyek.
- **cd** masuk ke folder tersebut.

Buat struktur folder awal:

```
mkdir data scripts logs database
mkdir data/raw data/clean data/rejected
```

Struktur awal:

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   ├── clean/
│   └── rejected/
├── scripts/
├── logs/
└── database/
```

Fungsi folder:

- **data/raw/** untuk data mentah.
- **data/clean/** untuk data bersih.
- **data/rejected/** untuk data yang gagal validasi.
- **scripts/** untuk kode Python.
- **logs/** untuk catatan proses.
- **database/** untuk schema dan script database.

Folder yang rapi adalah awal dari pipeline yang bisa dipercaya.

1.10 Membuat Virtual Environment

Buat lingkungan virtual:

```
python3 -m venv .venv
```

Penjelasan:

- `python3` menjalankan Python.
- `-m venv` menjalankan modul `venv`.
- `.venv` adalah nama folder lingkungan virtual.

Aktifkan:

```
source .venv/bin/activate
```

Penjelasan:

- `source` menjalankan script di shell aktif.
- `.venv/bin/activate` mengaktifkan lingkungan virtual.

Jika berhasil, biasanya terminal menampilkan tanda seperti:

```
(.venv)
```

Perbarui pengelola paket:

```
python -m pip install --upgrade pip
```

Penjelasan:

- `python -m pip` menjalankan pip dari Python aktif.
- `install --upgrade pip` memperbarui pip.

Panduan resmi Python Packaging juga menyarankan penggunaan `venv` dan `pip` untuk membuat lingkungan proyek serta memasang paket secara terisolasi.

([Panduan Pengemasan Python](#))

1.11 Instalasi Paket Awal

Untuk Bab 1, kita pasang paket dasar:

```
pip install python-dotenv
```

Penjelasan:

- `pip install` memasang paket Python.
- `python-dotenv` membaca file `.env` dan menjadikannya konfigurasi environment.

`python-dotenv` secara resmi dijelaskan sebagai paket yang membaca pasangan key-value dari file `.env` dan dapat mengaturnya sebagai environment variable.

([PyPI](#))

Buat file daftar paket:

```
pip freeze > requirements.txt
```

Penjelasan:

- `pip freeze` menampilkan paket yang terpasang.
- `>` mengarahkan output ke file.
- `requirements.txt` menyimpan daftar paket proyek.

Nanti, orang lain bisa memasang paket yang sama dengan:

```
pip install -r requirements.txt
```

Penjelasan:

- `-r` berarti membaca daftar paket dari file.
- `requirements.txt` adalah file daftar dependency.

1.12 Instalasi Editor Kode Open Source

Untuk menulis kode, gunakan **VSCodium** atau editor open source lain yang nyaman. VSCodium menyediakan build open source dari editor kode populer dan dapat dipasang melalui beberapa jalur paket di GNU/Linux. ([VSCodium](#))

Pada tahap ini, mahasiswa hanya perlu memastikan editor mampu:

- Membuka folder proyek.
- Menulis file Python.
- Membuka terminal.
- Menampilkan struktur folder.
- Membantu membaca error.

Buka folder proyek dari editor, lalu pastikan struktur folder terlihat rapi.

1.13 Membuat File .env

File .env dipakai untuk menyimpan konfigurasi sederhana. Contoh:

```
nano .env
```

Isi file:

```
PROJECT_NAME=webscraping-llm-database
ENVIRONMENT=development
RAW_DATA_DIR=data/raw
CLEAN_DATA_DIR=data/clean
LOG_DIR=logs
```

Penjelasan isi:

- **PROJECT_NAME** menyimpan nama proyek.
- **ENVIRONMENT** menandai mode kerja.
- **RAW_DATA_DIR** menunjukkan folder data mentah.
- **CLEAN_DATA_DIR** menunjukkan folder data bersih.
- **LOG_DIR** menunjukkan folder log.

Catatan penting:

Jangan menyimpan password asli, token, atau credential sensitif di file yang akan dipublikasikan.

Nanti pada bab berikutnya, file **.env** akan dipakai untuk konfigurasi database, model lokal, dan pipeline.

1.14 Script Pertama: “Hello Pipeline”

Buat file:

```
nano scripts/hello_pipeline.py
```

Isi:

```
from dotenv import load_dotenv
import os
from datetime import datetime

load_dotenv()

project_name = os.getenv("PROJECT_NAME",
"pipeline-data")
environment = os.getenv("ENVIRONMENT", "development")
raw_data_dir = os.getenv("RAW_DATA_DIR", "data/raw")

print("Hello Pipeline!")
print(f"Project      : {project_name}")
print(f"Environment  : {environment}")
print(f"Raw Data Dir: {raw_data_dir}")
print(f"Started At   : {datetime.now().isoformat()}")
```

Jalankan:

```
python scripts/hello_pipeline.py
```

Output yang diharapkan:

```
Hello Pipeline!
Project      : webscraping-llm-database
Environment  : development
Raw Data Dir: data/raw
Started At   : 2026-05-27T...
```

1.15 Penjelasan Script per Bagian

```
from dotenv import load_dotenv
```

Baris ini mengambil fungsi `load_dotenv` dari paket `python-dotenv`.

```
import os
```

Baris ini memanggil modul `os` untuk membaca environment variable.

```
from datetime import datetime
```

Baris ini mengambil fungsi waktu dari modul `datetime`.

```
load_dotenv()
```

Baris ini membaca file `.env`.

```
project_name = os.getenv("PROJECT_NAME", "pipeline-data")
```

Baris ini membaca nilai `PROJECT_NAME`. Jika tidak ada, nilai default-nya adalah `pipeline-data`.

```
environment = os.getenv("ENVIRONMENT", "development")
```

Baris ini membaca mode kerja proyek.

```
raw_data_dir = os.getenv("RAW_DATA_DIR", "data/raw")
```

Baris ini membaca lokasi folder data mentah.

```
print("Hello Pipeline!")
```

Baris ini menampilkan pesan awal.

```
datetime.now().isoformat()
```

Baris ini membuat timestamp dalam format yang mudah dibaca mesin dan manusia.

1.16 Checklist Praktik Bab 1

Mahasiswa dianggap selesai Bab 1 jika sudah memiliki:

- Folder proyek `webscraping-llm-database/`.
- Folder `data/raw/`, `data/clean/`, `data/rejected/`, `scripts/`, `logs/`, dan `database/`.
- Lingkungan virtual `.venv`.
- File `requirements.txt`.
- File `.env`.
- Script `scripts/hello_pipeline.py`.
- Output terminal yang menunjukkan konfigurasi berhasil dibaca.

Checklist singkat:

- ☐ Python berjalan
- ☐ virtual environment aktif
- ☐ paket python-dotenv terpasang
- ☐ struktur folder dibuat
- ☐ file `.env` dibuat
- ☐ script `hello_pipeline.py` berjalan
- ☐ `requirements.txt` dibuat

1.17 Kesalahan Umum di Bab 1

1. Virtual environment belum aktif

Gejala:

```
ModuleNotFoundError: No module named 'dotenv'
```

Solusi:

```
source .venv/bin/activate  
pip install python-dotenv
```

2. File .env salah lokasi

Gejala:

```
Project : pipeline-data
```

Padahal seharusnya membaca nama proyek dari .env.

Solusi:

- Pastikan .env berada di folder utama proyek.
- Jalankan script dari folder utama proyek.

3. Salah menjalankan path script

Gejala:

```
python: can't open file
```

Solusi:

```
python scripts/hello_pipeline.py
```

4. Paket belum tercatat

Solusi:

```
pip freeze > requirements.txt
```

1.18 Ringkasan Bab

Bab ini memperkenalkan fondasi pipeline data modern untuk mahasiswa. Mahasiswa belajar bahwa Python, database, dan LLM tidak boleh dipahami secara terpisah. Ketiganya harus disatukan dalam pipeline yang utuh, mulai dari pengambilan data web, pembersihan, validasi, penyimpanan, sampai pemanfaatan LLM untuk membuat data lebih bermakna.

Bab ini juga menekankan bahwa scraping harus dilakukan secara bertanggung jawab. Data harus legal, bersih, terstruktur, dan bisa diaudit. Praktik awal difokuskan pada persiapan proyek: instalasi Python, pembuatan *virtual environment*, struktur folder, file `.env`, dan script “Hello Pipeline”.

Output akhir Bab 1: mahasiswa memiliki fondasi proyek yang siap dikembangkan pada bab berikutnya menjadi pipeline scraping, cleaning, database, dan LLM yang lengkap.

Bab 2 — Dasar HTTP, HTML, dan Parsing Web dengan Python

Jangan mulai scraping sebelum paham apa yang dikirim, apa yang diterima, dan bagian HTML mana yang benar-benar ingin diambil.

Bab 2 difokuskan untuk membangun fondasi HTTP, HTML, *request*, *response*, *status code*, *header*, *selector*, dan parsing dengan Python sebelum mahasiswa masuk ke scraping yang lebih serius.

2.1 Mengapa HTTP dan HTML Wajib Dipahami

Banyak pemula langsung mencari kode scraping, lalu menyalin script dari internet. Cara ini cepat, tetapi rapuh. Begitu struktur halaman berubah, script rusak. Begitu server menolak request, mahasiswa bingung. Begitu data kosong, tidak tahu penyebabnya.

Karena itu, Bab 2 dimulai dari fondasi.

Saat Python mengambil halaman web, sebenarnya Python sedang melakukan komunikasi melalui **HTTP**. Program mengirim *request* ke server. Server mengirim *response* kembali. Isi *response* bisa berupa HTML, JSON, gambar, file, atau pesan error.

Untuk scraping dasar, yang paling sering diambil adalah **HTML**. HTML adalah struktur dokumen web yang berisi tag, atribut, teks, link, tabel, form, dan elemen lain. MDN Web Docs menjelaskan bahwa elemen HTML dibuat menggunakan tag dan dikelompokkan berdasarkan fungsi dalam dokumen web. ([MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/HTML))

Mahasiswa harus paham tiga hal:

- **HTTP** menjelaskan cara komunikasi.
- **HTML** menjelaskan struktur halaman.
- **Parser** membantu Python membaca HTML dan mengambil bagian tertentu.

Scraping yang baik dimulai dari membaca struktur, bukan menebak-nebak kode.

2.2 Cara Kerja HTTP Request dan Response

HTTP bekerja seperti percakapan sederhana.

```
Client / Python Script → HTTP Request → Web Server
Client / Python Script ← HTTP Response ← Web Server
```

Contoh:

1. Python meminta halaman:

```
GET https://example.org
```

2. Server membalas:

```
Status: 200 OK
Content-Type: text/html
Body: <html>...</html>
```

Dalam praktik scraping, mahasiswa perlu membaca:

- URL tujuan.
- Metode request.
- Status code.
- Header response.
- Isi response.
- Encoding teks.
- Struktur HTML.

Paket Python `requests` banyak digunakan karena memudahkan pengiriman HTTP request. Dokumentasi resminya menjelaskan bahwa `requests.get()` mengirim request dan mengembalikan objek `Response`, lalu isi response dapat dibaca melalui `r.text`, `r.content`, `r.status_code`, dan informasi lain. ([Requests Documentation](#))

2.3 Metode HTTP yang Paling Sering Ditemui

Untuk Bab 2, cukup pahami metode dasar berikut.

1. GET

Digunakan untuk mengambil data.

Contoh:

```
GET /artikel
```

Dalam scraping, metode ini paling sering dipakai.

2. POST

Digunakan untuk mengirim data ke server.

Contoh:

```
POST /search
```

Biasanya dipakai untuk form pencarian, login, atau pengiriman data. Untuk pemula, hindari scraping area login atau data privat.

3. HEAD

Digunakan untuk mengambil header tanpa body. Berguna untuk mengecek metadata.

4. OPTIONS

Kadang dipakai untuk mengetahui metode yang didukung server.

Untuk scraping awal, fokus cukup pada **GET request**.

2.4 Status Code yang Harus Dipahami

Status code adalah kode balasan server. Kode ini memberi tahu apakah request berhasil, dialihkan, ditolak, atau gagal.

200 — OK

Artinya request berhasil.

```
200 OK
```

Biasanya halaman bisa diproses.

301 — Moved Permanently

Artinya halaman dipindahkan secara permanen ke URL lain.

```
301 Moved Permanently
```

Scraper perlu mengikuti redirect jika diperlukan.

403 — Forbidden

Artinya server menolak akses.

```
403 Forbidden
```

Penyebab umum:

- Server tidak mengizinkan akses otomatis.
- Butuh izin tertentu.
- Header request dianggap tidak sesuai.
- Akses ke halaman memang dibatasi.

404 — Not Found

Artinya halaman tidak ditemukan.

```
404 Not Found
```

Penyebab umum:

- URL salah.
- Halaman dihapus.
- Link lama.

429 — Too Many Requests

Artinya request terlalu banyak.

```
429 Too Many Requests
```

Ini penting secara etika. Jika mendapat 429, scraper harus berhenti atau memperlambat request.

500 — Internal Server Error

Artinya server mengalami error.

```
500 Internal Server Error
```

Jangan langsung mengulang request agresif. Bisa jadi server sedang bermasalah.

MDN mendokumentasikan HTTP response status code sebagai kode yang menunjukkan apakah request HTTP berhasil, dialihkan, gagal karena kesalahan client, atau gagal karena kesalahan server. ([MDN Web Docs](#))

Mahasiswa harus membaca status code sebelum parsing HTML. Jangan memproses halaman yang sebenarnya gagal diambil.

2.5 Header, User-Agent, Content-Type, dan Encoding

HTTP tidak hanya berisi body. Ada juga **header**. Header adalah metadata komunikasi antara client dan server.

Contoh header penting:

```
User-Agent: Mozilla/5.0 ...  
Content-Type: text/html; charset=utf-8  
Accept-Language: id,en
```

User-Agent

User-Agent memberi tahu server jenis client yang mengakses. Dalam scraping etis, jangan memakai identitas palsu untuk menyamar secara agresif. Gunakan header yang wajar dan jangan melakukan bypass proteksi.

Content-Type

`Content-Type` memberi tahu jenis isi response.

Contoh:

```
text/html  
application/json  
text/csv
```

Jika `Content-Type` adalah `text/html`, data biasanya perlu diparsing sebagai HTML.

Jika `Content-Type` adalah `application/json`, lebih baik diproses sebagai JSON.

Encoding

Encoding menentukan cara byte diubah menjadi teks.

Contoh:

```
utf-8  
ISO-8859-1
```

Dokumentasi `requests` menjelaskan bahwa Requests membuat tebakan encoding berdasarkan HTTP headers, dan encoding tersebut dipakai saat mengakses `r.text`. Encoding juga dapat dilihat atau diubah melalui `r.encoding`. ([Requests Documentation](#))

Jika encoding salah, teks bisa terlihat rusak.

Contoh masalah:

```
Pendidikan menjadi Pendidikanâ€™™
```

Solusinya adalah memeriksa encoding dan membersihkan teks pada tahap berikutnya.

2.6 Struktur HTML: Tag, Class, ID, Link, Table, dan Form

HTML adalah struktur dokumen. Untuk scraping, mahasiswa perlu memahami bagian-bagian dasar berikut.

Tag

Tag adalah penanda elemen HTML.

Contoh:

```
<h1>Judul Artikel</h1>
<p>Isi paragraf.</p>
<a href="https://example.org">Link</a>
```

Tag umum:

- `<html>`: akar dokumen HTML.
- `<head>`: metadata halaman.
- `<body>`: isi utama halaman.
- `<h1>` sampai `<h6>`: judul.
- `<p>`: paragraf.
- `<a>`: link.
- `<table>`: tabel.
- `<tr>`: baris tabel.
- `<td>`: sel tabel.
- `<form>`: form input.

MDN menjelaskan bahwa elemen `<table>` merepresentasikan data berdimensi lebih dari satu dalam bentuk tabel, sedangkan `<form>` merepresentasikan bagian dokumen yang berisi kontrol interaktif untuk mengirim informasi. ([HTML Living Standard](#))

Class

`class` dipakai untuk memberi kategori elemen.

```
<p class="summary">Ringkasan artikel.</p>
```

Dalam scraping, class sering dipakai sebagai selector.

ID

`id` adalah identitas unik elemen.

```
<div id="content">Isi utama</div>
```

Jika stabil, `id` sangat berguna untuk scraping.

Link

Link memakai tag `<a>` dan atribut `href`.

```
<a href="/artikel/1">Baca artikel</a>
```

Dalam crawling, link dipakai untuk menemukan halaman berikutnya.

Table

Tabel dipakai untuk data yang sudah terstruktur.

```
<table>
  <tr>
    <th>Nama</th>
    <th>Nilai</th>
  </tr>
  <tr>
    <td>Ani</td>
    <td>90</td>
  </tr>
</table>
```

Form

Form dipakai untuk input pengguna.

```
<form action="/search" method="get">
  <input name="q">
</form>
```

Untuk Bab 2, form hanya dikenalkan. Praktik scraping form akan dilakukan hati-hati dan hanya pada halaman latihan yang legal.

2.7 HTML Statis dan Konten Dinamis

Tidak semua halaman web sama.

HTML Statis

HTML statis berarti data sudah ada di response awal.

Contoh:

```
<h1>Judul Berita</h1>
<p>Isi berita...</p>
```

Halaman seperti ini cocok untuk:

- `requests`
- `BeautifulSoup`
- parsing sederhana

Konten Dinamis

Konten dinamis sering dimuat setelah halaman dibuka. Data dapat muncul melalui script di sisi browser.

Ciri-cirinya:

- `requests` berhasil mengambil halaman, tetapi data tidak ada.
- Saat dilihat di browser, data muncul.
- HTML awal hanya berisi kerangka.
- Data muncul setelah proses JavaScript.

Untuk Bab 2, fokus kita adalah HTML statis. Website dinamis akan dibahas pada Bab 8 sesuai outline buku.

2.8 Dasar Selector: CSS Selector dan XPath

Selector adalah cara memilih bagian HTML.

CSS Selector

CSS selector mudah dipahami dan cukup untuk banyak kasus.

Contoh:

```
h1
.article-title
#content
a
table tr
```

Makna:

- `h1` memilih semua tag `<h1>`.
- `.article-title` memilih elemen dengan class `article-title`.
- `#content` memilih elemen dengan id `content`.
- `a` memilih semua link.
- `table tr` memilih baris dalam tabel.

Beautiful Soup mendukung navigasi, pencarian, dan modifikasi parse tree HTML/XML. Dokumentasi resminya menjelaskan bahwa Beautiful Soup membantu mengambil data dari file HTML dan XML, serta bekerja dengan parser untuk mencari struktur dokumen. ([Crummy](#))

XPath

XPath adalah cara memilih elemen berdasarkan jalur struktur dokumen.

Contoh:

```
//h1
//a/@href
//table//tr
```

XPath sangat kuat, tetapi untuk Bab 2 mahasiswa cukup mengenalnya. Praktik utama memakai CSS selector karena lebih mudah dipahami.

2.9 Etika Dasar Scraping

Sebelum praktik, pahami batas etika.

Jangan lakukan scraping seperti menyerang server.

Prinsip dasar:

- Ambil hanya data publik yang aman untuk latihan.
- Jangan mengambil data pribadi.
- Jangan mengambil data login.
- Jangan membanjiri server.
- Gunakan jeda jika mengambil banyak halaman.
- Baca aturan situs jika tersedia.
- Simpan sumber URL.
- Hentikan scraping jika mendapat 403 atau 429.
- Jangan bypass proteksi.
- Jangan memakai scraping untuk tindakan merugikan.

Dalam buku ini, scraping diarahkan untuk **belajar pipeline data**, bukan mengambil data sembarangan.

Bagian Praktik

2.10 Persiapan Folder Bab 2

Masuk ke folder proyek dari Bab 1.

```
cd webscraping-llm-database
source .venv/bin/activate
```

Penjelasan:

- `cd` masuk ke folder proyek.
- `source .venv/bin/activate` mengaktifkan *virtual environment*.

Buat folder praktik Bab 2:

```
mkdir -p scripts/bab02
mkdir -p data/raw/bab02
mkdir -p data/clean/bab02
```

Penjelasan:

- `mkdir` membuat folder.
- `-p` membuat folder bertingkat dan tidak error jika folder sudah ada.
- `scripts/bab02` menyimpan script Bab 2.
- `data/raw/bab02` menyimpan data mentah.
- `data/clean/bab02` menyimpan data hasil parsing.

2.11 Instalasi Paket Praktik

Instal paket yang diperlukan:

```
pip install requests beautifulsoup4 lxml
```

Penjelasan:

- `requests` untuk mengambil halaman web.
- `beautifulsoup4` untuk parsing HTML.
- `lxml` sebagai parser HTML/XML yang cepat.

Simpan ke requirements.txt:

```
pip freeze > requirements.txt
```

Penjelasan:

- `pip freeze` mencatat paket terpasang.
- `>` menyimpan output ke file.
- `requirements.txt` membuat proyek mudah dijalankan ulang.

2.12 Praktik 1 — Mengambil Halaman Web dengan Requests

Buat file:

```
nano scripts/bab02/01_fetch_page.py
```

Isi:

```
import requests

url = "https://example.org"

response = requests.get(url, timeout=10)

print("URL          :", url)
print("Status Code :", response.status_code)
print("Content-Type:", response.headers.get("Content-Type"))
print("Encoding     :", response.encoding)
print("Length       :", len(response.text))
print()
print(response.text[:500])
```

Jalankan:

```
python scripts/bab02/01_fetch_page.py
```

Penjelasan kode:

```
import requests
    Mengambil library requests.
url = "https://example.org"
    Menyimpan URL target.
response = requests.get(url, timeout=10)
    Mengirim HTTP GET request ke URL.
```

- `requests.get()` mengambil halaman.
- `timeout=10` membatasi waktu tunggu maksimal 10 detik.
- `response` berisi balasan server.

```
response.status_code
    Membaca status code.
response.headers.get("Content-Type")
    Membaca jenis konten.
response.encoding
    Membaca encoding teks.
len(response.text)
    Menghitung panjang teks response.
response.text[:500]
    Menampilkan 500 karakter pertama.
```

Pelajaran penting: sebelum parsing, baca dulu status code, content type, encoding, dan panjang response.

2.13 Praktik 2 — Menambahkan Header Sederhana

Buat file:

```
nano scripts/bab02/02_fetch_with_headers.py
```

Isi:

```
import requests

url = "https://example.org"

headers = {
    "User-Agent": "Mozilla/5.0 (compatible;
StudentDataPipeline/1.0)",
    "Accept": "text/html",
    "Accept-Language": "id,en;q=0.8",
}

response = requests.get(url, headers=headers,
timeout=10)

print("Status Code :", response.status_code)
print("Content-Type:",
response.headers.get("Content-Type"))
print("Final URL   :", response.url)
```

Jalankan:

```
python scripts/bab02/02_fetch_with_headers.py
```

Penjelasan:

- **headers** adalah metadata request.
- **User-Agent** memberi identitas client.
- **Accept** memberi tahu jenis konten yang diharapkan.
- **Accept-Language** memberi preferensi bahasa.
- **response.url** menunjukkan URL final setelah redirect.

Catatan etika:

Header bukan alat untuk menipu server. Header dipakai agar request jelas dan wajar.

2.14 Praktik 3 — Menyimpan HTML Mentah

Buat file:

```
nano scripts/bab02/03_save_raw_html.py
```

Isi:

```
import requests
from pathlib import Path

url = "https://example.org"

output_file = Path("data/raw/bab02/example.html")

headers = {
    "User-Agent": "Mozilla/5.0 (compatible;
StudentDataPipeline/1.0) "
}

response = requests.get(url, headers=headers,
timeout=10)
response.raise_for_status()

output_file.write_text(response.text, encoding="utf-8")

print(f"HTML berhasil disimpan ke: {output_file}")
```

Jalankan:

```
python scripts/bab02/03_save_raw_html.py
```

Penjelasan:

```
from pathlib import Path
    Menggunakan Path untuk mengelola path file.
output_file = Path("data/raw/bab02/example.html")
    Menentukan lokasi file output.
response.raise_for_status()
    Menghentikan program jika status code menunjukkan error.
output_file.write_text(response.text, encoding="utf-8")
    Menyimpan HTML ke file dengan encoding UTF-8.
```

Simpan data mentah. Jangan langsung membuang sumber asli.

2.15 Praktik 4 — Parsing HTML dengan BeautifulSoup

Buat file:

```
nano scripts/bab02/04_parse_title.py
```

Isi:

```
from pathlib import Path
from bs4 import BeautifulSoup

input_file = Path("data/raw/bab02/example.html")

html = input_file.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

title_tag = soup.find("title")
h1_tag = soup.find("h1")

title = title_tag.get_text(strip=True) if title_tag else None
h1 = h1_tag.get_text(strip=True) if h1_tag else None

print("Title:", title)
print("H1    :", h1)
```

Jalankan:

```
python scripts/bab02/04_parse_title.py
```

Penjelasan:

```
from bs4 import BeautifulSoup
    Mengambil BeautifulSoup.
html = input_file.read_text(encoding="utf-8")
    Membaca file HTML.
soup = BeautifulSoup(html, "lxml")
    Membuat objek parser.
soup.find("title")
    Mencari tag <title> pertama.
soup.find("h1")
    Mencari tag <h1> pertama.
get_text(strip=True)
    Mengambil teks dan membersihkan spasi awal/akhir.
```

2.16 Praktik 5 — Mengambil Semua Link

Buat file:

```
nano scripts/bab02/05_extract_links.py
```

Isi:

```
from pathlib import Path
from urllib.parse import urljoin
from bs4 import BeautifulSoup

base_url = "https://example.org"
input_file = Path("data/raw/bab02/example.html")

html = input_file.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

links = []

for a_tag in soup.find_all("a"):
    text = a_tag.get_text(strip=True)
    href = a_tag.get("href")

    if not href:
        continue

    absolute_url = urljoin(base_url, href)

    links.append({
        "text": text,
        "url": absolute_url,
    })

for item in links:
    print(item)
```

Jalankan:

```
python scripts/bab02/05_extract_links.py
```

Penjelasan:

```
from urllib.parse import urljoin
    Mengubah link relatif menjadi link absolut.
soup.find_all("a")
    Mengambil semua tag <a>.
a_tag.get("href")
    Mengambil alamat link.
if not href:
    continue
    Melewati tag link yang tidak punya href.
urljoin(base_url, href)
    Menggabungkan base URL dengan link relatif.
```

2.17 Praktik 6 — Mengambil Paragraf

Buat file:

```
nano scripts/bab02/06_extract_paragraphs.py
```

Isi:

```
from pathlib import Path
from bs4 import BeautifulSoup

input_file = Path("data/raw/bab02/example.html")

html = input_file.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

paragraphs = []

for p_tag in soup.find_all("p"):
    text = p_tag.get_text(" ", strip=True)

    if text:
        paragraphs.append(text)

for number, paragraph in enumerate(paragraphs, start=1):
    print(f"{number}. {paragraph}")
```

Jalankan:

```
python scripts/bab02/06_extract_paragraphs.py
```

Penjelasan:

```
soup.find_all("p")
    Mengambil semua paragraf.
get_text(" ", strip=True)
    Mengambil teks, mengganti pemisah dengan spasi, dan
    membersihkan spasi pinggir.
if text:
    paragraphs.append(text)
    Hanya menyimpan paragraf yang tidak kosong.
```

2.18 Praktik 7 — Menyimpan Hasil ke JSON

Buat file:

```
nano scripts/bab02/07_save_json.py
```

Isi:

```
import json
from pathlib import Path
from urllib.parse import urljoin
from bs4 import BeautifulSoup

base_url = "https://example.org"
input_file = Path("data/raw/bab02/example.html")
output_file = Path("data/clean/bab02/data_raw.json")

html = input_file.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

title_tag = soup.find("title")
h1_tag = soup.find("h1")

data = {
    "source_url": base_url,
    "title": title_tag.get_text(strip=True) if title_tag
else None,
    "h1": h1_tag.get_text(strip=True) if h1_tag else
None,
    "links": [],
    "paragraphs": [],
}

for a_tag in soup.find_all("a"):
    href = a_tag.get("href")
    text = a_tag.get_text(strip=True)

    if href:
        data["links"].append({
            "text": text,
            "url": urljoin(base_url, href),
        })

for p_tag in soup.find_all("p"):
    text = p_tag.get_text(" ", strip=True)

    if text:
        data["paragraphs"].append(text)

output_file.write_text(
```

```
        json.dumps(data, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

    print(f"JSON berhasil disimpan ke: {output_file}")
```

Jalankan:

```
python scripts/bab02/07_save_json.py
```

Penjelasan:

```
import json
```

Memakai modul JSON bawaan Python.

```
data = {...}
```

Membuat struktur data dictionary.

```
json.dumps(data, indent=2, ensure_ascii=False)
```

Mengubah dictionary menjadi JSON.

- `indent=2` membuat JSON mudah dibaca.
- `ensure_ascii=False` menjaga karakter non-ASCII tetap terbaca.

2.19 Praktik 8 — Menyimpan Hasil ke CSV

Buat file:

```
nano scripts/bab02/08_save_csv.py
```

Isi:

```
import csv
from pathlib import Path
from urllib.parse import urljoin
from bs4 import BeautifulSoup

base_url = "https://example.org"
input_file = Path("data/raw/bab02/example.html")
output_file = Path("data/clean/bab02/data_raw.csv")

html = input_file.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

rows = []

for a_tag in soup.find_all("a"):
    href = a_tag.get("href")
    text = a_tag.get_text(strip=True)

    if href:
        rows.append({
            "type": "link",
            "text": text,
            "url": urljoin(base_url, href),
        })

for p_tag in soup.find_all("p"):
    text = p_tag.get_text(" ", strip=True)

    if text:
        rows.append({
            "type": "paragraph",
            "text": text,
            "url": base_url,
        })

with output_file.open("w", newline="", encoding="utf-8")
as file:
    writer = csv.DictWriter(file, fieldnames=["type",
"text", "url"])
    writer.writeheader()
    writer.writerows(rows)
```

```
print(f"CSV berhasil disimpan ke: {output_file}")
```

Jalankan:

```
python scripts/bab02/08_save_csv.py
```

Penjelasan:

```
import csv
    Menggunakan modul CSV bawaan Python.
rows = []
    Menyimpan data baris.
csv.DictWriter(...)
    Menulis dictionary ke CSV.
writer.writeheader()
    Menulis nama kolom.
writer.writerows(rows)
    Menulis semua baris data.
```

2.20 Praktik 9 — Membaca Tabel Sederhana dari HTML

Untuk latihan aman, buat file HTML lokal.

Buat file:

```
nano data/raw/bab02/sample_table.html
```

Isi:

```
<!DOCTYPE html>
<html>
<head>
  <title>Data Nilai Mahasiswa</title>
</head>
<body>
  <h1>Data Nilai Mahasiswa</h1>

  <table>
    <tr>
      <th>Nama</th>
      <th>Mata Kuliah</th>
      <th>Nilai</th>
    </tr>
    <tr>
      <td>Ani</td>
      <td>Data Engineering</td>
      <td>90</td>
    </tr>
    <tr>
      <td>Budi</td>
      <td>Web Scraping</td>
      <td>85</td>
    </tr>
  </table>
</body>
</html>
```

Buat parser tabel:

```
nano scripts/bab02/09_parse_table.py
```

Isi:

```
import csv
from pathlib import Path
from bs4 import BeautifulSoup

input_file = Path("data/raw/bab02/sample_table.html")
output_file = Path("data/clean/bab02/table_result.csv")

html = input_file.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

table = soup.find("table")

headers = []
rows = []

if table:
    header_cells = table.find_all("th")
    headers = [cell.get_text(strip=True) for cell in header_cells]

    for tr in table.find_all("tr")[1:]:
        cells = tr.find_all("td")
        row = [cell.get_text(strip=True) for cell in cells]

        if row:
            rows.append(row)

with output_file.open("w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(headers)
    writer.writerows(rows)

print(f"Tabel berhasil diekstrak ke: {output_file}")
```

Jalankan:

```
python scripts/bab02/09_parse_table.py
```

Output:

```
data/clean/bab02/table_result.csv
```

Pelajaran:

- Tabel HTML dapat diparsing menjadi CSV.
- Header tabel diambil dari `<th>`.
- Isi tabel diambil dari `<td>`.
- Baris tabel diambil dari `<tr>`.

2.21 Praktik 10 — Scraper Mini untuk Halaman Statis

Sekarang gabungkan konsep Bab 2 menjadi satu scraper mini.

Buat file:

```
nano scripts/bab02/10_mini_static_scraper.py
```

Isi:

```
import csv
import json
import time
from pathlib import Path
from urllib.parse import urljoin

import requests
from bs4 import BeautifulSoup

URL = "https://example.org"

RAW_HTML_FILE = Path("data/raw/bab02/mini_page.html")
JSON_FILE = Path("data/clean/bab02/data_raw.json")
CSV_FILE = Path("data/clean/bab02/data_raw.csv")

HEADERS = {
    "User-Agent": "Mozilla/5.0 (compatible;
StudentDataPipeline/1.0)",
    "Accept": "text/html",
}

def fetch_page(url):
    response = requests.get(url, headers=HEADERS,
timeout=10)
    response.raise_for_status()
    return response

def parse_page(html, source_url):
    soup = BeautifulSoup(html, "lxml")

    title_tag = soup.find("title")
    h1_tag = soup.find("h1")

    result = {
        "source_url": source_url,
        "title": title_tag.get_text(strip=True) if
title_tag else None,
        "h1": h1_tag.get_text(strip=True) if h1_tag else
None,
        "links": [],
```

```

        "paragraphs": [],
    }

    for a_tag in soup.find_all("a"):
        href = a_tag.get("href")
        text = a_tag.get_text(strip=True)

        if href:
            result["links"].append({
                "text": text,
                "url": urljoin(source_url, href),
            })

    for p_tag in soup.find_all("p"):
        text = p_tag.get_text(" ", strip=True)

        if text:
            result["paragraphs"].append(text)

    return result

def save_json(data, output_file):
    output_file.write_text(
        json.dumps(data, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

def save_csv(data, output_file):
    rows = []

    for link in data["links"]:
        rows.append({
            "type": "link",
            "text": link["text"],
            "url": link["url"],
        })

    for paragraph in data["paragraphs"]:
        rows.append({
            "type": "paragraph",
            "text": paragraph,
            "url": data["source_url"],
        })

    with output_file.open("w", newline="",
        encoding="utf-8") as file:
        writer = csv.DictWriter(file,
            fieldnames=["type", "text", "url"])

```

```

        writer.writeheader()
        writer.writerows(rows)

def main():
    print("Mengambil halaman...")
    response = fetch_page(URL)

    print("Status Code :", response.status_code)
    print("Content-Type:",
response.headers.get("Content-Type"))

    RAW_HTML_FILE.write_text(response.text,
encoding="utf-8")

    print("Parsing halaman...")
    data = parse_page(response.text, URL)

    print("Menyimpan JSON...")
    save_json(data, JSON_FILE)

    print("Menyimpan CSV...")
    save_csv(data, CSV_FILE)

    time.sleep(1)

    print("Selesai.")
    print(f"Raw HTML: {RAW_HTML_FILE}")
    print(f"JSON      : {JSON_FILE}")
    print(f"CSV       : {CSV_FILE}")

if __name__ == "__main__":
    main()

```

Jalankan:

```
python scripts/bab02/10_mini_static_scraper.py
```

Output yang diharapkan:

```

Mengambil halaman...
Status Code : 200
Content-Type: text/html
Parsing halaman...
Menyimpan JSON...
Menyimpan CSV...
Selesai.
Raw HTML: data/raw/bab02/mini_page.html
JSON      : data/clean/bab02/data_raw.json
CSV       : data/clean/bab02/data_raw.csv

```

2.22 Penjelasan Struktur Scraper Mini

Script ini dibagi menjadi beberapa fungsi.

fetch_page()

```
def fetch_page(url):  
    response = requests.get(url, headers=HEADERS, timeout=10)  
    response.raise_for_status()  
    return response
```

Fungsi ini mengambil halaman.

- Input: URL.
- Output: response.
- Jika error, program berhenti.

parse_page()

```
def parse_page(html, source_url):
```

Fungsi ini membaca HTML dan mengambil data penting.

Output:

- URL sumber.
- Title.
- H1.
- Link.
- Paragraf.

save_json()

```
def save_json(data, output_file):
```

Fungsi ini menyimpan data ke JSON.

save_csv()

```
def save_csv(data, output_file):
```

Fungsi ini menyimpan data ke CSV.

main()

```
def main():
```

Fungsi ini mengatur alur utama:

Fetch → Save Raw HTML → Parse → Save JSON → Save CSV

Ini adalah bentuk awal pipeline. Sederhana, tetapi sudah benar secara struktur.

2.23 Catatan Struktur HTML Target

Setiap praktik scraping harus punya catatan struktur HTML target. Tujuannya agar mahasiswa tidak hanya punya kode, tetapi juga paham sumber data.

Contoh catatan:

Target URL:

`https://example.org`

Status:

Halaman statis.

Field yang diambil:

- title dari tag `<title>`
- heading dari tag `<h1>`
- paragraf dari tag `<p>`
- link dari tag `<a href="...">`

Selector:

- title
- h1
- p
- a

Output:

- data/raw/bab02/mini_page.html
- data/clean/bab02/data_raw.json
- data/clean/bab02/data_raw.csv

Catatan etika:

- Hanya mengambil satu halaman.
- Tidak mengambil data pribadi.
- Tidak login.
- Tidak melakukan request berulang agresif.

Catatan seperti ini penting untuk portofolio. Saat orang lain membaca proyek, mereka tahu:

- Data berasal dari mana.
- Struktur HTML yang dipakai.
- Apa yang diambil.
- Apa yang tidak diambil.
- Bagaimana batas etikanya.

2.24 Checklist Praktik Bab 2

Mahasiswa dianggap selesai Bab 2 jika sudah menghasilkan:

- Script mengambil halaman dengan `requests`.
- Script membaca status code, header, dan encoding.
- File HTML mentah.
- Script parsing dengan BeautifulSoup.
- Script mengambil judul, link, paragraf, dan tabel.
- File `data_raw.json`.
- File `data_raw.csv`.
- Catatan struktur HTML target.

Checklist:

- ☐ requests terpasang
- ☐ beautifulsoup4 terpasang
- ☐ lxml terpasang
- ☐ berhasil mengambil halaman statis
- ☐ memahami status code dasar
- ☐ memahami header dasar
- ☐ berhasil menyimpan raw HTML
- ☐ berhasil parsing title dan h1
- ☐ berhasil mengambil link
- ☐ berhasil mengambil paragraf
- ☐ berhasil mengambil tabel sederhana
- ☐ berhasil menyimpan JSON
- ☐ berhasil menyimpan CSV
- ☐ membuat catatan struktur HTML target

2.25 Kesalahan Umum Bab 2

1. Langsung parsing tanpa cek status code

Masalah:

```
soup = BeautifulSoup(response.text, "lxml")
```

Padahal response bisa saja 403, 404, atau 500.

Solusi:

```
response.raise_for_status()
```

2. Data kosong karena halaman dinamis

Masalah:

- Di browser data terlihat.
- Di `requests`, data tidak ada.

Penyebab:

- Data dimuat belakangan oleh JavaScript.

Solusi:

- Simpan HTML mentah.
- Periksa apakah data memang ada di HTML.
- Jika tidak ada, materi lanjutannya dibahas pada Bab 8.

3. Selector terlalu rapuh

Contoh rapuh:

```
body > div > div > div:nth-child(3) > p
```

Lebih baik cari selector yang stabil:

```
article p
.content p
#main-content p
```

4. Tidak menyimpan URL sumber

Data tanpa sumber sulit diaudit.

Solusi:

```
"source_url": source_url
```

5. Tidak menyimpan raw HTML

Jika parsing salah, mahasiswa tidak bisa mengecek ulang.

Solusi:

```
RAW_HTML_FILE.write_text(response.text,  
encoding="utf-8")
```

6. Terlalu banyak request

Masalah:

- Server lambat.
- IP dibatasi.
- Mendapat status 429.

Solusi:

- Gunakan delay.
- Batasi jumlah halaman.
- Jangan scraping agresif.
- Hormati aturan situs.

2.26 Ringkasan Bab

Bab ini membangun fondasi teknis scraping. Mahasiswa belajar bahwa mengambil data web bukan sekadar menjalankan script, tetapi memahami komunikasi HTTP, membaca status code, memeriksa header, mengenali encoding, memahami struktur HTML, dan memilih elemen dengan selector yang tepat.

Praktik Bab 2 memperkenalkan penggunaan `requests` untuk mengambil halaman, BeautifulSoup untuk parsing HTML, serta penyimpanan hasil ke JSON dan CSV. Mahasiswa juga belajar membuat scraper mini yang rapi, modular, dan etis.

Output akhir Bab 2:

- Script scraper dasar.
- File `data_raw.json`.
- File `data_raw.csv`.
- File raw HTML.
- Catatan struktur HTML target.
- Pemahaman awal tentang HTTP, HTML, dan parsing.

Pesan strategis Bab 2:

Mahasiswa yang paham HTTP dan HTML akan lebih tahan menghadapi error, perubahan layout, dan data kosong. Scraper yang baik bukan hanya berjalan, tetapi bisa dijelaskan, diperiksa, dan diperbaiki.

Bab 3 — Scraping Terstruktur dengan BeautifulSoup, lxml, dan Pandas

Scraping yang baik bukan hanya mengambil teks, tapi mengubah halaman web menjadi dataset yang rapi, konsisten, dan siap dianalisis.

Bab 3 diarahkan untuk memperkuat scraping sederhana menjadi scraping terstruktur dengan BeautifulSoup, lxml, dan Pandas, dengan output berupa dataset kecil yang rapi, script modular, CSV siap analisis, dan log error sederhana.

3.1 Dari Scraping Mentah ke Scraping Terstruktur

Pada Bab 2, mahasiswa sudah belajar mengambil halaman web, membaca status code, memahami HTML, dan menyimpan hasil ke JSON atau CSV. Namun, hasil scraping awal biasanya masih sederhana.

Scraping mentah biasanya hanya mengambil:

- Judul halaman.
- Semua link.
- Semua paragraf.
- Semua teks.

Scraping terstruktur lebih serius. Data harus punya field yang jelas, misalnya:

- `title`
- `url`
- `date`
- `author`
- `summary`
- `source_url`
- `scraped_at`

Perbedaannya penting.

Scraping mentah menjawab:

“Apa isi halaman ini?”

Scraping terstruktur menjawab:

“Data apa yang bisa disimpan, dibandingkan, dianalisis, dan dipakai ulang?”

Mahasiswa tidak sedang belajar mengambil teks. Mahasiswa sedang belajar membangun dataset.

3.2 Strategi Memilih Elemen HTML yang Stabil

Kesalahan terbesar pemula adalah memilih elemen HTML secara asal. Akibatnya, script mudah rusak ketika tampilan halaman berubah.

Selector yang rapuh biasanya terlalu panjang:

```
body > div > div > div:nth-child(2) > div > p
```

Selector yang lebih baik biasanya memakai elemen yang bermakna:

```
article
.article-item
.article-title
.post-title
.summary
```

BeautifulSoup mendukung pencarian elemen dengan `find()`, `find_all()`, dan CSS selector melalui `select()`. Dokumentasi resminya menjelaskan bahwa `select()` dapat memakai CSS selector untuk mencari elemen tertentu dalam dokumen HTML. ([Crummy](#))

Prinsip memilih selector

- Pilih **class** atau **id** yang bermakna.
- Hindari selector yang terlalu bergantung pada urutan.
- Cek apakah selector muncul konsisten di banyak item.
- Ambil elemen induk dulu, baru ambil field di dalamnya.
- Simpan URL sumber agar bisa diperiksa ulang.
- Siapkan fallback jika field kosong.

Contoh pola yang baik:

```
items = soup.select(".article-item")

for item in items:
    title = item.select_one(".article-title")
    summary = item.select_one(".summary")
```

Pola ini lebih rapi karena parser bekerja per item, bukan mengambil semua elemen secara terpisah.

3.3 Parsing Cepat dengan lxml

BeautifulSoup adalah alat yang nyaman untuk membaca HTML. Namun, BeautifulSoup membutuhkan parser di belakangnya. Salah satu parser yang sering dipakai adalah **lxml**.

Dokumentasi resmi lxml menyebut lxml sebagai library Python untuk memproses XML dan HTML, dengan fitur yang kaya dan mudah digunakan. ([lxml](#))

Dalam praktik buku ini, lxml dipakai sebagai parser:

```
soup = BeautifulSoup(html, "lxml")
```

Keuntungan memakai lxml:

- Parsing lebih cepat untuk banyak kasus.
- Cocok untuk HTML yang cukup besar.
- Mendukung pemrosesan HTML dan XML.
- Bisa dipakai bersama BeautifulSoup.

Namun, mahasiswa tidak perlu langsung memakai semua fitur kompleks lxml. Pada Bab 3, fokusnya adalah:

- Memakai lxml sebagai parser.
- Memahami bahwa parser mempengaruhi hasil parsing.
- Menulis kode yang rapi dan mudah diperiksa.

Parser cepat tidak menggantikan logika scraping yang benar. Selector tetap harus dipilih dengan hati-hati.

3.4 Ekstraksi Tabel HTML

Banyak data publik disajikan dalam tabel HTML. Contohnya:

- Data statistik.
- Jadwal.
- Nilai.
- Peringkat.
- Daftar kampus.
- Daftar produk.
- Data publik pemerintahan.

Tabel HTML biasanya memakai struktur:

```
<table>
  <tr>
    <th>Nama</th>
    <th>Nilai</th>
  </tr>
  <tr>
    <td>Ani</td>
    <td>90</td>
  </tr>
</table>
```

Untuk tabel sederhana, mahasiswa bisa memakai BeautifulSoup. Untuk tabel yang lebih rapi, Pandas menyediakan `read_html()`, yang membaca tabel HTML menjadi daftar `DataFrame`. Dokumentasi Pandas menjelaskan bahwa `read_html()` membaca tabel HTML dan mengembalikan daftar objek DataFrame. ([Pandas](#))

Contoh:

```
import pandas as pd

tables =
pd.read_html("data/raw/bab03/sample_table.html")
df = tables[0]
```

Pandas berguna karena hasil tabel langsung bisa dianalisis, dibersihkan, dan diekspor ke CSV.

3.5 Normalisasi Teks

Data hasil scraping hampir selalu kotor.

Masalah umum:

- Spasi berlebihan.
- Baris baru terlalu banyak.
- Tab.
- Karakter aneh.
- HTML tersisa.
- Encoding rusak.
- Tanggal tidak seragam.
- Teks kosong.
- Duplikasi.

Contoh teks kotor:

```
"\n\n Judul Artikel\t\tTerbaru \n"
```

Setelah dibersihkan:

```
"Judul Artikel Terbaru"
```

Normalisasi sederhana dapat dilakukan dengan Python:

```
def clean_text(text):  
    if text is None:  
        return ""  
  
    return " ".join(text.split())
```

Penjelasan:

- `text.split()` memecah teks berdasarkan spasi, tab, dan newline.
- `" ".join(...)` menyatukan kembali dengan satu spasi.
- Jika `text` kosong, fungsi mengembalikan string kosong.

Data yang bersih membuat analisis lebih mudah. Data yang kotor membuat kesimpulan mudah salah.

3.6 Struktur Data: List of Dict, JSON Lines, dan CSV

Scraping terstruktur membutuhkan format data yang jelas.

1. List of Dict

Format ini nyaman dipakai di Python.

```
items = [  
    {  
        "title": "Artikel 1",  
        "url": "https://example.org/a1",  
        "date": "2026-05-27",  
        "author": "Admin",  
        "summary": "Ringkasan artikel 1"  
    },  
    {  
        "title": "Artikel 2",  
        "url": "https://example.org/a2",  
        "date": "2026-05-27",  
        "author": "Editor",  
        "summary": "Ringkasan artikel 2"  
    }  
]
```

Setiap item adalah satu record.

2. JSON Lines

JSON Lines cocok untuk dataset besar karena setiap baris adalah satu JSON valid. Spesifikasi JSON Lines menekankan tiga aturan utama: encoding UTF-8, setiap baris adalah nilai JSON valid, dan pemisah baris memakai `\n`. ([JSON Lines](#))

Contoh:

```
{"title": "Artikel 1", "url": "https://example.org/a1"}  
{"title": "Artikel 2", "url": "https://example.org/a2"}
```

Kelebihan JSON Lines:

- Mudah ditambah baris baru.
- Cocok untuk pipeline.
- Bisa dibaca per baris.
- Tidak perlu memuat seluruh file sekaligus.

3. CSV

CSV cocok untuk analisis tabel.

Contoh:

```
title,url,date,author,summary
Artikel
1,https://example.org/a1,2026-05-27,Admin,Ringkasan
artikel 1
```

Pandas menyediakan fungsi I/O untuk membaca dan menulis data, termasuk `read_csv()` dan `DataFrame.to_csv()`. ([Pandas](#))

3.7 Kesalahan Umum Scraping Terstruktur

1. Selector rapuh

Selector terlalu panjang dan mudah rusak.

Solusi:

- Gunakan class/id yang bermakna.
- Ambil container item dulu.
- Hindari `nth-child` jika tidak perlu.

2. Data kosong

Penyebab:

- Selector salah.
- HTML berubah.
- Data dimuat secara dinamis.
- Server mengirim halaman error.

Solusi:

- Simpan raw HTML.
- Cek status code.
- Cek selector.
- Buat log error.

3. Duplikasi

Penyebab:

- Link sama muncul beberapa kali.
- Pagination berulang.
- Item direkomendasikan muncul di banyak tempat.

Solusi:

- Gunakan `url` sebagai kunci unik.
- Gunakan `content_hash`.
- Buang duplikasi sebelum disimpan.

4. Perubahan layout

Website bisa berubah kapan saja.

Solusi:

- Catat selector.
- Buat fungsi parsing yang modular.
- Buat validasi jumlah field.
- Simpan error log.

5. Tidak ada data sumber

Dataset tanpa `source_url` sulit diaudit.

Solusi:

- Selalu simpan `source_url`.
- Simpan waktu scraping.
- Simpan file mentah jika perlu.

Bagian Praktik

3.8 Persiapan Folder Bab 3

Masuk ke folder proyek:

```
cd webscraping-llm-database  
source .venv/bin/activate
```

Buat folder Bab 3:

```
mkdir -p scripts/bab03  
mkdir -p data/raw/bab03  
mkdir -p data/clean/bab03  
mkdir -p logs/bab03
```

Penjelasan:

- `scripts/bab03` menyimpan script Bab 3.
- `data/raw/bab03` menyimpan HTML mentah.
- `data/clean/bab03` menyimpan dataset bersih.
- `logs/bab03` menyimpan log error.

Instal paket:

```
pip install requests beautifulsoup4 lxml pandas
```

Simpan daftar paket:

```
pip freeze > requirements.txt
```

3.9 Praktik 1 — Membuat HTML Latihan Daftar Artikel

Agar latihan aman dan legal, kita mulai dari file HTML lokal. Ini penting agar mahasiswa memahami struktur sebelum mengambil halaman nyata.

Buat file:

```
nano data/raw/bab03/sample_articles.html
```

Isi:

```
<!DOCTYPE html>
<html>
<head>
  <title>Portal Artikel Kampus</title>
</head>
<body>
  <main id="content">
    <article class="article-item">
      <h2 class="article-title">
        <a
href="/artikel/data-engineering-mahasiswa">Data
Engineering untuk Mahasiswa</a>
      </h2>
      <p class="article-date">2026-05-27</p>
      <p class="article-author">Admin Kampus</p>
      <p class="article-summary">
        Mahasiswa perlu memahami pipeline data dari
pengambilan, pembersihan, penyimpanan, sampai analisis.
      </p>
    </article>

    <article class="article-item">
      <h2 class="article-title">
        <a href="/artikel/web-scraping-etis">Web
Scraping yang Etis</a>
      </h2>
      <p class="article-date">2026-05-28</p>
      <p class="article-author">Tim Data</p>
      <p class="article-summary">
        Scraping harus dilakukan secara bertanggung
jawab, tidak membebani server, dan tidak mengambil data
sensitif.
      </p>
    </article>

    <article class="article-item">
      <h2 class="article-title">
        <a href="/artikel/llm-lokal">LLM Lokal untuk
Ekstraksi Informasi</a>
      </h2>
    </article>
  </main>
</body>
</html>
```



```
</h2>
<p class="article-date">2026-05-29</p>
<p class="article-author">Lab AI</p>
<p class="article-summary">
    Model lokal dapat membantu membuat ringkasan,
    klasifikasi, dan struktur JSON dari teks hasil scraping.
</p>
</article>
</main>
</body>
</html>
```

Tujuan latihan:

- Memahami container artikel.
- Mengambil field terstruktur.
- Menyimpan dataset.
- Menulis parser modular.

3.10 Praktik 2 — Parsing Daftar Artikel

Buat file:

```
nano scripts/bab03/01_parse_articles.py
```

Isi:

```
from pathlib import Path
from urllib.parse import urljoin
from bs4 import BeautifulSoup

BASE_URL = "https://kampus.example"
INPUT_FILE = Path("data/raw/bab03/sample_articles.html")

def clean_text(text):
    if text is None:
        return ""

    return " ".join(text.split())

html = INPUT_FILE.read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")

article_nodes = soup.select("article.article-item")

items = []

for node in article_nodes:
    title_node = node.select_one(".article-title a")
    date_node = node.select_one(".article-date")
    author_node = node.select_one(".article-author")
    summary_node = node.select_one(".article-summary")

    title = clean_text(title_node.get_text()) if title_node else ""
    href = title_node.get("href") if title_node else ""
    url = urljoin(BASE_URL, href) if href else ""

    item = {
        "title": title,
        "url": url,
        "date": clean_text(date_node.get_text()) if date_node else "",
        "author": clean_text(author_node.get_text()) if author_node else "",
        "summary": clean_text(summary_node.get_text()) if summary_node else "",
    }
```

```
items.append(item)

for item in items:
    print(item)
```

Jalankan:

```
python scripts/bab03/01_parse_articles.py
```

Output yang diharapkan:

```
{'title': 'Data Engineering untuk Mahasiswa', 'url':
'https://kampus.example/artikel/data-engineering-mahasis
wa', 'date': '2026-05-27', 'author': 'Admin Kampus',
'summary': 'Mahasiswa perlu memahami pipeline data dari
pengambilan, pembersihan, penyimpanan, sampai
analisis.'}
```

...

3.11 Penjelasan Script Parsing Artikel

```
from pathlib import Path
```

Memakai Path agar pengelolaan file lebih rapi.

```
from urllib.parse import urljoin
```

Memakai urljoin untuk mengubah URL relatif menjadi URL absolut.

```
from bs4 import BeautifulSoup
```

Menggunakan BeautifulSoup untuk membaca HTML.

```
BASE_URL = "https://kampus.example"
```

Base URL dipakai untuk melengkapi link relatif.

```
def clean_text(text):
```

Fungsi ini membersihkan teks.

```
" ".join(text.split())
```

Menghapus newline, tab, dan spasi berlebihan.

```
soup.select("article.article-item")
```

Memilih semua elemen <article> dengan class article-item.

```
node.select_one(".article-title a")
```

Mengambil satu elemen judul dan link di dalam satu artikel.

Pola penting:

Ambil container artikel dulu, lalu ambil field di dalamnya. Ini membuat parsing lebih stabil.

3.12 Praktik 3 — Menyimpan ke JSON, JSON Lines, dan CSV

Buat file:

```
nano scripts/bab03/02_save_structured_articles.py
```

Isi:

```
import csv
import json
from pathlib import Path
from urllib.parse import urljoin
from bs4 import BeautifulSoup

BASE_URL = "https://kampus.example"
INPUT_FILE = Path("data/raw/bab03/sample_articles.html")

JSON_FILE = Path("data/clean/bab03/articles.json")
JSONL_FILE = Path("data/clean/bab03/articles.jsonl")
CSV_FILE = Path("data/clean/bab03/articles.csv")

def clean_text(text):
    if text is None:
        return ""

    return " ".join(text.split())

def parse_items(html):
    soup = BeautifulSoup(html, "lxml")
    article_nodes = soup.select("article.article-item")

    items = []

    for node in article_nodes:
        title_node = node.select_one(".article-title a")
        date_node = node.select_one(".article-date")
        author_node = node.select_one(".article-author")
        summary_node =
node.select_one(".article-summary")

        title = clean_text(title_node.get_text()) if
title_node else ""
        href = title_node.get("href") if title_node else
""
        url = urljoin(BASE_URL, href) if href else ""

        item = {
            "title": title,
            "url": url,
```

```

        "date": clean_text(date_node.get_text()) if
date_node else "",
        "author": clean_text(author_node.get_text())
if author_node else "",
        "summary":
clean_text(summary_node.get_text()) if summary_node else
"",
        "source_url": BASE_URL,
    }

    items.append(item)

return items

def save_json(items, output_file):
    output_file.write_text(
        json.dumps(items, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

def save_jsonl(items, output_file):
    with output_file.open("w", encoding="utf-8") as
file:
        for item in items:
            file.write(json.dumps(item,
ensure_ascii=False) + "\n")

def save_csv(items, output_file):
    fieldnames = ["title", "url", "date", "author",
"summary", "source_url"]

    with output_file.open("w", newline="",
encoding="utf-8") as file:
        writer = csv.DictWriter(file,
fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(items)

def main():
    html = INPUT_FILE.read_text(encoding="utf-8")
    items = parse_items(html)

    save_json(items, JSON_FILE)
    save_jsonl(items, JSONL_FILE)
    save_csv(items, CSV_FILE)

    print(f"Total item: {len(items)}")
    print(f"JSON          : {JSON_FILE}")

```

```
print(f"JSON Lines: {JSONL_FILE}")
print(f"CSV          : {CSV_FILE}")

if __name__ == "__main__":
    main()
```

Jalankan:

```
python scripts/bab03/02_save_structured_articles.py
```

Output:

```
Total item: 3
JSON       : data/clean/bab03/articles.json
JSON Lines: data/clean/bab03/articles.jsonl
CSV        : data/clean/bab03/articles.csv
```

3.13 Praktik 4 — Membaca CSV dengan Pandas

Buat file:

```
nano scripts/bab03/03_read_with_pandas.py
```

Isi:

```
from pathlib import Path
import pandas as pd

CSV_FILE = Path("data/clean/bab03/articles.csv")

df = pd.read_csv(CSV_FILE)

print("Jumlah baris dan kolom:")
print(df.shape)

print("\nNama kolom:")
print(df.columns.tolist())

print("\nLima data pertama:")
print(df.head())

print("\nJumlah data kosong per kolom:")
print(df.isna().sum())
```

Jalankan:

```
python scripts/bab03/03_read_with_pandas.py
```

Penjelasan:

```
pd.read_csv(CSV_FILE)
    Membaca CSV menjadi DataFrame.

df.shape
    Menampilkan jumlah baris dan kolom.

df.columns.tolist()
    Menampilkan nama kolom.

df.head()
    Menampilkan data awal.

df.isna().sum()
    Menghitung data kosong.
```

Pandas berguna untuk mengecek apakah hasil scraping sudah layak dianalisis. Dokumentasi Pandas menjelaskan bahwa `read_csv()` membaca file CSV menjadi DataFrame, dan `to_csv()` menulis DataFrame ke file CSV. ([Pandas](#))

3.14 Praktik 5 — Membersihkan Dataset dengan Pandas

Buat file:

```
nano scripts/bab03/04_clean_with_pandas.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE = Path("data/clean/bab03/articles.csv")
OUTPUT_FILE =
Path("data/clean/bab03/articles_clean.csv")

df = pd.read_csv(INPUT_FILE)

text_columns = ["title", "author", "summary"]

for column in text_columns:
    df[column] = df[column].fillna("")
    df[column] = df[column].astype(str)
    df[column] = df[column].str.split().str.join(" ")

df["url"] = df["url"].fillna("")
df = df.drop_duplicates(subset=["url"])
df = df[df["title"] != ""]
df = df[df["url"] != ""]

df.to_csv(OUTPUT_FILE, index=False)

print(f"Dataset bersih disimpan ke: {OUTPUT_FILE}")
print(f"Jumlah baris akhir: {len(df)}")
```

Jalankan:

```
python scripts/bab03/04_clean_with_pandas.py
```

Penjelasan:

```
df[column].fillna("")
    Mengganti nilai kosong dengan string kosong.
astype(str)
    Memastikan data diperlakukan sebagai teks.
str.split().str.join(" ")
    Membersihkan spasi berlebihan.
drop_duplicates(subset=["url"])
    Menghapus duplikasi berdasarkan URL.
df[df["title"] != ""]
    Membuang baris yang tidak punya judul.
to_csv(..., index=False)
    Menyimpan CSV tanpa kolom index tambahan.
```

3.15 Praktik 6 — Membuat Log Error Sederhana

Dalam scraping nyata, tidak semua item lengkap. Ada yang tidak punya tanggal, author, atau summary. Daripada error diam-diam, kita buat log.

Buat file HTML dengan data tidak lengkap:

```
nano data/raw/bab03/sample_articles_broken.html
```

Isi:

```
<!DOCTYPE html>
<html>
<body>
  <article class="article-item">
    <h2 class="article-title">
      <a href="/artikel/lengkap">Artikel Lengkap</a>
    </h2>
    <p class="article-date">2026-05-27</p>
    <p class="article-author">Admin</p>
    <p class="article-summary">Artikel ini lengkap.</p>
  </article>

  <article class="article-item">
    <h2 class="article-title">
      <a href="/artikel/tanpa-summary">Artikel Tanpa
Summary</a>
    </h2>
    <p class="article-date">2026-05-28</p>
    <p class="article-author">Editor</p>
  </article>

  <article class="article-item">
    <p class="article-date">2026-05-29</p>
    <p class="article-author">Anonim</p>
    <p class="article-summary">Artikel ini tidak punya
judul.</p>
  </article>
</body>
</html>
```

Buat script parser dengan log:

```
nano scripts/bab03/05_parse_with_error_log.py
```

Isi:

```
import json
from pathlib import Path
from urllib.parse import urljoin
from bs4 import BeautifulSoup

BASE_URL = "https://kampus.example"
INPUT_FILE =
Path("data/raw/bab03/sample_articles_broken.html")
OUTPUT_FILE =
Path("data/clean/bab03/articles_checked.json")
LOG_FILE = Path("logs/bab03/error_log.jsonl")

def clean_text(text):
    if text is None:
        return ""

    return " ".join(text.split())

def write_log(message, context):
    log_item = {
        "message": message,
        "context": context,
    }

    with LOG_FILE.open("a", encoding="utf-8") as file:
        file.write(json.dumps(log_item,
ensure_ascii=False) + "\n")

def parse_items(html):
    soup = BeautifulSoup(html, "lxml")
    article_nodes = soup.select("article.article-item")

    items = []

    for index, node in enumerate(article_nodes,
start=1):
        title_node = node.select_one(".article-title a")
        date_node = node.select_one(".article-date")
        author_node = node.select_one(".article-author")
        summary_node =
node.select_one(".article-summary")

        if not title_node:
            write_log(
```

```

        "Item tidak punya title",
        {"item_index": index}
    )
    continue

    href = title_node.get("href")

    if not href:
        write_log(
            "Item tidak punya URL",
            {"item_index": index}
        )
        continue

    if not summary_node:
        write_log(
            "Item tidak punya summary",
            {"item_index": index, "title":
clean_text(title_node.get_text())}
        )

    item = {
        "title": clean_text(title_node.get_text()),
        "url": urljoin(BASE_URL, href),
        "date": clean_text(date_node.get_text()) if
date_node else "",
        "author": clean_text(author_node.get_text())
if author_node else "",
        "summary":
clean_text(summary_node.get_text()) if summary_node else
"",
    }

    items.append(item)

return items

def main():
    if LOG_FILE.exists():
        LOG_FILE.unlink()

    html = INPUT_FILE.read_text(encoding="utf-8")
    items = parse_items(html)

    OUTPUT_FILE.write_text(
        json.dumps(items, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

```

```
print(f"Item valid: {len(items)}")
print(f"Output      : {OUTPUT_FILE}")
print(f"Log         : {LOG_FILE}")

if __name__ == "__main__":
    main()
```

Jalankan:

```
python scripts/bab03/05_parse_with_error_log.py
```

Output:

```
Item valid: 2
Output      : data/clean/bab03/articles_checked.json
Log         : logs/bab03/error_log.jsonl
```

Buka log:

```
cat logs/bab03/error_log.jsonl
```

Log seperti ini penting karena scraping harus bisa diaudit.

3.16 Praktik 7 — Scraping Tabel Publik dengan Pandas

Untuk memahami tabel HTML, gunakan file lokal terlebih dahulu.

Buat file:

```
nano data/raw/bab03/sample_public_table.html
```

Isi:

```
<!DOCTYPE html>
<html>
<head>
  <title>Data Kegiatan Mahasiswa</title>
</head>
<body>
  <h1>Data Kegiatan Mahasiswa</h1>

  <table>
    <thead>
      <tr>
        <th>Nama</th>
        <th>Kegiatan</th>
        <th>Tanggal</th>
        <th>Jam</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Ani</td>
        <td>Workshop Data</td>
        <td>2026-05-27</td>
        <td>09:00</td>
      </tr>
      <tr>
        <td>Budi</td>
        <td>Diskusi AI Lokal</td>
        <td>2026-05-28</td>
        <td>13:00</td>
      </tr>
      <tr>
        <td>Citra</td>
        <td>Praktik Scraping</td>
        <td>2026-05-29</td>
        <td>10:00</td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

Buat script:

```
nano scripts/bab03/06_read_html_table_pandas.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE =
Path("data/raw/bab03/sample_public_table.html")
OUTPUT_FILE = Path("data/clean/bab03/public_table.csv")

tables = pd.read_html(INPUT_FILE)

print(f"Jumlah tabel ditemukan: {len(tables)}")

df = tables[0]

print("\nData awal:")
print(df)

df.columns = [str(column).strip().lower().replace(" ",
"_") for column in df.columns]

df.to_csv(OUTPUT_FILE, index=False)

print(f"\nCSV tabel disimpan ke: {OUTPUT_FILE}")
```

Jalankan:

```
python scripts/bab03/06_read_html_table_pandas.py
```

Output:

```
Jumlah tabel ditemukan: 1
CSV tabel disimpan ke: data/clean/bab03/public_table.csv
```

Penjelasan:

```
pd.read_html(INPUT_FILE)
    Membaca tabel HTML.
tables[0]
    Mengambil tabel pertama.
df.columns = [...]
    Membersihkan nama kolom.
to_csv()
    Menyimpan ke CSV.
```

3.17 Praktik 8 — Fungsi Reusable: `fetch_page()`, `parse_items()`, `save_json()`

Sekarang kita buat script modular yang dapat dikembangkan pada bab berikutnya.

Buat file:

```
nano scripts/bab03/07_modular_scraper.py
```

Isi:

```
import json
import time
from pathlib import Path
from urllib.parse import urljoin

import requests
from bs4 import BeautifulSoup

BASE_URL = "https://kampus.example"
TARGET_URL = "https://kampus.example/artikel"

RAW_FILE = Path("data/raw/bab03/modular_page.html")
OUTPUT_JSON = Path("data/clean/bab03/modular_articles.json")
LOG_FILE = Path("logs/bab03/modular_error_log.jsonl")

HEADERS = {
    "User-Agent": "Mozilla/5.0 (compatible; StudentDataPipeline/1.0)",
    "Accept": "text/html",
}

def clean_text(text):
    if text is None:
        return ""

    return " ".join(text.split())

def write_log(message, context=None):
    log_item = {
        "message": message,
        "context": context or {},
    }

    with LOG_FILE.open("a", encoding="utf-8") as file:
        file.write(json.dumps(log_item, ensure_ascii=False) + "\n")

def fetch_page(url):
```



```

        response = requests.get(url, headers=HEADERS,
timeout=10)
        response.raise_for_status()
        return response.text

def parse_items(html, source_url):
    soup = BeautifulSoup(html, "lxml")
    article_nodes = soup.select("article.article-item")

    items = []

    for index, node in enumerate(article_nodes,
start=1):
        title_node = node.select_one(".article-title a")
        date_node = node.select_one(".article-date")
        author_node = node.select_one(".article-author")
        summary_node =
node.select_one(".article-summary")

        if not title_node:
            write_log("Title kosong", {"item_index":
index, "source_url": source_url})
            continue

        href = title_node.get("href")

        if not href:
            write_log("URL kosong", {"item_index":
index, "source_url": source_url})
            continue

        item = {
            "title": clean_text(title_node.get_text()),
            "url": urljoin(source_url, href),
            "date": clean_text(date_node.get_text()) if
date_node else "",
            "author": clean_text(author_node.get_text())
if author_node else "",
            "summary":
clean_text(summary_node.get_text()) if summary_node else
"",
            "source_url": source_url,
        }

        items.append(item)

    return items

```

```

def save_json(items, output_file):
    output_file.write_text(
        json.dumps(items, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

def main():
    if LOG_FILE.exists():
        LOG_FILE.unlink()

    print("Membaca HTML lokal untuk latihan...")

    html =
Path("data/raw/bab03/sample_articles.html").read_text(en
coding="utf-8")

    RAW_FILE.write_text(html, encoding="utf-8")

    print("Parsing item...")
    items = parse_items(html, TARGET_URL)

    print("Menyimpan JSON...")
    save_json(items, OUTPUT_JSON)

    time.sleep(1)

    print(f"Jumlah item: {len(items)}")
    print(f"Output      : {OUTPUT_JSON}")
    print(f"Log         : {LOG_FILE}")

if __name__ == "__main__":
    main()

```

Jalankan:

```
python scripts/bab03/07_modular_scraper.py
```

Catatan:

- Fungsi `fetch_page()` sudah disiapkan untuk halaman online.
- Dalam latihan ini, script memakai HTML lokal agar aman.
- Pada bab berikutnya, pola modular ini akan dipakai untuk scraping skala lebih besar.

3.18 Praktik 9 — Deduplication Berdasarkan URL

Duplikasi adalah masalah umum. Buat script untuk membersihkan duplikasi.

Buat file:

```
nano scripts/bab03/08_deduplicate.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE = Path("data/clean/bab03/articles.csv")
OUTPUT_FILE =
Path("data/clean/bab03/articles_dedup.csv")

df = pd.read_csv(INPUT_FILE)

before = len(df)

df["url"] = df["url"].fillna("")
df = df[df["url"] != ""]
df = df.drop_duplicates(subset=["url"], keep="first")

after = len(df)

df.to_csv(OUTPUT_FILE, index=False)

print(f"Jumlah awal : {before}")
print(f"Jumlah akhir: {after}")
print(f"Duplikasi dibuang: {before - after}")
print(f"Output: {OUTPUT_FILE}")
```

Jalankan:

```
python scripts/bab03/08_deduplicate.py
```

Prinsip:

URL sering menjadi kunci unik paling sederhana untuk dataset scraping.

Namun, pada kasus lain, URL saja tidak cukup. Nanti pada Bab 6, kita akan memakai `content_hash` untuk validasi yang lebih kuat.

3.19 Praktik 10 — Membuat Ringkasan Kualitas Dataset

Dataset kecil tetap perlu dicek kualitasnya.

Buat file:

```
nano scripts/bab03/09_dataset_quality_report.py
```

Isi:

```
from pathlib import Path
import json
import pandas as pd

INPUT_FILE = Path("data/clean/bab03/articles_clean.csv")
REPORT_FILE =
Path("data/clean/bab03/quality_report.json")

df = pd.read_csv(INPUT_FILE)

report = {
    "total_rows": int(len(df)),
    "total_columns": int(len(df.columns)),
    "columns": df.columns.tolist(),
    "missing_values": df.isna().sum().to_dict(),
    "duplicate_url_count":
int(df.duplicated(subset=["url"]).sum()),
    "empty_title_count": int((df["title"].fillna("") ==
    "").sum()),
    "empty_url_count": int((df["url"].fillna("") ==
    "").sum()),
}

REPORT_FILE.write_text(
    json.dumps(report, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print(json.dumps(report, indent=2, ensure_ascii=False))
print(f"\nLaporan kualitas disimpan ke: {REPORT_FILE}")
```

Jalankan:

```
python scripts/bab03/09_dataset_quality_report.py
```

Output laporan:

```
{
  "total_rows": 3,
  "total_columns": 6,
  "columns": [
    "title",
    "url",
    "date",
    "author",
    "summary",
    "source_url"
  ],
  "missing_values": {
    "title": 0,
    "url": 0,
    "date": 0,
    "author": 0,
    "summary": 0,
    "source_url": 0
  },
  "duplicate_url_count": 0,
  "empty_title_count": 0,
  "empty_url_count": 0
}
```

Dataset yang baik harus bisa diperiksa kualitasnya, bukan hanya dilihat sekilas.

3.20 Struktur Akhir Folder Bab 3

Setelah praktik, struktur folder akan menjadi:

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   │   └── bab03/
│   │       ├── sample_articles.html
│   │       ├── sample_articles_broken.html
│   │       └── sample_public_table.html
│   └── clean/
│       └── bab03/
│           ├── articles.json
│           ├── articles.jsonl
│           ├── articles.csv
│           ├── articles_clean.csv
│           ├── articles_dedup.csv
│           ├── public_table.csv
│           └── quality_report.json
├── logs/
│   └── bab03/
│       ├── error_log.jsonl
│       └── modular_error_log.jsonl
└── scripts/
    └── bab03/
        ├── 01_parse_articles.py
        ├── 02_save_structured_articles.py
        ├── 03_read_with_pandas.py
        ├── 04_clean_with_pandas.py
        ├── 05_parse_with_error_log.py
        ├── 06_read_html_table_pandas.py
        ├── 07_modular_scraper.py
        ├── 08_deduplicate.py
        └── 09_dataset_quality_report.py
```

Struktur ini sudah lebih profesional dibanding Bab 2. Mahasiswa mulai memiliki:

- Data mentah.
- Data bersih.
- Log error.
- Script modular.
- Laporan kualitas dataset.

3.21 Checklist Praktik Bab 3

Mahasiswa dianggap selesai Bab 3 jika sudah menghasilkan:

- Dataset artikel kecil yang rapi.
- Script parsing daftar artikel.
- Script penyimpanan JSON, JSON Lines, dan CSV.
- Script membaca CSV dengan Pandas.
- Script membersihkan teks.
- Script deduplication.
- Script membaca tabel HTML.
- Log error sederhana.
- Laporan kualitas dataset.

Checklist:

- ☐ Memahami selector stabil
- ☐ Bisa memakai BeautifulSoup dengan parser lxml
- ☐ Bisa mengambil title, url, date, author, summary
- ☐ Bisa membuat list of dict
- ☐ Bisa menyimpan JSON
- ☐ Bisa menyimpan JSON Lines
- ☐ Bisa menyimpan CSV
- ☐ Bisa membaca CSV dengan Pandas
- ☐ Bisa membersihkan spasi dan newline
- ☐ Bisa menghapus duplikasi berdasarkan URL
- ☐ Bisa membaca tabel HTML dengan Pandas
- ☐ Bisa membuat log error sederhana
- ☐ Bisa membuat laporan kualitas dataset

3.22 Kesalahan Umum Bab 3

1. Mengambil semua judul, semua tanggal, semua author secara terpisah

Masalah:

```
titles = soup.select(".article-title")
dates = soup.select(".article-date")
authors = soup.select(".article-author")
```

Jika jumlahnya tidak sama, data bisa bergeser.

Solusi:

```
for node in soup.select("article.article-item"):
    title = node.select_one(".article-title")
    date = node.select_one(".article-date")
```

Ambil data per container item.

2. Tidak membersihkan teks

Masalah:

```
"\n Judul Artikel \t"
```

Solusi:

```
" ".join(text.split())
```

3. Tidak menangani field kosong

Masalah:

```
title_node.get_text()
```

Jika title_node tidak ada, program error.

Solusi:

```
title = title_node.get_text(strip=True) if title_node
else ""
```


4. Tidak membuat log

Masalah:

- Data gagal diproses.
- Tidak tahu item mana yang bermasalah.

Solusi:

- Buat `error_log.jsonl`.
- Simpan konteks error.

5. Tidak menghapus duplikasi

Masalah:

Dataset terlihat besar, tetapi isinya berulang.

Solusi:

```
df.drop_duplicates(subset=["url"])
```

6. Langsung percaya hasil scraping

Masalah:

- Dataset belum dicek.
- Ada field kosong.
- Ada duplikasi.
- Ada encoding rusak.

Solusi:

- Buat laporan kualitas.
- Baca dengan Pandas.
- Cek missing value.
- Cek duplikasi.

3.23 Ringkasan Bab

Bab ini memperkuat scraping dasar menjadi scraping terstruktur. Mahasiswa belajar memilih selector yang stabil, memakai BeautifulSoup dengan parser lxml, mengambil daftar artikel menjadi field yang konsisten, menyimpan data ke JSON, JSON Lines, dan CSV, serta membaca dan membersihkan dataset dengan Pandas.

Praktik Bab 3 menekankan bahwa scraping bukan sekadar mengambil teks. Scraping adalah proses membangun dataset. Karena itu, mahasiswa belajar membuat fungsi modular, menangani data kosong, membuat log error, menghapus duplikasi, dan membuat laporan kualitas dataset.

Output akhir Bab 3:

- Dataset kecil yang rapi.
- Script modular.
- File CSV siap dianalisis.
- File JSON dan JSON Lines.
- Log error sederhana.
- Laporan kualitas dataset.

Pesan strategis Bab 3:

Scraper yang baik menghasilkan data yang bisa dipercaya. Data yang bisa dipercaya harus rapi, konsisten, terdokumentasi, dan bisa diperiksa ulang.

Bab 4 — Scrapy Framework untuk Crawling yang Lebih Serius

Gunakan Scrapy ketika scraping sudah berubah dari “ambil satu halaman” menjadi “kelola banyak halaman, banyak item, banyak error, dan banyak aturan.”

Bab 4 diarahkan untuk memperkenalkan Scrapy sebagai framework yang lebih cocok untuk crawling banyak halaman, dengan konsep *spider*, *item*, *pipeline*, *pagination*, dan ekspor data.

Scrapy sendiri didefinisikan dalam dokumentasi resminya sebagai *framework* tingkat tinggi yang cepat untuk *web crawling* dan *web scraping*, digunakan untuk menjelajah website dan mengekstrak data terstruktur dari halaman web. ([Scrapy](#))

4.1 Mengapa Perlu Scrapy?

Pada Bab 2 dan Bab 3, mahasiswa sudah memakai `requests`, BeautifulSoup, lxml, dan Pandas. Alat tersebut sangat baik untuk scraping sederhana. Namun, ketika jumlah halaman makin banyak, masalah mulai muncul.

Masalah umum scraping manual:

- Terlalu banyak fungsi buatan sendiri.
- Sulit mengatur banyak URL.
- Sulit mengikuti link halaman berikutnya.
- Sulit mengatur delay, retry, dan timeout.
- Sulit mengelola error.
- Sulit menjaga struktur data tetap konsisten.
- Sulit membuat pipeline pembersihan data.
- Sulit mengekspor data dalam beberapa format.

Scrapy membantu karena ia bukan sekadar library. Scrapy adalah **framework crawling**.

Artinya, Scrapy sudah menyediakan struktur kerja untuk:

- Mengelola request.
- Membaca response.
- Menulis spider.
- Mendefinisikan item.
- Menjalankan pipeline.
- Mengatur delay dan retry.
- Mengekspor data ke JSON, CSV, atau JSON Lines.
- Mengembangkan proyek scraping yang lebih rapi.

BeautifulSoup cocok untuk belajar dan scraping kecil. Scrapy cocok untuk proyek crawling yang mulai serius.

4.2 Kapan Memakai BeautifulSoup, Kapan Memakai Scrapy?

Mahasiswa perlu bisa memilih alat sesuai kebutuhan. Jangan semua masalah dipaksa memakai satu alat.

Gunakan BeautifulSoup jika:

- Hanya mengambil satu atau beberapa halaman.
- Struktur halaman sederhana.
- Tidak perlu pagination kompleks.
- Tidak perlu retry otomatis.
- Tidak perlu pipeline besar.
- Sedang eksplorasi awal struktur HTML.
- Ingin belajar parsing dengan cepat.

Gunakan Scrapy jika:

- Perlu crawling banyak halaman.
- Perlu mengikuti link otomatis.
- Perlu pagination.
- Perlu menyimpan banyak item.
- Perlu pipeline pembersihan data.
- Perlu kontrol delay, timeout, dan retry.
- Perlu ekspor dataset secara konsisten.
- Proyek akan dijadikan portofolio.

Ringkasnya

```
BeautifulSoup = alat parsing
Scrapy         = framework crawling + parsing + pipeline + export
```

Scrapy tidak menggantikan pemahaman HTML. Scrapy justru membutuhkan pemahaman HTML yang lebih disiplin.

Framework tidak menyelamatkan selector yang buruk.

4.3 Arsitektur Scrapy

Scrapy memiliki beberapa komponen penting. Mahasiswa tidak perlu menghafal semua detail, tetapi harus paham alur besarnya.

Dokumentasi resmi Scrapy menyediakan gambaran arsitektur yang menjelaskan interaksi komponen dan aliran data di dalam sistem Scrapy. ([Scrapy](#))

Alur sederhana:

```
Spider
  ↓ menghasilkan Request
Downloader
  ↓ mengambil halaman
Response
  ↓ dikirim kembali ke Spider
Spider
  ↓ menghasilkan Item
Item Pipeline
  ↓ membersihkan / memvalidasi / menyimpan
Feed Export
  ↓ JSON / CSV / JSON Lines
```

Komponen utama:

- **Project**
Folder kerja Scrapy.
- **Spider**
Kode yang menentukan halaman apa yang dikunjungi dan data apa yang diambil.
- **Request**
Permintaan untuk mengambil URL.
- **Response**
Balasan dari server.
- **Selector**
Cara memilih bagian HTML dengan CSS selector atau XPath.
- **Item**
Struktur data yang ingin dihasilkan.
- **Pipeline**
Tahap pemrosesan item setelah data diekstrak.
- **Feed Export**
Mekanisme ekspor hasil scraping ke format seperti JSON, CSV, dan JSON Lines. Scrapy mendukung format JSON, JSON Lines, CSV, dan XML melalui *feed exports*. ([Scrapy](#))

4.4 Konsep Spider

Spider adalah inti Scrapy. Di dalam spider, mahasiswa menentukan:

- Nama spider.
- URL awal.
- Cara parsing response.
- Data apa yang diambil.
- Link mana yang akan diikuti.

Contoh pola dasar:

```
import scrapy

class ArticleSpider(scrapy.Spider):
    name = "articles"
    start_urls = ["https://example.org"]

    def parse(self, response):
        yield {
            "title": response.css("h1::text").get()
        }
```

Penjelasan:

- `scrapy.Spider` adalah class dasar spider.
- `name` adalah nama spider saat dijalankan.
- `start_urls` adalah daftar URL awal.
- `parse()` adalah fungsi yang menerima response.
- `yield` mengirim item hasil scraping.

Spider adalah peta kerja crawler. Jika spider buruk, dataset ikut buruk.

4.5 Konsep Item

Item adalah struktur data yang ingin dihasilkan.

Tanpa item, spider bisa langsung menghasilkan dictionary. Namun, untuk proyek yang rapi, item membantu membuat field lebih jelas.

Contoh field artikel:

```
title
url
date
author
summary
source_url
scraped_at
```

Dengan item, mahasiswa memaksa dataset memiliki bentuk yang konsisten.

Contoh:

```
import scrapy

class ArticleItem(scrapy.Item):
    title = scrapy.Field()
    url = scrapy.Field()
    date = scrapy.Field()
    author = scrapy.Field()
    summary = scrapy.Field()
    source_url = scrapy.Field()
    scraped_at = scrapy.Field()
```

Item adalah kontrak data. Ia menjelaskan bentuk dataset yang ingin dibuat.

4.6 Konsep Pipeline

Setelah spider menghasilkan item, item dapat masuk ke pipeline.

Dokumentasi resmi Scrapy menjelaskan bahwa setelah item di-scrape oleh spider, item dikirim ke *Item Pipeline*, lalu diproses melalui beberapa komponen secara berurutan. Pipeline dapat membersihkan, memvalidasi, menyimpan, atau membuang item. ([Scrapy](#))

Pipeline dapat dipakai untuk:

- Membersihkan spasi.
- Menghapus item kosong.
- Mengecek field wajib.
- Membuat `content_hash`.
- Menghapus duplikasi.
- Menyimpan ke database.
- Membuat log item yang gagal.

Pada Bab 4, pipeline dibuat sederhana: membersihkan teks dan menolak item tanpa judul atau URL.

4.7 Pagination dan Follow Link

Crawling menjadi penting ketika data tersebar di banyak halaman.

Contoh pola pagination:

```
/page/1  
/page/2  
/page/3
```

Atau:

```
<a class="next" href="/page/2">Next</a>
```

Scrapy bisa mengikuti link dengan:

```
next_page = response.css("a.next::attr(href)").get()  
  
if next_page:  
    yield response.follow(next_page,  
        callback=self.parse)
```

Penjelasan:

- `response.css()` memilih elemen.
- `::attr(href)` mengambil atribut link.
- `response.follow()` membuat request baru.
- `callback=self.parse` berarti halaman berikutnya diproses oleh fungsi `parse()` yang sama.

Crawling bukan sekadar banyak request. Crawling adalah mengikuti struktur halaman secara sadar.

4.8 Delay, Retry, Timeout, dan Throttling

Scraping yang bertanggung jawab harus mengatur kecepatan.

Scrapy memiliki pengaturan seperti:

- `DOWNLOAD_DELAY`
- `RANDOMIZE_DOWNLOAD_DELAY`
- `RETRY_TIMES`
- `DOWNLOAD_TIMEOUT`
- `AUTOTHROTTLING_ENABLED`

Dokumentasi Scrapy menjelaskan bahwa `RANDOMIZE_DOWNLOAD_DELAY` membuat Scrapy menunggu waktu acak antara 0,5 sampai 1,5 kali nilai `DOWNLOAD_DELAY` saat mengambil request dari website yang sama. ([Scrapy](#))

Scrapy juga menyediakan ekstensi *AutoThrottle* untuk mengatur kecepatan crawling secara otomatis berdasarkan beban crawler dan website yang sedang diakses. ([Scrapy](#))

Untuk timeout, dokumentasi Scrapy menjelaskan bahwa `DOWNLOAD_TIMEOUT` menentukan berapa lama downloader menunggu sebelum timeout. ([Scrapy](#))

Contoh pengaturan aman untuk latihan:

```
DOWNLOAD_DELAY = 1
RANDOMIZE_DOWNLOAD_DELAY = True
RETRY_TIMES = 2
DOWNLOAD_TIMEOUT = 15
AUTOTHROTTLING_ENABLED = True
AUTOTHROTTLING_START_DELAY = 1
AUTOTHROTTLING_MAX_DELAY = 10
```

Prinsipnya:

Jangan membuat crawler lebih cepat daripada kemampuan server dan kebutuhan belajar.

Bagian Praktik

4.9 Persiapan Folder Bab 4

Masuk ke folder proyek:

```
cd webscraping-llm-database
source .venv/bin/activate
```

Penjelasan:

- `cd` masuk ke folder proyek.
- `source .venv/bin/activate` mengaktifkan virtual environment.

Buat folder Bab 4:

```
mkdir -p data/raw/bab04
mkdir -p data/clean/bab04
mkdir -p logs/bab04
```

Penjelasan:

- `data/raw/bab04` untuk data mentah.
- `data/clean/bab04` untuk hasil crawling.
- `logs/bab04` untuk log.

4.10 Instalasi Scrapy

Instal Scrapy:

```
pip install scrapy
```

Penjelasan:

- `pip` adalah pengelola paket Python.
- `install` berarti memasang paket.
- `scrapy` adalah framework crawling.

Cek versi:

```
scrapy version
```

Simpan dependency:

```
pip freeze > requirements.txt
```

Penjelasan:

- `pip freeze` mencatat paket terpasang.
- `>` menyimpan output ke file.
- `requirements.txt` memudahkan proyek dijalankan ulang.

4.11 Membuat Project Scrapy

Masuk ke folder scripts/bab04:

```
mkdir -p scripts/bab04
cd scripts/bab04
```

Buat project Scrapy:

```
scrapy startproject kampus_crawler
```

Penjelasan:

- **scrapy** memanggil command Scrapy.
- **startproject** membuat project baru.
- **kampus_crawler** adalah nama project.

Struktur yang terbentuk:

```
kampus_crawler/
├── scrapy.cfg
└── kampus_crawler/
    ├── __init__.py
    ├── items.py
    ├── middlewares.py
    ├── pipelines.py
    ├── settings.py
    └── spiders/
        └── __init__.py
```

Fungsi file penting:

- **scrapy.cfg**
Konfigurasi project.
- **items.py**
Definisi struktur item.
- **pipelines.py**
Pemrosesan item setelah scraping.
- **settings.py**
Pengaturan crawler.
- **spiders/**
Folder tempat spider ditulis.

4.12 Membuat Halaman Latihan Lokal

Agar praktik aman dan legal, kita mulai dari HTML lokal. Scrapy bisa membaca file lokal memakai skema file://.

Kembali ke root proyek:

```
cd ../../..
```

Buat halaman latihan:

```
nano data/raw/bab04/page1.html
```

Isi:

```
<!DOCTYPE html>
<html>
<head>
  <title>Artikel Kampus - Halaman 1</title>
</head>
<body>
  <h1>Artikel Kampus</h1>

  <article class="article-item">
    <h2 class="article-title">
      <a href="article-1.html">Data Engineering untuk
Mahasiswa</a>
    </h2>
    <p class="article-date">2026-05-27</p>
    <p class="article-author">Admin Kampus</p>
    <p class="article-summary">Pipeline data membantu
mahasiswa mengubah data mentah menjadi informasi.</p>
  </article>

  <article class="article-item">
    <h2 class="article-title">
      <a href="article-2.html">Web Scraping yang
Bertanggung Jawab</a>
    </h2>
    <p class="article-date">2026-05-28</p>
    <p class="article-author">Tim Data</p>
    <p class="article-summary">Scraping harus
memperhatikan legalitas, etika, dan beban server.</p>
  </article>

  <a class="next" href="page2.html">Next</a>
</body>
</html>
```

Buat halaman kedua:

```
nano data/raw/bab04/page2.html
```

```

Isi:
<!DOCTYPE html>
<html>
<head>
  <title>Artikel Kampus - Halaman 2</title>
</head>
<body>
  <h1>Artikel Kampus</h1>

  <article class="article-item">
    <h2 class="article-title">
      <a href="article-3.html">LLM Lokal untuk Ekstraksi
Informasi</a>
    </h2>
    <p class="article-date">2026-05-29</p>
    <p class="article-author">Lab AI</p>
    <p class="article-summary">Model lokal dapat
membantu ringkasan, klasifikasi, dan struktur JSON.</p>
  </article>

  <article class="article-item">
    <h2 class="article-title">
      <a href="article-4.html">Database untuk Dataset
Scraping</a>
    </h2>
    <p class="article-date">2026-05-30</p>
    <p class="article-author">Tim Database</p>
    <p class="article-summary">Database menjaga data
scraping tetap rapi, konsisten, dan mudah
ditelusuri.</p>
  </article>
</body>
</html>

```

Kita memakai halaman lokal agar mahasiswa fokus memahami Scrapy tanpa membebani server publik.

4.13 Membuat Spider Pertama

Masuk ke folder spider:

```
cd scripts/bab04/kampus_crawler/kampus_crawler/spiders
```

Buat file spider:

```
nano articles_spider.py
```

Isi:

```
from pathlib import Path
import scrapy
```

```
class ArticlesSpider(scrapy.Spider):
    name = "articles"

    def start_requests(self):
        root_path = Path(__file__).resolve().parents[4]
        start_file = root_path / "data" / "raw" / "bab04" / "page1.html"

        yield scrapy.Request(
            url=start_file.as_uri(),
            callback=self.parse
        )

    def parse(self, response):
        for article in response.css("article.article-item"):
            title_node = article.css(".article-title a")

            yield {
                "title": title_node.css("::text").get(),
                "url": response.urljoin(title_node.attrib.get("href", "")),
                "date": article.css(".article-date::text").get(),
                "author": article.css(".article-author::text").get(),
                "summary": article.css(".article-summary::text").get(),
                "source_url": response.url,
            }

        next_page = response.css("a.next::attr(href)").get()

        if next_page:
            yield response.follow(next_page, callback=self.parse)
```


Kembali ke folder project Scrapy yang berisi `scrapy.cfg`:

```
cd ../../
```

Jalankan spider:

```
scrapy crawl articles
```

Output di terminal akan menunjukkan Scrapy berjalan dan menemukan item.

4.14 Penjelasan Spider Pertama

```
from pathlib import Path
    Memakai Path untuk membuat path file lokal.

import scrapy
    Mengimpor Scrapy.

class ArticlesSpider(scrapy.Spider):
    Membuat spider baru dari class dasar Scrapy.

    name = "articles"
        Nama spider. Nama ini dipakai saat menjalankan:
        scrapy crawl articles

    def start_requests(self):
        Fungsi ini membuat request awal.

    root_path = Path(__file__).resolve().parents[4]
        Menghitung lokasi root proyek berdasarkan posisi file spider.

    start_file.as_uri()
        Mengubah path lokal menjadi URL file://.

    yield scrapy.Request(...)
        Mengirim request ke Scrapy engine.

    def parse(self, response):
        Fungsi parsing untuk membaca response.

    response.css("article.article-item")
        Mengambil semua artikel.

    article.css(".article-title a")
        Mengambil link judul di dalam artikel.

    yield {...}
        Menghasilkan item.

    response.follow(next_page, callback=self.parse)
        Mengikuti link halaman berikutnya dan memprosesnya dengan fungsi yang sama.
```

4.15 Export ke JSON Lines

Scrapy dapat langsung mengekspor hasil crawling ke file.

Jalankan:

```
scrapy crawl articles -O
../../../data/clean/bab04/items.jl
```

Penjelasan:

- `scrapy crawl articles` menjalankan spider.
- `-O` menulis ulang file output jika sudah ada.
- `items.jl` adalah file JSON Lines.

Cek hasil:

```
cat ../../../data/clean/bab04/items.jl
```

Contoh output:

```
{"title": "Data Engineering untuk Mahasiswa", "url":
"file:///.../article-1.html", "date": "2026-05-27",
"author": "Admin Kampus", "summary": "Pipeline data
membantu mahasiswa mengubah data mentah menjadi
informasi.", "source_url": "file:///.../page1.html"}
{"title": "Web Scraping yang Bertanggung Jawab", "url":
"file:///.../article-2.html", "date": "2026-05-28",
"author": "Tim Data", "summary": "Scraping harus
memperhatikan legalitas, etika, dan beban server.",
"source_url": "file:///.../page1.html"}
```

JSON Lines cocok untuk pipeline karena setiap baris adalah satu item.

4.16 Export ke JSON dan CSV

Export ke JSON:

```
scrapy crawl articles -O  
../../../../data/clean/bab04/items.json
```

Export ke CSV:

```
scrapy crawl articles -O  
../../../../data/clean/bab04/items.csv
```

Scrapy mendukung ekspor ke JSON, JSON Lines, dan CSV melalui feed exports.

([Scrapy](#))

Cek CSV:

```
cat ../../../../data/clean/bab04/items.csv
```

Format CSV cocok untuk analisis dengan Pandas pada bab berikutnya.

4.17 Menggunakan Item

Sekarang kita rapikan struktur data dengan Item.

Buka file:

```
nano kampus_crawler/items.py
```

Isi:

```
import scrapy

class ArticleItem(scrapy.Item):
    title = scrapy.Field()
    url = scrapy.Field()
    date = scrapy.Field()
    author = scrapy.Field()
    summary = scrapy.Field()
    source_url = scrapy.Field()
    scraped_at = scrapy.Field()
```

Penjelasan:

- **ArticleItem** adalah struktur data artikel.
- **scrapy.Field()** mendefinisikan field.
- Semua item akan mengikuti struktur ini.

Ubah spider:

```
nano kampus_crawler/spiders/articles_spider.py
```

Isi versi baru:

```
from datetime import datetime
from pathlib import Path

import scrapy
from kampus_crawler.items import ArticleItem

class ArticlesSpider(scrapy.Spider):
    name = "articles"

    def start_requests(self):
        root_path = Path(__file__).resolve().parents[4]
        start_file = root_path / "data" / "raw" /
        "bab04" / "page1.html"

        yield scrapy.Request(
            url=start_file.as_uri(),
            callback=self.parse
        )
```

```

def parse(self, response):
    for article in
response.css("article.article-item"):
        title_node = article.css(".article-title a")

        item = ArticleItem()
        item["title"] =
title_node.css("::text").get()
        item["url"] =
response.urljoin(title_node.attrib.get("href", ""))
        item["date"] =
article.css(".article-date::text").get()
        item["author"] =
article.css(".article-author::text").get()
        item["summary"] =
article.css(".article-summary::text").get()
        item["source_url"] = response.url
        item["scraped_at"] =
datetime.utcnow().isoformat()

        yield item

    next_page =
response.css("a.next::attr(href)").get()

    if next_page:
        yield response.follow(next_page,
callback=self.parse)

```

Jalankan ulang:

```

scrapy crawl articles -O
../../data/clean/bab04/items.jsonl

```

4.18 Membuat Pipeline Pembersihan Awal

Buka file:

```
nano kampus_crawler/pipelines.py
```

Isi:

```
from itemadapter import ItemAdapter
from scrapy.exceptions import DropItem

class CleanTextPipeline:
    def clean_text(self, value):
        if value is None:
            return ""

        return " ".join(str(value).split())

    def process_item(self, item, spider):
        adapter = ItemAdapter(item)

        text_fields = ["title", "date", "author",
                       "summary", "source_url"]

        for field in text_fields:
            adapter[field] =
self.clean_text(adapter.get(field))

        if not adapter.get("title"):
            raise DropItem("Item dibuang karena title
kosong")

        if not adapter.get("url"):
            raise DropItem("Item dibuang karena url
kosong")

        return item
```

Penjelasan:

```
from itemadapter import ItemAdapter
```

Membantu membaca dan mengubah item Scrapy.

```
from scrapy.exceptions import DropItem
```

Dipakai untuk membuang item yang tidak valid.

```
def clean_text(self, value):
```

Fungsi membersihkan teks.

```
process_item(self, item, spider)
```

Fungsi utama pipeline. Scrapy memanggil fungsi ini untuk setiap item.

```
raise DropItem(...)
```

Membuang item yang tidak memenuhi syarat.

Aktifkan pipeline di settings.py:

```
nano kampus_crawler/settings.py
```

Cari bagian ITEM_PIPELINES, lalu atur:

```
ITEM_PIPELINES = {  
    "kampus_crawler.pipelines.CleanTextPipeline": 300,  
}
```

Penjelasan:

- Key adalah lokasi class pipeline.
- Angka 300 adalah urutan prioritas.
- Semakin kecil angka, semakin awal dijalankan.

Jalankan:

```
scrapy crawl articles -O  
../../../../../data/clean/bab04/items.jsonl
```

Sekarang item akan melewati pipeline pembersihan.

4.19 Mengatur Delay, Retry, Timeout, dan AutoThrottle

Buka settings.py:

```
nano kampus_crawler/settings.py
```

Tambahkan atau ubah:

```
ROBOTSTXT_OBEY = True

DOWNLOAD_DELAY = 1
RANDOMIZE_DOWNLOAD_DELAY = True

RETRY_ENABLED = True
RETRY_TIMES = 2

DOWNLOAD_TIMEOUT = 15

AUTOTHROTTLER_ENABLED = True
AUTOTHROTTLER_START_DELAY = 1
AUTOTHROTTLER_MAX_DELAY = 10
AUTOTHROTTLER_TARGET_CONCURRENCY = 1.0
```

Penjelasan:

- `ROBOTSTXT_OBEY = True` meminta Scrapy menghormati `robots.txt`.
- `DOWNLOAD_DELAY = 1` memberi jeda antar request.
- `RANDOMIZE_DOWNLOAD_DELAY = True` membuat delay tidak terlalu mekanis.
- `RETRY_ENABLED = True` mengaktifkan retry.
- `RETRY_TIMES = 2` mencoba ulang maksimal dua kali.
- `DOWNLOAD_TIMEOUT = 15` membatasi waktu tunggu.
- `AUTOTHROTTLER_ENABLED = True` mengaktifkan pengaturan kecepatan otomatis.

Untuk latihan file lokal, pengaturan ini tidak terlalu terasa. Namun, untuk crawling web publik, ini penting secara etika.

Kecepatan crawler harus dikendalikan. Tujuannya belajar dan membangun data, bukan membebani server.

4.20 Menyimpan Log Scrapy

Jalankan spider dengan log ke file:

```
scrapy crawl articles -O
../../../../../data/clean/bab04/items.jl --logfile
../../../../../logs/bab04/scrapy.log
```

Cek log:

```
tail -n 30 ../../../../../logs/bab04/scrapy.log
```

Penjelasan:

- `--logfile` menyimpan log ke file.
- `tail -n 30` menampilkan 30 baris terakhir.
- Log membantu audit dan debugging.

Log berguna untuk melihat:

- Jumlah request.
- Jumlah response.
- Jumlah item.
- Error.
- Item yang dibuang.
- Status spider.

4.21 Membaca Hasil Scrapy dengan Pandas

Kembali ke root proyek:

```
cd ../../..
```

Buat script:

```
nano scripts/bab04/read_scrapy_output.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE = Path("data/clean/bab04/items.jsonl")
OUTPUT_FILE = Path("data/clean/bab04/items_checked.csv")

df = pd.read_json(INPUT_FILE, lines=True)

print("Jumlah baris dan kolom:")
print(df.shape)

print("\nKolom:")
print(df.columns.tolist())

print("\nData awal:")
print(df.head())

print("\nData kosong:")
print(df.isna().sum())

df = df.drop_duplicates(subset=["url"])
df.to_csv(OUTPUT_FILE, index=False)

print(f"\nCSV hasil cek disimpan ke: {OUTPUT_FILE}")
```

Jalankan:

```
python scripts/bab04/read_scrapy_output.py
```

Penjelasan:

- `pd.read_json(..., lines=True)` membaca JSON Lines.
- `df.shape` melihat jumlah baris dan kolom.
- `df.isna().sum()` mengecek data kosong.
- `drop_duplicates()` membuang duplikasi berdasarkan URL.
- `to_csv()` menyimpan hasil cek.

4.22 Struktur Akhir Project Bab 4

Struktur akhir yang diharapkan:

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   │   └── bab04/
│   │       ├── page1.html
│   │       └── page2.html
│   └── clean/
│       └── bab04/
│           ├── items.jl
│           ├── items.json
│           ├── items.csv
│           └── items_checked.csv
├── logs/
│   └── bab04/
│       └── scrapy.log
└── scripts/
    └── bab04/
        ├── read_scrapy_output.py
        ├── kampus_crawler/
        │   ├── scrapy.cfg
        │   └── kampus_crawler/
        │       ├── items.py
        │       ├── pipelines.py
        │       ├── settings.py
        │       └── spiders/
        │           └── articles_spider.py
```

Output utama Bab 4:

- Project Scrapy lengkap.
- Spider yang mengikuti pagination.
- Item terstruktur.
- Pipeline pembersihan awal.
- Dataset `items.jl`.
- Export JSON dan CSV.
- Log crawling.
- CSV hasil pengecekan.

4.23 Checklist Praktik Bab 4

Mahasiswa dianggap selesai Bab 4 jika sudah mampu:

- ☐ Menginstal Scrapy
- ☐ Membuat project Scrapy
- ☐ Memahami struktur project Scrapy
- ☐ Membuat spider pertama
- ☐ Mengambil item dari HTML
- ☐ Mengikuti link pagination
- ☐ Membuat Item
- ☐ Membuat pipeline pembersihan
- ☐ Mengaktifkan pipeline di settings.py
- ☐ Export ke JSON Lines
- ☐ Export ke JSON
- ☐ Export ke CSV
- ☐ Mengatur delay, retry, timeout, dan AutoThrottle
- ☐ Menyimpan log Scrapy
- ☐ Membaca output Scrapy dengan Pandas

4.24 Kesalahan Umum Bab 4

1. Menjalankan spider dari folder yang salah

Masalah:

Scrapy ... no active project

Solusi:

Jalankan dari folder yang berisi `scrapy.cfg`.

```
cd scripts/bab04/kampus_crawler
scrapy crawl articles
```

2. Nama spider tidak cocok

Masalah:

Spider not found

Solusi:

Cek nama spider:

```
name = "articles"
```

Jalankan:

```
scrapy crawl articles
```

3. Selector salah

Masalah:

- Item kosong.
- Field banyak yang kosong.

Solusi:

- Cek HTML mentah.
- Cek selector.
- Mulai dari container:

```
response.css("article.article-item")
```

4. Lupa mengaktifkan pipeline

Masalah:

- Data belum dibersihkan.
- Item kosong tetap lolos.

Solusi:

Aktifkan di `settings.py`:

```
ITEM_PIPELINES = {  
    "kampus_crawler.pipelines.CleanTextPipeline": 300,  
}
```

5. Output lama tercampur

Masalah:

File lama masih berisi data sebelumnya.

Solusi:

Gunakan `-O`, bukan `-o`.

```
scrapy crawl articles -O  
../../data/clean/bab04/items.json
```

6. Crawling terlalu agresif

Masalah:

- Server menolak.
- Mendapat 403 atau 429.
- Scraper dianggap mengganggu.

Solusi:

- Gunakan `DOWNLOAD_DELAY`.
- Aktifkan `AUTOTHROTTLE`.
- Batasi jumlah halaman.
- Hormati `robots.txt`.
- Jangan scraping data sensitif.

4.25 Praktik Scraping yang Bertanggung Jawab

Scrapy membuat crawling lebih mudah. Tetapi kemudahan ini juga berbahaya jika tidak dikendalikan.

Prinsip wajib:

- **Jangan mengambil data pribadi.**
- **Jangan bypass login atau proteksi.**
- **Jangan membanjiri server.**
- **Jangan mengambil semua halaman tanpa tujuan jelas.**
- **Simpan sumber URL.**
- **Simpan waktu scraping.**
- **Baca aturan situs jika tersedia.**
- **Gunakan delay dan throttling.**
- **Hentikan crawler jika muncul banyak error.**
- **Dokumentasikan tujuan scraping.**

Scrapy adalah alat kuat. Tetapi profesionalisme mahasiswa terlihat dari cara menggunakannya.

Crawler yang baik bukan yang paling cepat, tetapi yang paling terkendali, etis, dan dapat diaudit.

4.26 Ringkasan Bab

Bab ini memperkenalkan Scrapy sebagai framework untuk crawling yang lebih serius. Mahasiswa belajar kapan harus tetap memakai BeautifulSoup dan kapan harus naik ke Scrapy. Bab ini juga menjelaskan arsitektur Scrapy, mulai dari project, spider, request, response, item, pipeline, pagination, sampai feed export.

Pada bagian praktik, mahasiswa membuat project Scrapy lengkap, menulis spider pertama, melakukan crawling beberapa halaman lokal, memakai item, mengeksport data ke JSON Lines, JSON, dan CSV, serta membuat pipeline pembersihan awal. Mahasiswa juga belajar mengatur delay, retry, timeout, AutoThrottle, dan log agar crawler lebih aman dan bertanggung jawab.

Output akhir Bab 4:

- Project Scrapy lengkap.
- Spider yang dapat mengambil banyak halaman.
- Dataset `items.jl`.
- File JSON dan CSV.
- Pipeline pembersihan awal.
- Log crawling.
- Data siap diperiksa dengan Pandas.

Pesan strategis Bab 4:

Scrapy mengubah scraping dari script kecil menjadi sistem crawling. Mahasiswa yang menguasainya tidak hanya bisa mengambil data, tetapi juga mengelola proses, error, struktur, etika, dan kualitas dataset.

Bab 5 — Database Open Source untuk Data Scraping: SQLite, PostgreSQL, dan MongoDB

Data scraping yang hanya berhenti di file adalah arsip; data scraping yang masuk database menjadi aset yang bisa dicari, dianalisis, diaudit, dan dipakai aplikasi.

Bab 5 diarahkan untuk mengubah hasil scraping dari file mentah menjadi data yang dapat dicari, dianalisis, dan dipakai aplikasi melalui SQLite, PostgreSQL, dan MongoDB.

SQLite cocok untuk latihan ringan karena merupakan database SQL yang kecil, cepat, *self-contained*, dan tidak membutuhkan server terpisah. PostgreSQL cocok untuk data relasional yang lebih kuat karena merupakan sistem database objek-relasional open source yang memakai dan memperluas SQL. MongoDB Community Edition cocok untuk dokumen semi-terstruktur, terutama data JSON hasil scraping. ([SQLite](#))

5.1 Mengapa Data Scraping Perlu Database?

Pada Bab 2 sampai Bab 4, hasil scraping disimpan ke file seperti:

- HTML mentah.
- JSON.
- JSON Lines.
- CSV.

File sangat penting untuk latihan dan audit awal. Namun, file memiliki batas.

Jika data makin banyak, file mulai menyulitkan:

- Sulit mencari data tertentu.
- Sulit menghapus duplikasi.
- Sulit melakukan filter.
- Sulit menjaga relasi antar data.
- Sulit dipakai oleh aplikasi.
- Sulit mencatat status validasi.
- Sulit membedakan data mentah dan data bersih.
- Sulit membuat laporan berkala.

Database membantu menyelesaikan masalah itu.

Dengan database, mahasiswa bisa melakukan:

- **Search** berdasarkan judul, URL, penulis, tanggal, atau topik.
- **Filter** berdasarkan waktu scraping.
- **Sort** berdasarkan tanggal.
- **Count** jumlah item.
- **Deduplication** berdasarkan URL atau hash.
- **Audit trail** berdasarkan `source_url` dan `scraped_at`.
- **Integrasi** dengan aplikasi, API, dashboard, dan LLM.

File adalah titik awal. Database adalah fondasi pipeline data yang serius.

5.2 CSV, SQLite, PostgreSQL, dan MongoDB: Apa Bedanya?

Mahasiswa perlu bisa memilih tempat penyimpanan sesuai kebutuhan.

Penyimpanan	Cocok untuk	Kelebihan	Batasan
CSV	Data kecil, analisis cepat	Mudah dibuka, mudah dibaca Pandas	Tidak kuat untuk relasi, validasi, dan query kompleks
SQLite	Latihan lokal, prototipe, dataset kecil-menengah	Ringan, satu file, tidak perlu server	Kurang cocok untuk banyak pengguna bersamaan
PostgreSQL	Data relasional, aplikasi, pipeline serius	Kuat, stabil, mendukung SQL, index, relasi	Perlu server/container
MongoDB	Data JSON, semi-terstruktur, raw page, dokumen fleksibel	Cocok untuk dokumen bervariasi	Relasi tidak seketat database SQL

Ringkasan praktis:

- Gunakan **CSV** untuk hasil sementara.
- Gunakan **SQLite** untuk latihan lokal.
- Gunakan **PostgreSQL** untuk pipeline relasional yang kuat.
- Gunakan **MongoDB** untuk menyimpan dokumen mentah atau JSON semi-terstruktur.

Dalam buku ini, ketiganya dipakai agar mahasiswa paham pilihan arsitektur.

5.3 Konsep Dasar Database Relasional

Database relasional menyimpan data dalam bentuk tabel.

Contoh tabel:

```
scraped_items
```

Kolom:

```
id
title
url
date
author
summary
source_url
scraped_at
content_hash
```

Setiap baris adalah satu record hasil scraping.

Tabel

Tabel adalah tempat menyimpan data sejenis.

Contoh:

- `raw_pages`
- `scraped_items`
- `clean_items`
- `llm_extractions`
- `pipeline_logs`

Struktur tabel ini juga selaras dengan lampiran schema database pada outline buku.

Kolom

Kolom adalah atribut data.

Contoh:

- `title`
- `url`
- `author`
- `summary`

Primary Key

Primary key adalah identitas unik setiap baris.

Contoh:

```
id INTEGER PRIMARY KEY
```

Index

Index mempercepat pencarian.

Contoh:

```
CREATE INDEX idx_scraped_items_url ON  
scraped_items(url);
```

Relasi

Relasi menghubungkan tabel.

Contoh:

- Satu halaman mentah di `raw_pages`.
- Beberapa hasil ekstraksi di `scraped_items`.
- Hasil LLM di `llm_extractions`.

Database relasional membuat data lebih disiplin.

5.4 Konsep Document Database

MongoDB adalah contoh *document database*. Data disimpan sebagai dokumen mirip JSON.

Contoh dokumen:

```
{
  "source_url": "https://example.org/artikel",
  "raw_text": "Isi halaman...",
  "scraped_at": "2026-05-27T10:00:00",
  "metadata": {
    "status_code": 200,
    "content_type": "text/html"
  }
}
```

Kelebihan *document database*:

- Cocok untuk data semi-terstruktur.
- Cocok untuk JSON hasil scraping.
- Tidak semua dokumen harus punya field yang sama.
- Cocok untuk menyimpan raw data sebelum dibersihkan.

Namun, fleksibilitas juga punya risiko:

- Struktur bisa terlalu bebas.
- Field bisa tidak konsisten.
- Duplikasi mudah terjadi.
- Query analitik bisa lebih sulit jika desain buruk.

MongoDB bagus untuk raw document. PostgreSQL bagus untuk data bersih dan relasional.

5.5 Desain Schema untuk Data Scraping

Untuk proyek buku ini, kita mulai dengan satu tabel utama:

```
scraped_items
```

Field utama:

Field	Fungsi
<code>id</code>	Identitas record
<code>title</code>	Judul artikel/item
<code>url</code>	URL item
<code>date</code>	Tanggal artikel jika tersedia
<code>author</code>	Penulis jika tersedia
<code>summary</code>	Ringkasan atau deskripsi singkat
<code>source_url</code>	URL halaman asal scraping
<code>scraped_at</code>	Waktu data diambil
<code>content_hash</code>	Sidik data untuk deteksi duplikasi
<code>validation_status</code>	Status validasi data

Desain ini sengaja sederhana agar mahasiswa fokus memahami alur.

Prinsip desain:

- Simpan **URL sumber**.
- Simpan **waktu scraping**.
- Simpan **hash konten**.
- Jangan hanya menyimpan teks final.
- Bedakan data mentah dan data bersih.
- Gunakan field validasi.

Schema yang baik membuat pipeline mudah diaudit.

5.6 Risiko Data Duplikat dan Data Kotor

Data scraping sering bermasalah.

Contoh risiko:

- Artikel yang sama muncul di beberapa halaman.
- URL sama tersimpan berkali-kali.
- Judul kosong.
- Tanggal tidak seragam.
- Penulis kosong.
- Summary berisi spasi berlebihan.
- Encoding rusak.
- Data lama tercampur data baru.

Strategi awal:

- Gunakan `url` sebagai *unique field*.
- Gunakan `content_hash`.
- Buang baris tanpa `title`.
- Simpan data gagal ke log.
- Jalankan validasi sebelum insert.
- Jangan menimpa raw data tanpa alasan.

Database tidak otomatis membuat data bersih. Database hanya membantu menegakkan disiplin.

Bagian Praktik

5.7 Persiapan Folder Bab 5

Masuk ke folder proyek:

```
cd webscraping-llm-database
source .venv/bin/activate
```

Buat folder Bab 5:

```
mkdir -p scripts/bab05
mkdir -p data/clean/bab05
mkdir -p database/bab05
mkdir -p logs/bab05
```

Penjelasan:

- `scripts/bab05` menyimpan script Python.
- `data/clean/bab05` menyimpan data bersih.
- `database/bab05` menyimpan file database dan schema.
- `logs/bab05` menyimpan log.

Instal paket:

```
pip install pandas psycopg2-binary pymongo python-dotenv
```

Penjelasan:

- `pandas` untuk membaca CSV/JSON Lines.
- `psycopg2-binary` untuk koneksi PostgreSQL.
- `pymongo` untuk koneksi MongoDB.
- `python-dotenv` untuk membaca konfigurasi `.env`.

Simpan dependency:

```
pip freeze > requirements.txt
```

5.8 Menyiapkan Dataset Latihan

Gunakan dataset hasil Bab 4 jika tersedia:

```
data/clean/bab04/items.jl
```

Jika belum ada, buat dataset latihan:

```
nano data/clean/bab05/items.jl
```

Isi:

```
{ "title": "Data Engineering untuk  
Mahasiswa", "url": "https://kampus.example/artikel/data-en  
gineering", "date": "2026-05-27", "author": "Admin  
Kampus", "summary": "Pipeline data membantu mahasiswa  
mengubah data mentah menjadi  
informasi.", "source_url": "https://kampus.example/artikel  
", "scraped_at": "2026-05-27T09:00:00" }  
{ "title": "Web Scraping yang Bertanggung  
Jawab", "url": "https://kampus.example/artikel/scraping-et  
is", "date": "2026-05-28", "author": "Tim  
Data", "summary": "Scraping harus memperhatikan legalitas,  
etika, dan beban  
server.", "source_url": "https://kampus.example/artikel", "  
scraped_at": "2026-05-27T09:10:00" }  
{ "title": "LLM Lokal untuk Ekstraksi  
Informasi", "url": "https://kampus.example/artikel/llm-lok  
al", "date": "2026-05-29", "author": "Lab  
AI", "summary": "Model lokal dapat membantu ringkasan,  
klasifikasi, dan struktur  
JSON.", "source_url": "https://kampus.example/artikel", "sc  
raped_at": "2026-05-27T09:20:00" }
```

Pastikan satu baris berisi satu JSON. Ini adalah format **JSON Lines**.

5.9 Praktik 1 — Membaca JSON Lines dengan Pandas

Buat file:

```
nano scripts/bab05/01_read_items.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE = Path("data/clean/bab05/items.jl")

df = pd.read_json(INPUT_FILE, lines=True)

print("Jumlah baris dan kolom:")
print(df.shape)

print("\nKolom:")
print(df.columns.tolist())

print("\nData awal:")
print(df.head())
```

Jalankan:

```
python scripts/bab05/01_read_items.py
```

Penjelasan:

- `Path` mengelola lokasi file.
- `pd.read_json(..., lines=True)` membaca JSON Lines.
- `df.shape` menampilkan jumlah baris dan kolom.
- `df.head()` menampilkan data awal.

5.10 Praktik 2 — Membuat Content Hash

`content_hash` membantu mendeteksi duplikasi berdasarkan isi data.

Buat file:

```
nano scripts/bab05/02_add_content_hash.py
```

Isi:

```
from pathlib import Path
import hashlib
import pandas as pd

INPUT_FILE = Path("data/clean/bab05/items.json")
OUTPUT_FILE = Path("data/clean/bab05/items_with_hash.csv")

def make_hash(text):
    text = text or ""
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

df = pd.read_json(INPUT_FILE, lines=True)

df["title"] = df["title"].fillna("")
df["summary"] = df["summary"].fillna("")
df["url"] = df["url"].fillna("")

df["content_hash"] = df.apply(
    lambda row: make_hash(row["title"] + "|" + row["summary"] +
    "|" + row["url"]),
    axis=1
)

df["validation_status"] = "raw"

df.to_csv(OUTPUT_FILE, index=False)

print(f"File dengan hash disimpan ke: {OUTPUT_FILE}")
print(df[["title", "url", "content_hash"]])
```

Jalankan:

```
python scripts/bab05/02_add_content_hash.py
```

Penjelasan:

- `hashlib.sha256()` membuat hash.
- `encode("utf-8")` mengubah teks menjadi byte.
- `hexdigest()` menghasilkan string hash.
- `df.apply(..., axis=1)` menerapkan fungsi per baris.

Hash bukan pengganti validasi, tetapi membantu mendeteksi duplikasi.

SQLite

5.11 Instalasi dan Pemeriksaan SQLite

SQLite biasanya sudah tersedia pada banyak sistem GNU/Linux. SQLite adalah database engine yang *self-contained*, *serverless*, dan *zero-configuration*, sehingga cocok untuk latihan lokal mahasiswa. ([SQLite](#))

Cek SQLite:

```
sqlite3 --version
```

Jika belum tersedia:

```
sudo apt update  
sudo apt install sqlite3
```

Python juga memiliki modul bawaan `sqlite3` untuk bekerja dengan database SQLite. Dokumentasi Python menyediakan tutorial dan referensi modul `sqlite3` sebagai antarmuka DB-API 2.0 untuk SQLite. ([Python documentation](#))

5.12 Praktik 3 — Membuat Schema SQLite

Buat file schema:

```
nano database/bab05/schema_sqlite.sql
```

Isi:

```
CREATE TABLE IF NOT EXISTS scraped_items (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    url TEXT NOT NULL UNIQUE,  
    date TEXT,  
    author TEXT,  
    summary TEXT,  
    source_url TEXT,  
    scraped_at TEXT,  
    content_hash TEXT,  
    validation_status TEXT  
);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_url  
ON scraped_items(url);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_date  
ON scraped_items(date);
```

Penjelasan:

- **CREATE TABLE IF NOT EXISTS** membuat tabel jika belum ada.
- **id INTEGER PRIMARY KEY AUTOINCREMENT** membuat ID otomatis.
- **title TEXT NOT NULL** berarti judul wajib ada.
- **url TEXT NOT NULL UNIQUE** berarti URL wajib unik.
- **CREATE INDEX** mempercepat pencarian.

5.13 Praktik 4 — Membuat Database SQLite

Buat file:

```
nano scripts/bab05/03_create_sqlite_db.py
```

Isi:

```
from pathlib import Path
import sqlite3

DB_FILE = Path("database/bab05/scraping.db")
SCHEMA_FILE = Path("database/bab05/schema_sqlite.sql")

schema_sql = SCHEMA_FILE.read_text(encoding="utf-8")

connection = sqlite3.connect(DB_FILE)
cursor = connection.cursor()

cursor.executescript(schema_sql)

connection.commit()
connection.close()

print(f"Database SQLite siap: {DB_FILE}")
```

Jalankan:

```
python scripts/bab05/03_create_sqlite_db.py
```

Penjelasan:

- `sqlite3.connect(DB_FILE)` membuat atau membuka database.
- `cursor.executescript(schema_sql)` menjalankan banyak perintah SQL.
- `commit()` menyimpan perubahan.
- `close()` menutup koneksi.

5.14 Praktik 5 — Insert Data ke SQLite

Buat file:

```
nano scripts/bab05/04_insert_sqlite.py
```

Isi:

```
from pathlib import Path
import sqlite3
import pandas as pd

DB_FILE = Path("database/bab05/scraping.db")
INPUT_FILE = Path("data/clean/bab05/items_with_hash.csv")

df = pd.read_csv(INPUT_FILE)
df = df.fillna("")

connection = sqlite3.connect(DB_FILE)
cursor = connection.cursor()

sql = """
INSERT OR IGNORE INTO scraped_items
(title, url, date, author, summary, source_url, scraped_at,
content_hash, validation_status)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
"""

for _, row in df.iterrows():
    cursor.execute(
        sql,
        (
            row["title"],
            row["url"],
            row["date"],
            row["author"],
            row["summary"],
            row["source_url"],
            row["scraped_at"],
            row["content_hash"],
            row["validation_status"],
        )
    )

connection.commit()
connection.close()

print(f"Insert ke SQLite selesai. Total data dibaca: {len(df)}")
```

Jalankan:

```
python scripts/bab05/04_insert_sqlite.py
```

Penjelasan:

- **INSERT OR IGNORE** mencegah error jika URL sudah ada.
- **?** adalah placeholder aman untuk parameter SQL.
- **iterrows()** membaca DataFrame per baris.

5.15 Praktik 6 — Query Dasar SQLite

Buat file:

```
nano scripts/bab05/05_query_sqlite.py
```

Isi:

```
from pathlib import Path
import sqlite3

DB_FILE = Path("database/bab05/scraping.db")

connection = sqlite3.connect(DB_FILE)
cursor = connection.cursor()

print("Total item:")
cursor.execute("SELECT COUNT(*) FROM scraped_items")
print(cursor.fetchone()[0])

print("\nDaftar item:")
cursor.execute("""
SELECT title, url, date
FROM scraped_items
ORDER BY date ASC
""")

for row in cursor.fetchall():
    print(row)

print("\nFilter berdasarkan author:")
cursor.execute("""
SELECT title, author
FROM scraped_items
WHERE author = ?
""", ("Tim Data",))

for row in cursor.fetchall():
    print(row)

connection.close()
```

Jalankan:

```
python scripts/bab05/05_query_sqlite.py
```

Query yang dipelajari:

- **SELECT**
- **COUNT**
- **WHERE**
- **ORDER BY**

Ini fondasi query yang akan dipakai terus sampai proyek akhir.

PostgreSQL

5.16 Mengapa PostgreSQL?

PostgreSQL dipakai ketika proyek mulai membutuhkan database relasional yang lebih kuat.

Cocok untuk:

- Dataset scraping yang tumbuh besar.
- Relasi antar tabel.
- Query analitik.
- Index yang lebih serius.
- Integrasi aplikasi.
- Pipeline produksi.
- Audit data.

PostgreSQL adalah sistem database objek-relasional open source yang kuat, memakai dan memperluas SQL, serta dirancang untuk menyimpan dan menskalakan beban kerja data kompleks. ([PostgreSQL](#))

5.17 Menjalankan PostgreSQL dengan Docker Compose

Docker Compose memudahkan pengelolaan service, network, dan volume dalam satu file YAML. Dokumentasi Docker menyebut Compose sebagai alat untuk mendefinisikan dan menjalankan aplikasi multi-container, dengan konfigurasi service dalam satu file YAML. ([Docker Documentation](#))

Buat file:

```
nano database/bab05/docker-compose.yml
```

Isi:

```
services:
  postgres:
    image: postgres:16
    container_name: scraping_postgres
    environment:
      POSTGRES_DB: scraping_db
      POSTGRES_USER: scraping_user
      POSTGRES_PASSWORD: scraping_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  mongo:
    image:
      mongodb/mongodb-community-server:7.0-ubuntu2204
    container_name: scraping_mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

volumes:
  postgres_data:
  mongo_data:
```

Penjelasan:

- services berisi daftar container.
- postgres adalah service PostgreSQL.
- mongo adalah service MongoDB.
- image menentukan image yang dipakai.
- ports membuka port ke host lokal.
- volumes menyimpan data agar tidak hilang saat container dihentikan.

Jalankan:

```
cd database/bab05
docker compose up -d
cd ../../
```

Penjelasan:

- `docker compose up -d` menjalankan service di belakang layar.
- `-d` berarti *detached mode*.

Cek container:

```
docker ps
```

5.18 Praktik 7 — Membuat File .env untuk Database

Buat file:

```
nano .env
```

Tambahkan:

```
POSTGRES_HOST=localhost  
POSTGRES_PORT=5432  
POSTGRES_DB=scraping_db  
POSTGRES_USER=scraping_user  
POSTGRES_PASSWORD=scraping_password
```

```
MONGO_URI=mongodb://localhost:27017  
MONGO_DB=scraping_db
```

Catatan:

Untuk latihan lokal, password sederhana masih dapat diterima. Untuk proyek nyata, gunakan credential yang kuat dan jangan mempublikasikan file `.env`.

5.19 Praktik 8 — Membuat Tabel PostgreSQL

Buat file:

```
nano database/bab05/schema_postgres.sql
```

Isi:

```
CREATE TABLE IF NOT EXISTS scraped_items (  
    id SERIAL PRIMARY KEY,  
    title TEXT NOT NULL,  
    url TEXT NOT NULL UNIQUE,  
    date TEXT,  
    author TEXT,  
    summary TEXT,  
    source_url TEXT,  
    scraped_at TIMESTAMP,  
    content_hash TEXT,  
    validation_status TEXT  
);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_url  
ON scraped_items(url);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_date  
ON scraped_items(date);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_author  
ON scraped_items(author);
```

Buat script:

```
nano scripts/bab05/06_create_postgres_table.py
```

Isi:

```
from pathlib import Path  
import os  
import psycopg2  
from dotenv import load_dotenv  
  
load_dotenv()  
  
SCHEMA_FILE = Path("database/bab05/schema_postgres.sql")  
  
connection = psycopg2.connect(  
    host=os.getenv("POSTGRES_HOST"),  
    port=os.getenv("POSTGRES_PORT"),  
    dbname=os.getenv("POSTGRES_DB"),  
    user=os.getenv("POSTGRES_USER"),  
    password=os.getenv("POSTGRES_PASSWORD"),  
)
```

```
cursor = connection.cursor()
schema_sql = SCHEMA_FILE.read_text(encoding="utf-8")

cursor.execute(schema_sql)

connection.commit()
cursor.close()
connection.close()

print("Tabel PostgreSQL siap.")
```

Jalankan:

```
python scripts/bab05/06_create_postgres_table.py
```

Penjelasan:

- `load_dotenv()` membaca `.env`.
- `psycopg2.connect()` membuka koneksi.
- `cursor.execute()` menjalankan SQL.
- `commit()` menyimpan perubahan.

5.20 Praktik 9 — Insert Data ke PostgreSQL

Buat file:

```
nano scripts/bab05/07_insert_postgres.py
```

Isi:

```
import os
import pandas as pd
import psycopg2
from dotenv import load_dotenv
from pathlib import Path

load_dotenv()

INPUT_FILE =
Path("data/clean/bab05/items_with_hash.csv")

df = pd.read_csv(INPUT_FILE)
df = df.fillna("")

connection = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

cursor = connection.cursor()

sql = """
INSERT INTO scraped_items
(title, url, date, author, summary, source_url,
scraped_at, content_hash, validation_status)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (url) DO NOTHING
"""

for _, row in df.iterrows():
    cursor.execute(
        sql,
        (
            row["title"],
            row["url"],
            row["date"],
            row["author"],
            row["summary"],
            row["source_url"],
```



```

        row["scraped_at"],
        row["content_hash"],
        row["validation_status"],
    )

    connection.commit()
    cursor.close()
    connection.close()

    print(f"Insert ke PostgreSQL selesai. Total data dibaca:
    {len(df)}")

```

Jalankan:

```
python scripts/bab05/07_insert_postgres.py
```

Penjelasan:

- `%s` adalah placeholder parameter untuk `psycopg2`.
- `ON CONFLICT (url) DO NOTHING` mencegah duplikasi URL.
- Data masuk ke tabel `scraped_items`.

5.21 Praktik 10 — Query Dasar PostgreSQL

Buat file:

```
nano scripts/bab05/08_query_postgres.py
```

Isi:

```
import os
import psycopg2
from dotenv import load_dotenv

load_dotenv()

connection = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

cursor = connection.cursor()

print("Total item:")
cursor.execute("SELECT COUNT(*) FROM scraped_items")
print(cursor.fetchone()[0])

print("\nItem terbaru:")
cursor.execute("""
SELECT title, author, date
FROM scraped_items
ORDER BY date DESC
LIMIT 5
""")

for row in cursor.fetchall():
    print(row)

print("\nFilter author:")
cursor.execute("""
SELECT title, url
FROM scraped_items
WHERE author = %s
""", ("Tim Data",))

for row in cursor.fetchall():
    print(row)

cursor.close()
```

```
connection.close()
```

Jalankan:

```
python scripts/bab05/08_query_postgres.py
```

Query yang dipelajari:

- COUNT
- ORDER BY
- LIMIT
- WHERE
- parameter query aman

MongoDB

5.22 Mengapa MongoDB?

MongoDB cocok untuk menyimpan dokumen JSON hasil scraping, terutama data mentah yang strukturnya belum stabil.

Cocok untuk:

- Raw page.
- JSON hasil scraping.
- Metadata response.
- Data semi-terstruktur.
- Output awal LLM.
- Dokumen yang field-nya belum seragam.

MongoDB menyediakan Community Edition, dan dokumentasi resminya menyediakan panduan instalasi termasuk penggunaan image komunitas resmi yang dikelola MongoDB untuk Docker. ([MongoDB](#))

Dalam praktik Bab 5, MongoDB sudah dijalankan bersama PostgreSQL melalui file Compose.

5.23 Praktik 11 — Insert Dokumen JSON ke MongoDB

Buat file:

```
nano scripts/bab05/09_insert_mongodb.py
```

Isi:

```
import os
from pathlib import Path
import pandas as pd
from pymongo import MongoClient
from dotenv import load_dotenv

load_dotenv()

INPUT_FILE = Path("data/clean/bab05/items.json")

mongo_uri = os.getenv("MONGO_URI")
mongo_db = os.getenv("MONGO_DB")

client = MongoClient(mongo_uri)
db = client[mongo_db]
collection = db["raw_pages"]

df = pd.read_json(INPUT_FILE, lines=True)
records = df.to_dict(orient="records")

if records:
    collection.insert_many(records)

print(f"Total dokumen dimasukkan ke MongoDB:
{len(records)}")

client.close()
```

Jalankan:

```
python scripts/bab05/09_insert_mongodb.py
```

Penjelasan:

- `MongoClient()` membuka koneksi MongoDB.
- `db["raw_pages"]` memilih koleksi.
- `to_dict(orient="records")` mengubah DataFrame menjadi list of dict.
- `insert_many()` memasukkan banyak dokumen.

5.24 Praktik 12 — Query Dasar MongoDB

Buat file:

```
nano scripts/bab05/10_query_mongodb.py
```

Isi:

```
import os
from pymongo import MongoClient
from dotenv import load_dotenv

load_dotenv()

mongo_uri = os.getenv("MONGO_URI")
mongo_db = os.getenv("MONGO_DB")

client = MongoClient(mongo_uri)
db = client[mongo_db]
collection = db["raw_pages"]

print("Total dokumen:")
print(collection.count_documents({}))

print("\nFilter author = Tim Data:")
for doc in collection.find({"author": "Tim Data"}, {"_id": 0, "title": 1, "url": 1}):
    print(doc)

print("\nSort berdasarkan date descending:")
for doc in collection.find({}, {"_id": 0, "title": 1, "date": 1}).sort("date", -1):
    print(doc)

client.close()
```

Jalankan:

```
python scripts/bab05/10_query_mongodb.py
```

Query yang dipelajari:

- `count_documents({})`
- `find({})`
- filter field
- projection field
- `sort()`

5.25 Membandingkan Query SQLite, PostgreSQL, dan MongoDB

SQLite

```
SELECT title, url
FROM scraped_items
WHERE author = 'Tim Data';
```

PostgreSQL

```
SELECT title, url
FROM scraped_items
WHERE author = 'Tim Data';
```

MongoDB

```
collection.find(
  {"author": "Tim Data"},
  {"_id": 0, "title": 1, "url": 1}
)
```

Pelajaran:

- SQLite dan PostgreSQL memakai SQL.
- MongoDB memakai query berbasis dokumen.
- SQL kuat untuk tabel dan relasi.
- MongoDB fleksibel untuk dokumen JSON.

Pilih database berdasarkan bentuk data, bukan berdasarkan tren.

5.26 Praktik 13 — Membuat Index MongoDB

Agar pencarian lebih cepat dan URL tidak duplikat, buat index.

Buat file:

```
nano scripts/bab05/11_create_mongodb_index.py
```

Isi:

```
import os
from pymongo import MongoClient, ASCENDING
from dotenv import load_dotenv

load_dotenv()

client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("MONGO_DB")]
collection = db["raw_pages"]

collection.create_index([("url", ASCENDING)],
unique=True)
collection.create_index([("date", ASCENDING)])
collection.create_index([("author", ASCENDING)])

print("Index MongoDB selesai dibuat.")

client.close()
```

Jalankan:

```
python scripts/bab05/11_create_mongodb_index.py
```

Catatan:

Jika sebelumnya sudah ada data duplikat, pembuatan unique index bisa gagal. Bersihkan duplikasi terlebih dahulu.

5.27 Praktik 14 — Membuat Ringkasan Kualitas Data dari Database

Buat script untuk membandingkan jumlah data.

```
nano scripts/bab05/12_database_quality_check.py
```

Isi:

```
import os
import sqlite3
import psycopg2
from pymongo import MongoClient
from dotenv import load_dotenv
from pathlib import Path

load_dotenv()

SQLITE_DB = Path("database/bab05/scraping.db")

sqlite_conn = sqlite3.connect(SQLITE_DB)
sqlite_cur = sqlite_conn.cursor()
sqlite_cur.execute("SELECT COUNT(*) FROM scraped_items")
sqlite_count = sqlite_cur.fetchone()[0]
sqlite_conn.close()

pg_conn = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)
pg_cur = pg_conn.cursor()
pg_cur.execute("SELECT COUNT(*) FROM scraped_items")
postgres_count = pg_cur.fetchone()[0]
pg_cur.close()
pg_conn.close()

mongo_client = MongoClient(os.getenv("MONGO_URI"))
mongo_db = mongo_client[os.getenv("MONGO_DB")]
mongo_count = mongo_db["raw_pages"].count_documents({})
mongo_client.close()

print("Ringkasan jumlah data:")
print(f"SQLite      : {sqlite_count}")
print(f"PostgreSQL: {postgres_count}")
print(f"MongoDB     : {mongo_count}")
```

Jalankan:

```
python scripts/bab05/12_database_quality_check.py
```

Tujuan:

- Mengecek apakah data berhasil masuk.
- Membandingkan jumlah record.
- Menemukan anomali awal.

5.28 Struktur Akhir Folder Bab 5

Setelah praktik selesai, struktur folder menjadi:

```
webscraping-llm-database/
├── data/
│   ├── clean/
│   │   └── bab05/
│   │       ├── items.jl
│   │       └── items_with_hash.csv
├── database/
│   └── bab05/
│       ├── docker-compose.yml
│       ├── schema_sqlite.sql
│       ├── schema_postgres.sql
│       └── scraping.db
├── logs/
│   └── bab05/
├── scripts/
│   └── bab05/
│       ├── 01_read_items.py
│       ├── 02_add_content_hash.py
│       ├── 03_create_sqlite_db.py
│       ├── 04_insert_sqlite.py
│       ├── 05_query_sqlite.py
│       ├── 06_create_postgres_table.py
│       ├── 07_insert_postgres.py
│       ├── 08_query_postgres.py
│       ├── 09_insert_mongodb.py
│       ├── 10_query_mongodb.py
│       ├── 11_create_mongodb_index.py
│       └── 12_database_quality_check.py
```

Output utama Bab 5:

- Database SQLite lokal.
- Container PostgreSQL.
- Container MongoDB.
- Tabel `scraped_items`.
- Koleksi MongoDB `raw_pages`.
- Query dasar untuk membaca hasil scraping.

5.29 Checklist Praktik Bab 5

Mahasiswa dianggap selesai Bab 5 jika sudah mampu:

- ☐ Memahami perbedaan CSV, SQLite, PostgreSQL, dan MongoDB
- ☐ Membaca JSON Lines hasil scraping
- ☐ Membuat content_hash
- ☐ Membuat schema SQLite
- ☐ Membuat database SQLite lokal
- ☐ Insert data ke SQLite
- ☐ Query data dari SQLite
- ☐ Menjalankan PostgreSQL dengan Docker Compose
- ☐ Membuat tabel PostgreSQL
- ☐ Insert data ke PostgreSQL
- ☐ Query data dari PostgreSQL
- ☐ Menjalankan MongoDB Community dengan Docker Compose
- ☐ Insert dokumen JSON ke MongoDB
- ☐ Query dokumen MongoDB
- ☐ Membuat index dasar
- ☐ Membuat ringkasan kualitas data dari database

5.30 Kesalahan Umum Bab 5

1. Menganggap CSV sudah cukup untuk semua kasus

CSV baik untuk awal. Namun, untuk pipeline yang tumbuh, database lebih kuat.

Solusi:

- Gunakan CSV untuk pertukaran sederhana.
- Gunakan SQLite untuk latihan.
- Gunakan PostgreSQL untuk data relasional.
- Gunakan MongoDB untuk dokumen JSON.

2. Tidak membuat field unik

Masalah:

- URL sama masuk berkali-kali.
- Dataset terlihat besar, tetapi banyak duplikasi.

Solusi:

`url TEXT NOT NULL UNIQUE`

3. Menyimpan data tanpa source_url

Masalah:

- Data sulit diaudit.

Solusi:

Selalu simpan:

```
source_url
scraped_at
content_hash
```

4. Memasukkan data kosong ke database

Masalah:

- Judul kosong.
- URL kosong.
- Summary kosong tanpa log.

Solusi:

- Validasi sebelum insert.
- Gunakan `NOT NULL`.
- Buat log rejected data.

5. Credential ditulis langsung di kode

Masalah:

- Kode sulit dipakai ulang.
- Credential bisa bocor.

Solusi:

- Simpan di `.env`.
- Jangan publikasikan `.env`.
- Buat `.env.example`.

6. Container hidup, tetapi koneksi gagal

Cek:

```
docker ps
```

Cek port:

```
docker logs scraping_postgres
docker logs scraping_mongo
```

Solusi:

- Pastikan service berjalan.
- Pastikan port benar.
- Pastikan `.env` sesuai.

7. MongoDB terlalu bebas

Masalah:

- Field tidak konsisten.
- Struktur dokumen berantakan.

Solusi:

- Tetap buat standar field.
- Gunakan validasi di Python.
- Buat index.
- Pisahkan raw document dan processed document.

5.31 Ringkasan Bab

Bab ini mengubah hasil scraping dari file menjadi database yang bisa dicari, dianalisis, dan dipakai aplikasi. Mahasiswa belajar bahwa file seperti CSV, JSON, dan JSON Lines penting untuk tahap awal, tetapi database dibutuhkan agar pipeline data menjadi lebih kuat, rapi, dan bisa diaudit.

SQLite digunakan untuk latihan lokal karena ringan dan tidak membutuhkan server. PostgreSQL digunakan untuk data relasional yang lebih serius, terutama ketika data perlu query kuat, index, dan integrasi aplikasi. MongoDB digunakan untuk menyimpan dokumen JSON dan data semi-terstruktur seperti raw page, metadata scraping, atau output awal LLM.

Pada bagian praktik, mahasiswa membuat database SQLite lokal, menjalankan PostgreSQL dan MongoDB dengan Docker Compose, membuat tabel `scraped_items`, membuat koleksi `raw_pages`, memasukkan data hasil scraping, menjalankan query dasar, membuat index, dan memeriksa kualitas data.

Output akhir Bab 5:

- Database SQLite lokal.
- Container PostgreSQL.
- Container MongoDB.
- Tabel `scraped_items`.
- Koleksi MongoDB `raw_pages`.
- Query dasar untuk membaca hasil scraping.
- Fondasi penyimpanan untuk pipeline end-to-end.

Pesan strategis Bab 5:

Data scraping baru bernilai ketika bisa dicari, diuji, diaudit, dan dipakai ulang.

Database adalah jembatan dari script latihan menuju pipeline data yang profesional.

Bab 6 — Cleaning, Validation, dan Data Quality sebelum Masuk Database

Jangan masukkan data kotor ke database; bersihkan, validasi, catat yang gagal, lalu simpan hanya data yang layak dipakai.

Bab 6 menekankan bahwa data hasil scraping hampir selalu kotor, sehingga mahasiswa perlu belajar *cleaning*, *deduplication*, validasi schema, logging, dan *data lineage* sebelum data masuk database.

Pydantic relevan karena dokumentasi resminya menekankan validasi berbasis *type hints*, schema, dan serialisasi data. Dengan Pydantic, mahasiswa dapat mendefinisikan bentuk data yang benar, lalu menolak data yang tidak memenuhi aturan. ([Pydantic](#))

6.1 Mengapa Cleaning dan Validasi Wajib?

Data hasil scraping jarang bersih. Halaman web dirancang untuk dibaca manusia, bukan selalu untuk diproses mesin. Akibatnya, data yang diambil sering berisi masalah.

Contoh masalah:

- Judul kosong.
- URL kosong.
- Tanggal tidak seragam.
- Author hilang.
- Summary berisi newline berlebihan.
- HTML tersisa di teks.
- Encoding rusak.
- Item duplikat.
- Data dari halaman error ikut masuk.
- Field berubah karena layout website berubah.

Jika data seperti ini langsung masuk database, maka database hanya menjadi **tempat sampah digital**.

Prinsipnya:

Scraping → Cleaning → Validation → Rejected Log → Clean Dataset → Database

Database bukan tempat membersihkan kekacauan. Database adalah tempat menyimpan data yang sudah layak.

6.2 Masalah Umum Data Scraping

1. Data kosong

Contoh:

```
{"title": "", "url": "https://example.org/a1"}
```

Jika **title** kosong, data sulit dianalisis.

2. Duplikasi

Contoh:

```
{"title": "Artikel A", "url": "https://example.org/a"}  
{"title": "Artikel A", "url": "https://example.org/a"}
```

Duplikasi membuat statistik menyesatkan.

3. Tanggal tidak seragam

Contoh:

```
2026-05-27  
27 Mei 2026  
May 27, 2026
```

Jika format tidak seragam, query dan analisis waktu menjadi sulit.

4. HTML tersisa

Contoh:

```
<p>Ini ringkasan <b>penting</b></p>
```

Teks harus dibersihkan menjadi:

Ini ringkasan penting

5. Encoding rusak

Contoh:

Pendidikan â€™™ digital

Biasanya terjadi karena encoding salah baca.

6. Field tidak konsisten

Contoh:

```
{"author": "Admin"}  
{"writer": "Admin"}
```

Jika nama field berubah, pipeline menjadi rapuh.

6.3 Cleaning dengan Python dan Pandas

Cleaning dapat dilakukan dengan fungsi Python sederhana dan Pandas.

Pandas menyediakan fungsi untuk menangani data hilang, duplikasi, dan operasi teks. Dokumentasi Pandas menjelaskan `drop_duplicates()` untuk menghapus baris duplikat, dan parameter `subset` dapat dipakai untuk menentukan kolom yang menjadi dasar deteksi duplikasi. ([Pandas](#))

Contoh cleaning teks:

```
def clean_text(text):
    if text is None:
        return ""

    return " ".join(str(text).split())
```

Fungsi ini:

- Mengubah `None` menjadi string kosong.
- Mengubah nilai menjadi teks.
- Menghapus newline, tab, dan spasi berlebihan.
- Menyatukan teks dengan satu spasi.

Contoh Pandas:

```
df["title"] = df["title"].fillna("")
df["title"] =
df["title"].astype(str).str.split().str.join(" ")
df = df.drop_duplicates(subset=["url"])
```

Cleaning sederhana yang konsisten lebih baik daripada cleaning kompleks yang tidak terdokumentasi.

6.4 Validasi Field Wajib

Validasi memastikan data memenuhi aturan minimal.

Field wajib untuk dataset scraping artikel:

- `title`
- `url`
- `source_url`
- `scraped_at`
- `content_hash`

Field opsional:

- `date`
- `author`
- `summary`
- `category`

Contoh aturan:

Field	Aturan
<code>title</code>	Tidak boleh kosong
<code>url</code>	Harus URL valid
<code>date</code>	Format tanggal harus konsisten jika tersedia
<code>summary</code>	Teks, boleh kosong
<code>source_url</code>	Harus ada
<code>scraped_at</code>	Harus timestamp valid
<code>content_hash</code>	Harus ada untuk deduplication

Validasi bukan hanya untuk mencegah error. Validasi adalah **batas kualitas data**.

6.5 Validasi URL, Tanggal, Angka, dan Kategori

URL

URL harus memiliki format yang benar.

Contoh valid:

```
https://kampus.example/artikel/data-engineering
```

Contoh tidak valid:

```
artikel/data-engineering
```

URL relatif bisa diproses sebelumnya dengan `urljoin()`, tetapi sebelum masuk database sebaiknya sudah menjadi URL absolut.

Tanggal

Tanggal sebaiknya distandarkan.

Format yang disarankan:

```
YYYY-MM-DD
```

Contoh:

```
2026-05-27
```

Angka

Jika ada field angka, misalnya views, pastikan menjadi integer.

Contoh:

```
"1,200"
```

Dibersihkan menjadi:

```
1200
```

Kategori

Kategori perlu dibatasi agar tidak liar.

Contoh kategori valid:

- `data`
- `ai`
- `database`
- `scrapping`
- `other`

Jika kategori di luar daftar, ubah menjadi `other` atau masukkan ke rejected log.

6.6 Deduplication Berdasarkan URL dan Hash

Duplikasi dapat dibuang berdasarkan:

- URL.
- `content_hash`.
- Kombinasi title + URL.
- Kombinasi title + summary.

Python memiliki modul `hashlib` yang menyediakan algoritma hash seperti SHA256. Dokumentasi Python menjelaskan bahwa `hashlib` menyediakan antarmuka umum untuk algoritma hash, termasuk SHA256. ([Python documentation](#))

Contoh:

```
import hashlib

def make_hash(text):
    return
    hashlib.sha256(text.encode("utf-8")).hexdigest()
```

Untuk scraping, hash berguna untuk:

- Menandai konten.
- Mendeteksi konten sama.
- Membantu audit perubahan.
- Mengurangi duplikasi.

URL mendeteksi duplikasi alamat. Hash mendeteksi duplikasi isi.

6.7 Logging Data yang Gagal Diproses

Data yang gagal validasi jangan hilang diam-diam. Simpan ke file rejected.

Contoh file:

```
data/rejected/bab06/rejected_rows.json
```

Isi:

```
[
  {
    "reason": "title kosong",
    "row": {
      "title": "",
      "url": "https://example.org/a"
    }
  }
]
```

Python menyediakan modul logging untuk mencatat kejadian saat program berjalan. Dokumentasi Python menjelaskan bahwa logging dipakai untuk melacak event yang terjadi ketika perangkat lunak berjalan, lengkap dengan level atau tingkat kepentingannya. ([Python documentation](#))

Pada Bab 6, kita memakai dua bentuk pencatatan:

- File `rejected_rows.json` untuk data gagal.
- File `quality_report.json` untuk ringkasan kualitas.

6.8 Konsep Data Lineage

Data lineage berarti jejak asal-usul data.

Untuk setiap record, mahasiswa perlu tahu:

- Data berasal dari URL mana.
- Kapan data diambil.
- Script mana yang memproses.
- Data mentahnya seperti apa.
- Data bersihnya seperti apa.
- Apakah data lolos validasi.
- Jika ditolak, alasannya apa.

Field penting:

```
source_url  
scraped_at  
content_hash  
validation_status
```

Tanpa *data lineage*, dataset sulit dipercaya.

Data yang tidak bisa ditelusuri asalnya tidak layak dipakai untuk keputusan serius.

Bagian Praktik

6.9 Persiapan Folder Bab 6

Masuk ke folder proyek:

```
cd webscraping-llm-database  
source .venv/bin/activate
```

Buat folder:

```
mkdir -p scripts/bab06  
mkdir -p data/raw/bab06  
mkdir -p data/clean/bab06  
mkdir -p data/rejected/bab06  
mkdir -p logs/bab06
```

Instal paket:

```
pip install pandas pydantic python-dotenv
```

Simpan dependency:

```
pip freeze > requirements.txt
```


6.10 Membuat Dataset Kotor untuk Latihan

Buat file:

```
nano data/raw/bab06/scraped_dirty.jsonl
```

Isi:

```
{ "title": " Data Engineering untuk Mahasiswa
", "url": "https://kampus.example/artikel/data-engineering
", "date": "2026-05-27", "author": " Admin Kampus
", "summary": "\nPipeline data membantu
mahasiswa.\n", "source_url": "https://kampus.example/artik
el", "scraped_at": "2026-05-27T09:00:00" }
{ "title": "", "url": "https://kampus.example/artikel/tanpa-
judul", "date": "2026-05-28", "author": "Tim
Data", "summary": "Artikel tanpa
judul.", "source_url": "https://kampus.example/artikel", "s
craped_at": "2026-05-27T09:10:00" }
{ "title": "Web Scraping
Etis", "url": "bukan-url-valid", "date": "27 Mei
2026", "author": "Tim Data", "summary": "Scraping harus
bertanggung
jawab.", "source_url": "https://kampus.example/artikel", "s
craped_at": "2026-05-27T09:20:00" }
{ "title": "LLM Lokal untuk
Ekstraksi", "url": "https://kampus.example/artikel/llm-lok
al", "date": "2026-05-29", "author": "", "summary": "Model
lokal membantu struktur
JSON.", "source_url": "https://kampus.example/artikel", "sc
raped_at": "2026-05-27T09:30:00" }
{ "title": "LLM Lokal untuk
Ekstraksi", "url": "https://kampus.example/artikel/llm-lok
al", "date": "2026-05-29", "author": "", "summary": "Model
lokal membantu struktur
JSON.", "source_url": "https://kampus.example/artikel", "sc
raped_at": "2026-05-27T09:30:00" }
```

Dataset ini sengaja kotor:

- Ada spasi berlebihan.
- Ada title kosong.
- Ada URL tidak valid.
- Ada tanggal tidak seragam.
- Ada author kosong.
- Ada duplikasi URL.

6.11 Praktik 1 — Membaca dan Mengecek Dataset Kotor

Buat file:

```
nano scripts/bab06/01_inspect_dirty_data.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE = Path("data/raw/bab06/scraped_dirty.jsonl")

df = pd.read_json(INPUT_FILE, lines=True)

print("Jumlah baris dan kolom:")
print(df.shape)

print("\nKolom:")
print(df.columns.tolist())

print("\nData awal:")
print(df)

print("\nData kosong:")
print(df.isna().sum())

print("\nDuplikasi URL:")
print(df.duplicated(subset=["url"]).sum())
```

Jalankan:

```
python scripts/bab06/01_inspect_dirty_data.py
```

Tujuan:

- Melihat struktur data.
- Melihat data kosong.
- Melihat duplikasi.
- Mengetahui masalah sebelum cleaning.

6.12 Praktik 2 — Membersihkan Teks dengan Python

Buat file:

```
nano scripts/bab06/02_clean_text.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE = Path("data/raw/bab06/scraped_dirty.jsonl")
OUTPUT_FILE =
Path("data/clean/bab06/scraped_clean_step1.csv")

def clean_text(value):
    if value is None:
        return ""

    return " ".join(str(value).split())

df = pd.read_json(INPUT_FILE, lines=True)

text_columns = ["title", "url", "date", "author",
                "summary", "source_url", "scraped_at"]

for column in text_columns:
    df[column] = df[column].fillna("")
    df[column] = df[column].apply(clean_text)

df.to_csv(OUTPUT_FILE, index=False)

print(f"Data hasil cleaning awal disimpan ke:
{OUTPUT_FILE}")
print(df)
```

Jalankan:

```
python scripts/bab06/02_clean_text.py
```

Penjelasan:

- `fillna("")` mengganti nilai kosong.
- `apply(clean_text)` membersihkan setiap field teks.
- Output disimpan sebagai CSV agar mudah diperiksa.

6.13 Praktik 3 — Membuat Content Hash

Buat file:

```
nano scripts/bab06/03_add_lineage_hash.py
```

Isi:

```
from pathlib import Path
import hashlib
import pandas as pd

INPUT_FILE =
Path("data/clean/bab06/scraped_clean_step1.csv")
OUTPUT_FILE =
Path("data/clean/bab06/scraped_with_lineage.csv")

def make_hash(*values):
    text = "|".join(str(value or "") for value in
values)
    return
hashlib.sha256(text.encode("utf-8")).hexdigest()

df = pd.read_csv(INPUT_FILE).fillna("")

df["content_hash"] = df.apply(
    lambda row: make_hash(row["title"], row["url"],
row["summary"]),
    axis=1
)

df["validation_status"] = "pending"

df.to_csv(OUTPUT_FILE, index=False)

print(f>Data dengan lineage awal disimpan ke:
{OUTPUT_FILE}")
print(df[["title", "url", "content_hash",
"validation_status"]])
```

Jalankan:

```
python scripts/bab06/03_add_lineage_hash.py
```

Output penting:

- `content_hash`
- `validation_status`

Dua field ini akan dipakai untuk proses validasi.

6.14 Praktik 4 — Deduplication Berdasarkan URL dan Hash

Buat file:

```
nano scripts/bab06/04_deduplicate.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE =
Path("data/clean/bab06/scraped_with_lineage.csv")
OUTPUT_FILE = Path("data/clean/bab06/scraped_dedup.csv")

df = pd.read_csv(INPUT_FILE).fillna("")

before = len(df)

df = df.drop_duplicates(subset=["url"], keep="first")
df = df.drop_duplicates(subset=["content_hash"],
keep="first")

after = len(df)

df.to_csv(OUTPUT_FILE, index=False)

print(f"Jumlah awal : {before}")
print(f"Jumlah akhir: {after}")
print(f"Duplikasi dibuang: {before - after}")
print(f"Output: {OUTPUT_FILE}")
```

Jalankan:

```
python scripts/bab06/04_deduplicate.py
```

Penjelasan:

- Duplikasi pertama dibuang berdasarkan `url`.
- Duplikasi kedua dibuang berdasarkan `content_hash`.
- `keep="first"` mempertahankan data pertama.

6.15 Praktik 5 — Membuat Schema Pydantic

Buat file:

```
nano scripts/bab06/05_validate_with_pydantic.py
```

Isi:

```
from datetime import datetime, date
from pathlib import Path
import json
import pandas as pd
from pydantic import BaseModel, Field, HttpUrl, ValidationError,
field_validator

INPUT_FILE = Path("data/clean/bab06/scraped_dedup.csv")
CLEAN_FILE = Path("data/clean/bab06/validated_items.json")
REJECTED_FILE = Path("data/rejected/bab06/rejected_rows.json")

class ScrapedItem(BaseModel):
    title: str = Field(min_length=1)
    url: HttpUrl
    date: date | None = None
    author: str = ""
    summary: str = ""
    source_url: HttpUrl
    scraped_at: datetime
    content_hash: str = Field(min_length=20)
    validation_status: str = "valid"

    @field_validator("title", "author", "summary",
mode="before")
    @classmethod
    def clean_string(cls, value):
        if value is None:
            return ""

        return " ".join(str(value).split())

    @field_validator("date", mode="before")
    @classmethod
    def empty_date_to_none(cls, value):
        if value in ("", None):
            return None

        return value

def main():
    df = pd.read_csv(INPUT_FILE).fillna("")

    valid_items = []
    rejected_rows = []

    for index, row in df.iterrows():
        raw_item = row.to_dict()
```

```

try:
    item = ScrapedItem(**raw_item)
    valid_items.append(item.model_dump(mode="json"))
except ValidationError as error:
    rejected_rows.append({
        "row_index": int(index),
        "reason": error.errors(),
        "row": raw_item,
    })

CLEAN_FILE.write_text(
    json.dumps(valid_items, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

REJECTED_FILE.write_text(
    json.dumps(rejected_rows, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print(f"Valid item      : {len(valid_items)}")
print(f"Rejected item: {len(rejected_rows)}")
print(f"Clean file       : {CLEAN_FILE}")
print(f"Rejected file: {REJECTED_FILE}")

if __name__ == "__main__":
    main()

```

Jalankan:

```
python scripts/bab06/05_validate_with_pydantic.py
```

Penjelasan inti:

- `BaseModel` membuat schema.
- `Field(min_length=1)` memastikan field tidak kosong.
- `HttpUrl` memvalidasi URL.
- `date` memvalidasi format tanggal.
- `datetime` memvalidasi timestamp.
- `ValidationError` menangkap data yang gagal.
- `rejected_rows.json` menyimpan data yang ditolak.

Dokumentasi Pydantic juga menjelaskan penggunaan validator untuk mengubah atau memeriksa nilai sebelum/selama validasi. ([Pydantic](#))

6.16 Penjelasan Schema Pydantic

```
class ScrappedItem(BaseModel):  
    Membuat model validasi.  
  
    title: str = Field(min_length=1)  
        Field title harus string dan minimal satu karakter.  
  
    url: HttpUrl  
        Field url harus URL valid.  
  
    date: date | None = None  
        Field date boleh berupa tanggal atau kosong.  
  
    source_url: HttpUrl  
        URL asal scraping wajib valid.  
  
    scraped_at: datetime  
        Waktu scraping wajib timestamp valid.  
  
    content_hash: str = Field(min_length=20)  
        Hash harus ada dan tidak terlalu pendek.  
  
    @field_validator(...)  
        Validator khusus untuk membersihkan atau mengubah data sebelum final.
```

Schema Pydantic adalah pagar kualitas sebelum data masuk database.

6.17 Praktik 6 — Mengubah Data Valid ke CSV Siap Database

Buat file:

```
nano scripts/bab06/06_validated_json_to_csv.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE =
Path("data/clean/bab06/validated_items.json")
OUTPUT_FILE =
Path("data/clean/bab06/validated_items.csv")

df = pd.read_json(INPUT_FILE)

df.to_csv(OUTPUT_FILE, index=False)

print(f"CSV siap database disimpan ke: {OUTPUT_FILE}")
print(df)
```

Jalankan:

```
python scripts/bab06/06_validated_json_to_csv.py
```

Output:

```
data/clean/bab06/validated_items.csv
```

File ini adalah dataset bersih yang siap masuk database.

6.18 Praktik 7 — Membuat Ringkasan Kualitas Data

Buat file:

```
nano scripts/bab06/07_quality_report.py
```

Isi:

```
from pathlib import Path
import json
import pandas as pd

RAW_FILE = Path("data/raw/bab06/scraped_dirty.jsonl")
CLEAN_FILE =
Path("data/clean/bab06/validated_items.csv")
REJECTED_FILE =
Path("data/rejected/bab06/rejected_rows.json")
REPORT_FILE =
Path("data/clean/bab06/quality_report.json")

raw_df = pd.read_json(RAW_FILE, lines=True)
clean_df = pd.read_csv(CLEAN_FILE)

rejected_rows =
json.loads(REJECTED_FILE.read_text(encoding="utf-8"))

report = {
    "raw_rows": int(len(raw_df)),
    "clean_rows": int(len(clean_df)),
    "rejected_rows": int(len(rejected_rows)),
    "duplicate_url_raw":
int(raw_df.duplicated(subset=["url"]).sum()),
    "clean_columns": clean_df.columns.tolist(),
    "missing_values_clean":
clean_df.isna().sum().to_dict(),
    "validation_pass_rate": round(len(clean_df) /
len(raw_df), 4) if len(raw_df) else 0,
}

REPORT_FILE.write_text(
    json.dumps(report, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print(json.dumps(report, indent=2, ensure_ascii=False))
print(f"\nLaporan kualitas disimpan ke: {REPORT_FILE}")
```

Jalankan:

```
python scripts/bab06/07_quality_report.py
```


Contoh output:

```
{
  "raw_rows": 5,
  "clean_rows": 2,
  "rejected_rows": 2,
  "duplicate_url_raw": 1,
  "clean_columns": [
    "title",
    "url",
    "date",
    "author",
    "summary",
    "source_url",
    "scraped_at",
    "content_hash",
    "validation_status"
  ],
  "missing_values_clean": {
    "title": 0,
    "url": 0,
    "date": 0,
    "author": 1,
    "summary": 0,
    "source_url": 0,
    "scraped_at": 0,
    "content_hash": 0,
    "validation_status": 0
  },
  "validation_pass_rate": 0.4
}
```

Makna laporan:

- **raw_rows**: jumlah data awal.
- **clean_rows**: jumlah data lolos.
- **rejected_rows**: jumlah data gagal.
- **duplicate_url_raw**: jumlah duplikasi awal.
- **validation_pass_rate**: rasio data yang lolos.

6.19 Praktik 8 — Membuat Script Cleaning Terpadu

Sekarang gabungkan proses menjadi satu script.

Buat file:

```
nano scripts/bab06/08_clean_validate_pipeline.py
```

Isi:

```
from datetime import datetime, date
from pathlib import Path
import hashlib
import json
import pandas as pd
from pydantic import BaseModel, Field, HttpUrl,
ValidationError, field_validator

RAW_FILE = Path("data/raw/bab06/scraped_dirty.jsonl")
CLEAN_FILE =
Path("data/clean/bab06/final_clean_items.csv")
REJECTED_FILE =
Path("data/rejected/bab06/final_rejected_rows.json")
REPORT_FILE =
Path("data/clean/bab06/final_quality_report.json")

class ScrapedItem(BaseModel):
    title: str = Field(min_length=1)
    url: HttpUrl
    date: date | None = None
    author: str = ""
    summary: str = ""
    source_url: HttpUrl
    scraped_at: datetime
    content_hash: str = Field(min_length=20)
    validation_status: str = "valid"

    @field_validator("title", "author", "summary",
mode="before")
    @classmethod
    def clean_string(cls, value):
        if value is None:
            return ""

        return " ".join(str(value).split())

    @field_validator("date", mode="before")
    @classmethod
    def empty_date_to_none(cls, value):
        if value in ("", None):
```

```

        return None

    return value

def clean_text(value):
    if value is None:
        return ""

    return " ".join(str(value).split())

def make_hash(*values):
    text = "|".join(clean_text(value) for value in
values)
    return
hashlib.sha256(text.encode("utf-8")).hexdigest()

def load_raw_data():
    return pd.read_json(RAW_FILE, lines=True).fillna("")

def clean_dataframe(df):
    text_columns = ["title", "url", "date", "author",
"summary", "source_url", "scraped_at"]

    for column in text_columns:
        df[column] = df[column].apply(clean_text)

    df["content_hash"] = df.apply(
        lambda row: make_hash(row["title"], row["url"],
row["summary"]),
        axis=1
    )

    df["validation_status"] = "pending"

    df = df.drop_duplicates(subset=["url"],
keep="first")
    df = df.drop_duplicates(subset=["content_hash"],
keep="first")

    return df

def validate_rows(df):
    valid_items = []
    rejected_rows = []

    for index, row in df.iterrows():
        raw_item = row.to_dict()

```

```

        try:
            item = ScrapedItem(**raw_item)

valid_items.append(item.model_dump(mode="json"))
        except ValidationError as error:
            rejected_rows.append({
                "row_index": int(index),
                "reason": error.errors(),
                "row": raw_item,
            })

    return valid_items, rejected_rows

def save_outputs(raw_df, clean_df, valid_items,
rejected_rows):
    final_df = pd.DataFrame(valid_items)
    final_df.to_csv(CLEAN_FILE, index=False)

    REJECTED_FILE.write_text(
        json.dumps(rejected_rows, indent=2,
ensure_ascii=False),
        encoding="utf-8"
    )

    report = {
        "raw_rows": int(len(raw_df)),
        "after_clean_dedup_rows": int(len(clean_df)),
        "valid_rows": int(len(valid_items)),
        "rejected_rows": int(len(rejected_rows)),
        "duplicate_url_raw":
int(raw_df.duplicated(subset=["url"]).sum()),
        "validation_pass_rate": round(len(valid_items) /
len(raw_df), 4) if len(raw_df) else 0,
        "output_clean_file": str(CLEAN_FILE),
        "output_rejected_file": str(REJECTED_FILE),
    }

    REPORT_FILE.write_text(
        json.dumps(report, indent=2,
ensure_ascii=False),
        encoding="utf-8"
    )

    return report

def main():
    raw_df = load_raw_data()
    clean_df = clean_dataframe(raw_df.copy())

```

```

        valid_items, rejected_rows = validate_rows(clean_df)
        report = save_outputs(raw_df, clean_df, valid_items,
                              rejected_rows)

        print(json.dumps(report, indent=2,
                          ensure_ascii=False))

if __name__ == "__main__":
    main()

```

Jalankan:

```
python scripts/bab06/08_clean_validate_pipeline.py
```

Output utama:

```

data/clean/bab06/final_clean_items.csv
data/rejected/bab06/final_rejected_rows.json
data/clean/bab06/final_quality_report.json

```

Script ini adalah bentuk awal **cleaning pipeline**.

6.20 Praktik 9 — Validasi Sebelum Insert ke SQLite

Gunakan file bersih:

```
data/clean/bab06/final_clean_items.csv
```

Buat file:

```
nano scripts/bab06/09_insert_clean_to_sqlite.py
```

Isi:

```
from pathlib import Path
import sqlite3
import pandas as pd

DB_FILE = Path("database/bab05/scraping.db")
INPUT_FILE =
Path("data/clean/bab06/final_clean_items.csv")

df = pd.read_csv(INPUT_FILE).fillna("")

connection = sqlite3.connect(DB_FILE)
cursor = connection.cursor()

sql = """
INSERT OR IGNORE INTO scraped_items
(title, url, date, author, summary, source_url,
scraped_at, content_hash, validation_status)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
"""

for _, row in df.iterrows():
    cursor.execute(
        sql,
        (
            row["title"],
            row["url"],
            row["date"],
            row["author"],
            row["summary"],
            row["source_url"],
            row["scraped_at"],
            row["content_hash"],
            row["validation_status"],
        )
    )

connection.commit()
connection.close()
```

```
print(f"Data bersih berhasil dicoba masuk SQLite:  
{len(df)} baris")
```

Jalankan:

```
python scripts/bab06/09_insert_clean_to_sqlite.py
```

Catatan:

Script ini memakai database dari Bab 5. Jika database belum dibuat, jalankan kembali script Bab 5 untuk membuat schema SQLite.

6.21 Struktur Akhir Folder Bab 6

Struktur akhir:

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   │   └── bab06/
│   │       └── scraped_dirty.jsonl
│   ├── clean/
│   │   └── bab06/
│   │       ├── scraped_clean_step1.csv
│   │       ├── scraped_with_lineage.csv
│   │       ├── scraped_dedup.csv
│   │       ├── validated_items.json
│   │       ├── validated_items.csv
│   │       ├── quality_report.json
│   │       ├── final_clean_items.csv
│   │       └── final_quality_report.json
│   └── rejected/
│       └── bab06/
│           ├── rejected_rows.json
│           └── final_rejected_rows.json
└── scripts/
    └── bab06/
        ├── 01_inspect_dirty_data.py
        ├── 02_clean_text.py
        ├── 03_add_lineage_hash.py
        ├── 04_deduplicate.py
        ├── 05_validate_with_pydantic.py
        ├── 06_validated_json_to_csv.py
        ├── 07_quality_report.py
        ├── 08_clean_validate_pipeline.py
        └── 09_insert_clean_to_sqlite.py
```

6.22 Checklist Praktik Bab 6

Mahasiswa dianggap selesai Bab 6 jika sudah mampu:

- ☐ Membaca dataset scraping kotor
- ☐ Mengidentifikasi field kosong
- ☐ Mengidentifikasi duplikasi URL
- ☐ Membersihkan spasi, newline, dan teks berantakan
- ☐ Membuat content_hash
- ☐ Menambahkan source_url dan scraped_at
- ☐ Melakukan deduplication berdasarkan URL
- ☐ Melakukan deduplication berdasarkan content_hash
- ☐ Membuat schema Pydantic
- ☐ Memvalidasi URL
- ☐ Memvalidasi tanggal
- ☐ Memvalidasi timestamp
- ☐ Menyimpan rejected_rows.json
- ☐ Membuat quality_report.json
- ☐ Menghasilkan final_clean_items.csv
- ☐ Insert data bersih ke database

6.23 Kesalahan Umum Bab 6

1. Membersihkan data setelah masuk database

Ini membuat database cepat kotor.

Solusi:

Cleaning dan validasi dilakukan sebelum insert.

2. Tidak menyimpan rejected data

Data yang gagal validasi penting untuk audit.

Solusi:

Simpan ke `rejected_rows.json`.

3. Hanya cek URL, tidak cek isi

URL bisa berbeda, tetapi isi sama.

Solusi:

Gunakan `content_hash`.

4. Menganggap data kosong sebagai normal

Tidak semua field boleh kosong.

Solusi:

Tentukan field wajib dan field opsional.

5. Tidak membuat laporan kualitas

Tanpa laporan, kualitas dataset hanya asumsi.

Solusi:

Buat `quality_report.json`.

6. Tidak menyimpan lineage

Data tanpa jejak asal sulit dipercaya.

Solusi:

Simpan `source_url`, `scraped_at`, `content_hash`, `validation_status`.

6.24 Ringkasan Bab

Bab ini menekankan bahwa data hasil scraping hampir selalu kotor. Mahasiswa belajar membersihkan teks, menghapus duplikasi, membuat hash konten, menambahkan field *lineage*, membuat schema validasi dengan Pydantic, menyimpan data gagal ke `rejected_rows.json`, dan membuat laporan kualitas data.

Praktik Bab 6 mengubah dataset kotor menjadi dataset bersih yang siap masuk database. Mahasiswa tidak hanya belajar menulis script, tetapi juga belajar membangun disiplin kualitas data. Ini penting karena pipeline yang buruk akan menghasilkan keputusan buruk, laporan salah, dan output LLM yang tidak bisa dipercaya.

Output akhir Bab 6:

- Script cleaning.
- Schema validasi Pydantic.
- Dataset bersih.
- File `rejected_rows.json`.
- File `quality_report.json`.
- Data siap masuk database.

Pesan strategis Bab 6:

Garbage in, garbage out. Pipeline data modern harus berani menolak data buruk sebelum data itu merusak database, analisis, dan keputusan.

Bab 7 — Open Source LLM untuk Ekstraksi, Ringkasan, dan Struktur JSON

LLM bukan pengganti scraper; LLM adalah lapisan interpretasi yang harus dibatasi, divalidasi, dan selalu dikaitkan kembali ke sumber data.

Bab 7 memperkenalkan penggunaan *open source LLM* untuk ekstraksi, ringkasan, klasifikasi, dan struktur JSON, dengan Ollama sebagai jalur menjalankan model lokal, serta Crawl4AI sebagai alat pengambilan konten yang lebih ramah untuk LLM.

Ollama menyediakan API lokal yang berjalan secara default pada <http://localhost:11434/api>, sehingga model dapat dipanggil dari script Python atau terminal. ([Ollama Documentation](#)) Crawl4AI dirancang untuk menghasilkan Markdown bersih, *structured extraction*, dan output yang cocok untuk pipeline RAG, LLM, serta agen data. ([Crawl4AI Documentation](#))

7.1 Mengapa LLM Masuk ke Pipeline Scraping?

Scraper mengambil data dari web. Namun, scraper biasa tidak selalu memahami makna teks.

Scraper dapat mengambil:

- Judul artikel.
- Paragraf.
- Tanggal.
- Link.
- Author.
- Tabel.
- Ringkasan pendek jika tersedia di HTML.

Tetapi LLM dapat membantu menafsirkan isi teks.

LLM dapat membantu:

- Membuat ringkasan.
- Mengekstrak entitas.
- Mengambil topik utama.
- Mengubah teks panjang menjadi JSON.
- Mengklasifikasi artikel.
- Mendeteksi informasi sensitif.
- Membuat metadata.
- Membantu validasi isi.

Namun, LLM juga berisiko:

- Membuat jawaban yang tidak ada di sumber.
- Mengubah fakta.
- Menghasilkan JSON rusak.
- Menebak field kosong.
- Tidak konsisten antar pemanggilan.
- Terpengaruh prompt yang buruk.

Karena itu, posisi LLM harus jelas.

Scraper mengambil fakta dari sumber. LLM membantu menata dan menafsirkan fakta. Validasi tetap wajib.

7.2 Scraping Biasa vs LLM-Assisted Extraction

Scraping biasa

Scraping biasa memakai selector.

Contoh:

```
title = soup.select_one("h1").get_text(strip=True)
author = soup.select_one(".author").get_text(strip=True)
date = soup.select_one(".date").get_text(strip=True)
```

Kelebihan:

- Cepat.
- Deterministik.
- Mudah diuji.
- Cocok untuk field yang jelas di HTML.
- Murah secara komputasi.

Kekurangan:

- Sulit memahami teks panjang.
- Tidak bisa menyimpulkan topik.
- Tidak bisa mengekstrak entitas jika tidak ada tag khusus.
- Rapuh jika layout berubah.

LLM-assisted extraction

LLM membaca teks hasil scraping, lalu mengubahnya menjadi struktur.

Contoh output:

```
{
  "entities": [
    {"name": "Python", "type": "TECHNOLOGY"},
    {"name": "PostgreSQL", "type": "DATABASE"}
  ],
  "summary": "Artikel membahas pipeline data scraping dengan Python dan database.",
  "topics": ["web scraping", "database", "LLM"]
}
```

Kelebihan:

- Cocok untuk teks panjang.
- Bisa ekstraksi entitas.
- Bisa ringkasan.
- Bisa klasifikasi.
- Bisa mengubah narasi menjadi JSON.

Kekurangan:

- Perlu validasi.
- Bisa halusinasi.
- Butuh resource lebih besar.
- Output bisa tidak konsisten.
- Tidak cocok untuk field sederhana yang sudah jelas di HTML.

Gunakan scraper untuk field eksplisit. Gunakan LLM untuk makna yang tidak langsung terlihat di HTML.

7.3 Kapan LLM Berguna, Kapan Tidak Perlu?

LLM berguna ketika:

- Teks panjang dan perlu diringkas.
- Perlu ekstraksi entitas dari paragraf.
- Perlu klasifikasi topik.
- Perlu membuat metadata.
- Perlu mengubah teks naratif menjadi JSON.
- Struktur HTML tidak cukup menjelaskan makna isi.
- Data akan dipakai untuk RAG atau analisis teks.

LLM tidak perlu ketika:

- Data sudah ada dalam tabel.
- Field sudah jelas di HTML.
- Hanya perlu mengambil judul, link, tanggal.
- Data harus deterministik 100%.
- Komputasi terbatas.
- Output dapat diambil langsung lewat selector.
- Risiko salah tafsir terlalu tinggi.

Contoh:

Ambil semua link artikel → tidak perlu LLM.

Ringkas isi artikel panjang → LLM berguna.

Ambil nilai dari tag HTML → tidak perlu LLM.

Klasifikasi topik artikel → LLM berguna.

LLM yang dipakai tanpa alasan hanya menambah biaya, risiko, dan kompleksitas.

7.4 Memilih Model Lokal Ringan untuk Mahasiswa

Untuk mahasiswa, model lokal sebaiknya:

- Ringan.
- Mudah dijalankan.
- Cukup baik untuk ringkasan dan ekstraksi.
- Tidak membutuhkan perangkat ekstrem.
- Bisa dipanggil dari Python.
- Cocok untuk latihan JSON.

Pilihan umum:

- Model kecil untuk laptop biasa.
- Model menengah untuk mesin dengan RAM/VRAM lebih besar.
- Model instruksi untuk ekstraksi dan ringkasan.

Prinsip pemilihan:

- Mulai dari model kecil.
- Uji dengan teks pendek.
- Uji konsistensi JSON.
- Jangan langsung memakai model besar.
- Simpan nama model di konfigurasi `.env`.

Contoh konfigurasi:

```
OLLAMA_MODEL=llama3.2:3b
OLLAMA_HOST=http://localhost:11434
```

Nama model dapat berubah mengikuti ketersediaan lokal. Yang penting bukan nama modelnya, tetapi cara menguji, memvalidasi, dan menyimpan hasilnya.

7.5 Risiko Halusinasi dan Cara Mengurangnya

Halusinasi terjadi ketika LLM menghasilkan informasi yang tidak ada di sumber.

Contoh buruk:

Teks sumber:

Artikel ini membahas Python dan SQLite.

Output LLM:

```
{
  "tools": ["Python", "SQLite", "PostgreSQL", "MongoDB"]
}
```

Masalah:

- PostgreSQL dan MongoDB tidak ada di teks sumber.
- LLM menebak berdasarkan konteks umum.

Cara mengurangi halusinasi:

- Beri instruksi: **jangan menambahkan fakta di luar teks.**
- Minta output JSON saja.
- Batasi field.
- Gunakan schema.
- Validasi dengan Pydantic.
- Simpan teks sumber.
- Simpan output mentah LLM.
- Simpan nama model.
- Gunakan temperature rendah jika API mendukung opsi tersebut.
- Tolak output jika JSON tidak valid.

Prompt anti-halusinasi:

```
Gunakan hanya informasi yang muncul eksplisit dalam
teks.
Jika informasi tidak ada, isi dengan null atau list
kosong.
Jangan menebak.
Jangan menambah fakta dari pengetahuan umum.
Output harus JSON valid.
```

LLM boleh membantu, tetapi tidak boleh dipercaya tanpa validasi.

7.6 JSON sebagai Format Utama Output LLM

JSON cocok untuk pipeline karena:

- Mudah dibaca Python.
- Mudah divalidasi.
- Mudah disimpan ke database.
- Cocok untuk API.
- Cocok untuk output ekstraksi.
- Cocok untuk proses lanjutan.

Contoh schema output:

```
{
  "summary": "string",
  "topics": ["string"],
  "entities": [
    {
      "name": "string",
      "type": "string"
    }
  ],
  "sensitive_data_detected": false
}
```

Namun, LLM sering menambahkan teks pembuka seperti:

```
Berikut adalah JSON yang diminta:
```

```
Ini masalah. Pipeline butuh JSON murni.
```

Karena itu, prompt harus tegas:

```
Kembalikan hanya JSON valid. Jangan tambahkan penjelasan
di luar JSON.
```

Pydantic dapat menghasilkan dan memakai JSON Schema untuk validasi model. Dokumentasi Pydantic menjelaskan bahwa Pydantic dapat membuat JSON Schema dari model, sesuai JSON Schema Draft 2020-12 dan OpenAPI 3.1.0. ([Pydantic](#))

Bagian Praktik

7.7 Persiapan Folder Bab 7

Masuk ke folder proyek:

```
cd webscraping-llm-database
source .venv/bin/activate
```

Buat folder:

```
mkdir -p scripts/bab07
mkdir -p data/raw/bab07
mkdir -p data/clean/bab07
mkdir -p data/rejected/bab07
mkdir -p logs/bab07
```

Instal paket Python:

```
pip install requests pydantic pandas psycopg2-binary
python-dotenv
```

Simpan dependency:

```
pip freeze > requirements.txt
```

Penjelasan:

- **requests** untuk memanggil API Ollama.
- **pydantic** untuk validasi output JSON.
- **pandas** untuk membaca dan menulis dataset.
- **psycopg2-binary** untuk PostgreSQL.
- **python-dotenv** untuk konfigurasi **.env**.

7.8 Instalasi Ollama

Instal Ollama mengikuti dokumentasi resmi pada sistem GNU/Linux.

Perintah umum:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Cek apakah Ollama berjalan:

```
ollama --version
```

Jalankan model ringan:

```
ollama pull llama3.2:3b
```

Tes model:

```
ollama run llama3.2:3b
```

Jika model sudah berjalan, API lokal tersedia pada:

```
http://localhost:11434/api
```

Dokumentasi Ollama menjelaskan bahwa API lokal tersedia secara default pada base URL tersebut, dan endpoint seperti `/api/generate` dapat dipanggil setelah Ollama berjalan. ([Ollama Documentation](#))

7.9 Konfigurasi .env

Tambahkan konfigurasi:

```
nano .env
```

Isi atau tambahkan:

```
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=scraping_db
POSTGRES_USER=scraping_user
POSTGRES_PASSWORD=scraping_password
```

Catatan:

- OLLAMA_HOST adalah alamat server lokal Ollama.
- OLLAMA_MODEL adalah model yang dipakai.
- Konfigurasi PostgreSQL mengikuti Bab 5.

Jangan menulis konfigurasi penting langsung di kode. Gunakan `.env`.

7.10 Membuat Data Artikel untuk Latihan

Buat file:

```
nano data/clean/bab07/articles_for_llm.jsonl
```

Isi:

```
{"id":1,"title":"Data Engineering untuk
Mahasiswa","url":"https://kampus.example/artikel/data-en
gineering","clean_text":"Data engineering adalah
kemampuan membangun pipeline data dari pengambilan,
pembersihan, validasi, penyimpanan, sampai analisis.
Mahasiswa perlu memahami Python, database, dan LLM
sebagai satu ekosistem kerja data
modern.","source_url":"https://kampus.example/artikel/da
ta-engineering","scraped_at":"2026-05-27T09:00:00"}
{"id":2,"title":"Web Scraping yang Bertanggung
Jawab","url":"https://kampus.example/artikel/scraping-et
is","clean_text":"Web scraping harus dilakukan secara
etis. Praktisi perlu memperhatikan legalitas, privasi,
terms of service, robots.txt, beban server, dan kualitas
data. Scraping bukan sekadar mengambil data, tetapi
membangun pipeline yang dapat
diaudit.","source_url":"https://kampus.example/artikel/s
craping-etis","scraped_at":"2026-05-27T09:10:00"}
{"id":3,"title":"LLM Lokal untuk Ekstraksi
Informasi","url":"https://kampus.example/artikel/llm-lok
al","clean_text":"LLM lokal dapat membantu ekstraksi
entitas, ringkasan, klasifikasi topik, dan konversi teks
menjadi JSON. Namun output LLM harus divalidasi dengan
schema agar tidak menghasilkan informasi palsu atau
format
rusak.","source_url":"https://kampus.example/artikel/llm
-lokal","scraped_at":"2026-05-27T09:20:00"}
```

Dataset ini adalah teks bersih dari hasil scraping.

7.11 Praktik 1 — Memanggil Ollama dari Python

Buat file:

```
nano scripts/bab07/01_call_ollama.py
```

Isi:

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()

OLLAMA_HOST = os.getenv("OLLAMA_HOST", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:3b")

prompt = """
Ringkas teks berikut dalam satu kalimat pendek.

Teks:
Data engineering adalah kemampuan membangun pipeline data dari
pengambilan,
pembersihan, validasi, penyimpanan, sampai analisis.
"""

payload = {
    "model": OLLAMA_MODEL,
    "prompt": prompt,
    "stream": False
}

response = requests.post(
    f"{OLLAMA_HOST}/api/generate",
    json=payload,
    timeout=120
)

response.raise_for_status()
result = response.json()
print(result["response"])
```

Jalankan:

```
python scripts/bab07/01_call_ollama.py
```

Penjelasan:

- `load_dotenv()` membaca `.env`.
- `requests.post()` mengirim prompt ke Ollama.
- `/api/generate` adalah endpoint generate.
- `stream: False` meminta respons lengkap, bukan streaming.
- `response.json()` membaca output JSON dari API.

Dokumentasi Ollama menunjukkan contoh pemanggilan `/api/generate` dengan payload model dan prompt. ([Ollama Documentation](#))

7.12 Praktik 2 — Prompt Ringkasan Artikel

Buat file:

```
nano scripts/bab07/02_summarize_article.py
```

Isi:

```
import os
import json
import requests
import pandas as pd
from dotenv import load_dotenv
from pathlib import Path

load_dotenv()

OLLAMA_HOST = os.getenv("OLLAMA_HOST",
                        "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:3b")

INPUT_FILE =
Path("data/clean/bab07/articles_for_llm.jsonl")
OUTPUT_FILE =
Path("data/clean/bab07/article_summaries.json")

def call_ollama(prompt):
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False
    }

    response = requests.post(
        f"{OLLAMA_HOST}/api/generate",
        json=payload,
        timeout=120
    )

    response.raise_for_status()
    return response.json()["response"].strip()

df = pd.read_json(INPUT_FILE, lines=True)

results = []

for _, row in df.iterrows():
    prompt = f"""
Anda adalah asisten ekstraksi data.
Ringkas teks berikut dalam maksimal 2 kalimat.
```

Gunakan hanya informasi dari teks.
Jangan menambah fakta dari luar teks.

Judul:

```
{row["title"]}
```

Teks:

```
{row["clean_text"]}
"""
```

```
summary = call_ollama(prompt)

results.append({
    "id": int(row["id"]),
    "title": row["title"],
    "url": row["url"],
    "summary": summary,
    "model_name": OLLAMA_MODEL
})

OUTPUT_FILE.write_text(
    json.dumps(results, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print(f"Ringkasan disimpan ke: {OUTPUT_FILE}")
```

Jalankan:

```
python scripts/bab07/02_summarize_article.py
```

Output:

```
data/clean/bab07/article_summaries.json
```

7.13 Praktik 3 — Prompt Ekstraksi Entitas ke JSON

Buat file:

```
nano scripts/bab07/03_extract_entities_json.py
```

Isi:

```
import os
import json
import requests
import pandas as pd
from dotenv import load_dotenv
from pathlib import Path

load_dotenv()

OLLAMA_HOST = os.getenv("OLLAMA_HOST", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:3b")

INPUT_FILE = Path("data/clean/bab07/articles_for_llm.jsonl")
OUTPUT_FILE = Path("data/clean/bab07/llm_raw_extractions.json")

def call_ollama(prompt):
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False
    }

    response = requests.post(
        f"{OLLAMA_HOST}/api/generate",
        json=payload,
        timeout=120
    )

    response.raise_for_status()
    return response.json()["response"].strip()

df = pd.read_json(INPUT_FILE, lines=True)

results = []

for _, row in df.iterrows():
    prompt = f"""
Anda adalah mesin ekstraksi informasi.

Tugas:
Ekstrak informasi dari teks berikut menjadi JSON valid.

Aturan penting:
- Gunakan hanya informasi yang eksplisit muncul dalam teks.
- Jangan menebak.
- Jangan menambah fakta dari luar teks.
- Jika tidak ada entitas, gunakan list kosong.
```

- Output hanya JSON valid.
- Jangan menulis penjelasan di luar JSON.

Format JSON:

```
{{
  "summary": "ringkasan pendek",
  "topics": ["topik1", "topik2"],
  "entities": [
    {{
      "name": "nama entitas",
      "type": "TECHNOLOGY|CONCEPT|RISK|DATABASE|OTHER"
    }}
  ],
  "sensitive_data_detected": false
}}
```

Judul:

```
{row["title"]}
```

Teks:

```
{row["clean_text"]}
"""
```

```
raw_output = call_ollama(prompt)

results.append({
    "article_id": int(row["id"]),
    "title": row["title"],
    "url": row["url"],
    "model_name": OLLAMA_MODEL,
    "raw_llm_output": raw_output
})

OUTPUT_FILE.write_text(
    json.dumps(results, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print(f"Output mentah LLM disimpan ke: {OUTPUT_FILE}")
```

Jalankan:

```
python scripts/bab07/03_extract_entities_json.py
```

Catatan penting:

Output mentah LLM tetap disimpan. Ini penting untuk audit jika validasi gagal.

7.14 Praktik 4 — Membersihkan Output JSON dari LLM

Kadang LLM tetap menambahkan teks di luar JSON. Kita perlu fungsi ekstraksi JSON sederhana.

Buat file:

```
nano scripts/bab07/04_parse_llm_json.py

Isi:
import json
from pathlib import Path

INPUT_FILE =
Path("data/clean/bab07/llm_raw_extractions.json")
OUTPUT_FILE =
Path("data/clean/bab07/llm_parsed_extractions.json")
REJECTED_FILE =
Path("data/rejected/bab07/llm_parse_failed.json")

def extract_json_block(text):
    start = text.find("{")
    end = text.rfind("}")

    if start == -1 or end == -1 or end <= start:
        raise ValueError("JSON object tidak ditemukan")

    return text[start:end + 1]

raw_items =
json.loads(INPUT_FILE.read_text(encoding="utf-8"))

parsed_items = []
rejected_items = []

for item in raw_items:
    try:
        json_text =
extract_json_block(item["raw_llm_output"])
        parsed_json = json.loads(json_text)

        parsed_items.append({
            "article_id": item["article_id"],
            "title": item["title"],
            "url": item["url"],
            "model_name": item["model_name"],
            "llm_output": parsed_json
        })
```

```

except Exception as error:
    rejected_items.append({
        "article_id": item["article_id"],
        "title": item["title"],
        "url": item["url"],
        "error": str(error),
        "raw_llm_output": item["raw_llm_output"]
    })

OUTPUT_FILE.write_text(
    json.dumps(parsed_items, indent=2,
ensure_ascii=False),
    encoding="utf-8"
)

REJECTED_FILE.write_text(
    json.dumps(rejected_items, indent=2,
ensure_ascii=False),
    encoding="utf-8"
)

print(f"Parsed    : {len(parsed_items)}")
print(f"Rejected  : {len(rejected_items)}")
print(f"Output     : {OUTPUT_FILE}")
print(f"Rejected   : {REJECTED_FILE}")

```

Jalankan:

```
python scripts/bab07/04_parse_llm_json.py
```

Pelajaran:

- Jangan langsung percaya output LLM.
- Parse JSON secara eksplisit.
- Simpan output gagal.
- Pisahkan data valid dan rejected.

7.15 Praktik 5 — Validasi JSON LLM dengan Pydantic

Buat file:

```
nano scripts/bab07/05_validate_llm_json.py
```

Isi:

```
import json
from pathlib import Path
from pydantic import BaseModel, Field, ValidationError

INPUT_FILE =
Path("data/clean/bab07/llm_parsed_extractions.json")
OUTPUT_FILE =
Path("data/clean/bab07/llm_validated_extractions.json")
REJECTED_FILE =
Path("data/rejected/bab07/llm_validation_failed.json")

class Entity(BaseModel):
    name: str = Field(min_length=1)
    type: str =
Field(pattern="^(TECHNOLOGY|CONCEPT|RISK|DATABASE|OTHER)
$")

class LLMExtraction(BaseModel):
    summary: str = Field(min_length=1)
    topics: list[str] = []
    entities: list[Entity] = []
    sensitive_data_detected: bool = False

items =
json.loads(INPUT_FILE.read_text(encoding="utf-8"))

valid_items = []
rejected_items = []

for item in items:
    try:
        validated = LLMExtraction(**item["llm_output"])

        valid_items.append({
            "article_id": item["article_id"],
            "title": item["title"],
            "url": item["url"],
            "model_name": item["model_name"],
            "llm_output": validated.model_dump()
        })

    except ValidationError as error:
```

```

        rejected_items.append({
            "article_id": item["article_id"],
            "title": item["title"],
            "url": item["url"],
            "error": error.errors(),
            "llm_output": item["llm_output"]
        })

OUTPUT_FILE.write_text(
    json.dumps(valid_items, indent=2,
ensure_ascii=False),
    encoding="utf-8"
)

REJECTED_FILE.write_text(
    json.dumps(rejected_items, indent=2,
ensure_ascii=False),
    encoding="utf-8"
)

print(f"Valid      : {len(valid_items)}")
print(f"Rejected   : {len(rejected_items)}")
print(f"Output      : {OUTPUT_FILE}")
print(f"Rejected     : {REJECTED_FILE}")

```

Jalankan:

```
python scripts/bab07/05_validate_llm_json.py
```

Penjelasan:

- **Entity** adalah schema entitas.
- **LLMExtraction** adalah schema output LLM.
- **Field(pattern=...)** membatasi nilai kategori.
- Output yang tidak sesuai schema ditolak.

Pydantic berbasis *type hints* dan dapat melakukan validasi serta serialisasi data dengan model yang jelas. ([Pydantic](#))

7.16 Praktik 6 — Membuat Tabel PostgreSQL untuk Hasil LLM

Pastikan PostgreSQL dari Bab 5 berjalan:

```
cd database/bab05
docker compose up -d
cd ../..
```

Buat schema:

```
nano database/bab05/schema_llm_extractions.sql
```

Isi:

```
CREATE TABLE IF NOT EXISTS llm_extractions (
    id SERIAL PRIMARY KEY,
    article_id INTEGER,
    title TEXT,
    url TEXT NOT NULL,
    model_name TEXT NOT NULL,
    summary TEXT,
    topics_json JSONB,
    entities_json JSONB,
    sensitive_data_detected BOOLEAN,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(url, model_name)
);

CREATE INDEX IF NOT EXISTS idx_llm_extractions_url
ON llm_extractions(url);

CREATE INDEX IF NOT EXISTS idx_llm_extractions_model
ON llm_extractions(model_name);
```

Penjelasan:

- **JSONB** dipakai untuk menyimpan list topik dan entitas.
- **UNIQUE(url, model_name)** mencegah duplikasi hasil model yang sama.
- **created_at** mencatat waktu insert.
- Index membantu pencarian.

PostgreSQL mendukung **INSERT ... ON CONFLICT** untuk menentukan tindakan alternatif saat terjadi konflik unique constraint, misalnya **DO NOTHING** atau **DO UPDATE**. ([PostgreSQL](#))

7.17 Praktik 7 — Membuat Tabel dari Python

Buat file:

```
nano scripts/bab07/06_create_llm_table.py
```

Isi:

```
import os
from pathlib import Path
import psycopg2
from dotenv import load_dotenv

load_dotenv()

SCHEMA_FILE =
Path("database/bab05/schema_llm_extractions.sql")

connection = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

cursor = connection.cursor()
schema_sql = SCHEMA_FILE.read_text(encoding="utf-8")

cursor.execute(schema_sql)

connection.commit()
cursor.close()
connection.close()

print("Tabel llm_extractions siap.")
```

Jalankan:

```
python scripts/bab07/06_create_llm_table.py
```

7.18 Praktik 8 — Insert Hasil LLM ke PostgreSQL

Buat file:

```
nano scripts/bab07/07_insert_llm_to_postgres.py
```

Isi:

```
import os
import json
from pathlib import Path
import psycopg2
from psycopg2.extras import Json
from dotenv import load_dotenv

load_dotenv()

INPUT_FILE =
Path("data/clean/bab07/llm_validated_extractions.json")

items =
json.loads(INPUT_FILE.read_text(encoding="utf-8"))

connection = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

cursor = connection.cursor()

sql = """
INSERT INTO llm_extractions
(article_id, title, url, model_name, summary,
topics_json, entities_json, sensitive_data_detected)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (url, model_name) DO UPDATE SET
summary = EXCLUDED.summary,
topics_json = EXCLUDED.topics_json,
entities_json = EXCLUDED.entities_json,
sensitive_data_detected =
EXCLUDED.sensitive_data_detected,
created_at = CURRENT_TIMESTAMP
"""

for item in items:
    output = item["llm_output"]
```

```

        cursor.execute(
            sql,
            (
                item["article_id"],
                item["title"],
                item["url"],
                item["model_name"],
                output["summary"],
                Json(output["topics"]),
                Json(output["entities"]),
                output["sensitive_data_detected"],
            )
        )

    connection.commit()
    cursor.close()
    connection.close()

    print(f"Insert hasil LLM selesai: {len(items)} item")

```

Jalankan:

```
python scripts/bab07/07_insert_llm_to_postgres.py
```

Penjelasan:

- `Json(...)` mengirim list/dict Python sebagai JSONB.
- `ON CONFLICT` mencegah duplikasi.
- Jika data sudah ada, hasil diperbarui.

7.19 Praktik 9 — Query Hasil LLM

Buat file:

```
nano scripts/bab07/08_query_llm_results.py
```

Isi:

```
import os
import psycopg2
from dotenv import load_dotenv

load_dotenv()

connection = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

cursor = connection.cursor()

print("Total hasil LLM:")
cursor.execute("SELECT COUNT(*) FROM llm_extractions")
print(cursor.fetchone()[0])

print("\nRingkasan artikel:")
cursor.execute("""
SELECT title, summary
FROM llm_extractions
ORDER BY id ASC
""")

for row in cursor.fetchall():
    print("-" * 60)
    print("Title   :", row[0])
    print("Summary:", row[1])

print("\nArtikel yang punya entitas:")
cursor.execute("""
SELECT title, entities_json
FROM llm_extractions
WHERE jsonb_array_length(entities_json) > 0
""")

for row in cursor.fetchall():
    print("-" * 60)
    print("Title   :", row[0])
```

```
print("Entities:", row[1])

cursor.close()
connection.close()
```

Jalankan:

```
python scripts/bab07/08_query_llm_results.py
```

Output yang diharapkan:

- Total hasil LLM.
- Ringkasan tiap artikel.
- Daftar entitas yang berhasil diekstrak.

7.20 Praktik 10 — Contoh Anti-Halusinasi Sederhana

Buat file:

```
nano scripts/bab07/09_anti_hallucination_check.py
```

Isi:

```
import json
from pathlib import Path

SOURCE_FILE =
Path("data/clean/bab07/articles_for_llm.jsonl")
LLM_FILE =
Path("data/clean/bab07/llm_validated_extractions.json")
REPORT_FILE =
Path("data/clean/bab07/anti_hallucination_report.json")

source_texts = {}

for line in
SOURCE_FILE.read_text(encoding="utf-8").splitlines():
    item = json.loads(line)
    source_texts[item["id"]] =
item["clean_text"].lower()

llm_items =
json.loads(LLM_FILE.read_text(encoding="utf-8"))

report = []

for item in llm_items:
    article_id = item["article_id"]
    source_text = source_texts.get(article_id, "")
    entities = item["llm_output"]["entities"]

    suspicious_entities = []

    for entity in entities:
        entity_name = entity["name"].lower()

        if entity_name not in source_text:
            suspicious_entities.append(entity)

    report.append({
        "article_id": article_id,
        "title": item["title"],
        "suspicious_entities": suspicious_entities,
        "suspicious_count": len(suspicious_entities)
    })
```

```
REPORT_FILE.write_text(  
    json.dumps(report, indent=2, ensure_ascii=False),  
    encoding="utf-8"  
)  
  
print(json.dumps(report, indent=2, ensure_ascii=False))  
print(f"\nLaporan anti-halusinasi disimpan ke:  
{REPORT_FILE}")
```

Jalankan:

```
python scripts/bab07/09_anti_hallucination_check.py
```

Catatan:

Ini bukan pemeriksa sempurna. Namun, ini memberi contoh awal:

- Entitas yang tidak muncul di teks sumber perlu dicurigai.
- Output LLM harus dibandingkan dengan sumber.
- Audit sederhana lebih baik daripada percaya penuh.

Anti-halusinasi bukan satu teknik tunggal. Ia adalah disiplin pipeline.

7.21 Crawl4AI untuk Konten yang Lebih Bersih

Pada scraping biasa, HTML sering berantakan. LLM lebih nyaman membaca teks bersih atau Markdown.

Crawl4AI relevan karena dapat menghasilkan Markdown bersih, mendukung structured extraction dengan CSS, XPath, atau LLM-based extraction, dan dirancang untuk pipeline RAG serta LLM. ([Crawl4AI Documentation](#))

Instalasi:

```
pip install crawl4ai
```

Contoh script awal:

```
nano scripts/bab07/10_crawl4ai_basic.py
```

Isi:

```
import asyncio
from pathlib import Path
from crawl4ai import AsyncWebCrawler

OUTPUT_FILE =
Path("data/clean/bab07/crawl4ai_markdown.md")

async def main():
    async with AsyncWebCrawler() as crawler:
        result = await
crawler.arun(url="https://example.org")
        OUTPUT_FILE.write_text(result.markdown,
encoding="utf-8")
        print(f"Markdown disimpan ke: {OUTPUT_FILE}")

if __name__ == "__main__":
    asyncio.run(main())
```

Jalankan:

```
python scripts/bab07/10_crawl4ai_basic.py
```

Catatan:

- Gunakan URL latihan yang legal.
- Jangan mengambil data sensitif.
- Jangan melakukan crawling agresif.
- Simpan Markdown sebagai input LLM.

Crawl4AI bukan alasan untuk mengabaikan etika. Ia hanya membantu membuat konten lebih siap dipakai LLM.

7.22 Struktur Akhir Folder Bab 7

Struktur akhir:

```
webscraping-llm-database/
├── data/
│   ├── clean/
│   │   └── bab07/
│   │       ├── articles_for_llm.jsonl
│   │       ├── article_summaries.json
│   │       ├── llm_raw_extractions.json
│   │       ├── llm_parsed_extractions.json
│   │       ├── llm_validated_extractions.json
│   │       ├── anti_hallucination_report.json
│   │       └── crawl4ai_markdown.md
│   └── rejected/
│       └── bab07/
│           ├── llm_parse_failed.json
│           └── llm_validation_failed.json
├── database/
│   └── bab05/
│       └── schema_llm_extractions.sql
└── scripts/
    └── bab07/
        ├── 01_call_ollama.py
        ├── 02_summarize_article.py
        ├── 03_extract_entities_json.py
        ├── 04_parse_llm_json.py
        ├── 05_validate_llm_json.py
        ├── 06_create_llm_table.py
        ├── 07_insert_llm_to_postgres.py
        ├── 08_query_llm_results.py
        ├── 09_anti_hallucination_check.py
        └── 10_crawl4ai_basic.py
```

7.23 Checklist Praktik Bab 7

Mahasiswa dianggap selesai Bab 7 jika sudah mampu:

- ☐ Memahami perbedaan scraping biasa dan LLM-assisted extraction
- ☐ Menjalankan Ollama lokal
- ☐ Memilih model lokal ringan
- ☐ Memanggil Ollama dari Python
- ☐ Membuat prompt ringkasan artikel
- ☐ Membuat prompt ekstraksi JSON
- ☐ Menyimpan output mentah LLM
- ☐ Parsing JSON output LLM
- ☐ Validasi output LLM dengan Pydantic
- ☐ Menyimpan output gagal ke rejected file
- ☐ Membuat tabel llm_extractions di PostgreSQL
- ☐ Insert hasil LLM ke PostgreSQL
- ☐ Query hasil ringkasan dan entitas
- ☐ Membuat cek anti-halusinasi sederhana
- ☐ Mencoba Crawl4AI untuk menghasilkan Markdown bersih

7.24 Kesalahan Umum Bab 7

1. Menganggap LLM selalu benar

Masalah:

- Output terlihat meyakinkan.
- Tetapi bisa berisi fakta yang tidak ada di sumber.

Solusi:

Simpan sumber, simpan output mentah, validasi JSON, dan cek entitas terhadap teks.

2. Tidak meminta JSON valid

Masalah:

LLM menjawab dengan teks bebas.

Solusi:

Prompt harus menyatakan: output hanya JSON valid.

3. Tidak menyimpan output mentah

Masalah:

Jika validasi gagal, tidak ada bahan audit.

Solusi:

Simpan raw_llm_output.

4. Tidak memakai schema

Masalah:

Field berubah-ubah.

Solusi:

Gunakan Pydantic.

5. Memakai LLM untuk semua hal

Masalah:

Pipeline menjadi lambat dan mahal secara komputasi.

Solusi:

Gunakan selector untuk field jelas. Gunakan LLM untuk interpretasi.

6. Tidak menyimpan nama model

Masalah:

Hasil sulit direproduksi.

Solusi:

Simpan model_name.

7. Tidak melakukan upsert

Masalah:

Data LLM duplikat.

Solusi:

ON CONFLICT (url, model_name) DO UPDATE

7.25 Ringkasan Bab

Bab ini memperkenalkan *open source LLM* sebagai lapisan interpretasi dalam pipeline scraping. Mahasiswa belajar bahwa LLM bukan pengganti scraper, tetapi alat untuk membantu ringkasan, ekstraksi entitas, klasifikasi topik, dan konversi teks menjadi JSON. Scraper tetap bertugas mengambil data dari sumber. LLM bertugas membantu memberi struktur makna. Validasi tetap menjadi pagar utama.

Pada bagian praktik, mahasiswa menjalankan Ollama lokal, memanggil model dari Python, membuat prompt ringkasan, membuat prompt ekstraksi JSON, menyimpan output mentah, melakukan parsing JSON, memvalidasi output dengan Pydantic, menyimpan hasil valid ke PostgreSQL, dan membuat cek anti-halusinasi sederhana. Crawl4AI juga diperkenalkan sebagai alat untuk menghasilkan Markdown yang lebih bersih dan ramah untuk LLM.

Output akhir Bab 7:

- Ollama berjalan lokal.
- Script interaksi Python ke LLM.
- Prompt ekstraksi JSON.
- File output mentah dan tervalidasi.
- Tabel `llm_extractions`.
- Hasil ekstraksi tersimpan di PostgreSQL.
- Contoh anti-halusinasi sederhana.
- Contoh penggunaan Crawl4AI untuk Markdown bersih.

Pesan strategis Bab 7:

LLM membuat scraping lebih cerdas, tetapi juga lebih berisiko. Pipeline yang profesional tidak hanya bertanya ke LLM, tetapi membatasi, memvalidasi, mencatat, dan mengaudit setiap output yang dihasilkan.

Bab 8 — Playwright dan Scraping Website Dinamis

Gunakan Playwright hanya ketika konten memang muncul setelah proses JavaScript; jangan memakai browser automation untuk masalah yang cukup diselesaikan dengan HTTP scraping biasa.

Bab 8 membahas website modern yang tidak cukup diambil dengan `requests` karena kontennya muncul melalui JavaScript. Bab ini menyiapkan mahasiswa membangun pipeline: **Playwright → Clean → Database**.

Playwright menyediakan API untuk mengendalikan halaman browser, melakukan navigasi, mengambil screenshot, mengisi form, klik tombol, dan membaca konten hasil render. Dokumentasi Playwright Python menjelaskan bahwa objek `Page` merepresentasikan satu tab browser dan menyediakan metode untuk berinteraksi dengan halaman. ([Playwright](#))

8.1 Mengapa Website JavaScript-Heavy Sulit di-Scrape?

Pada Bab 2 sampai Bab 4, mahasiswa memakai `requests`, BeautifulSoup, Scrapy, dan Pandas. Alat tersebut sangat baik untuk halaman statis. Masalah muncul ketika halaman modern tidak langsung mengirim data lengkap di HTML awal.

Website modern sering bekerja seperti ini:

```
Browser membuka halaman
↓
HTML awal dikirim
↓
JavaScript berjalan
↓
Data diminta dari server
↓
Konten baru muncul di halaman
```

Jika memakai `requests`, yang diterima sering hanya HTML awal.

Gejala umum:

- `requests.get()` berhasil.
- Status code `200`.
- Tetapi data yang terlihat di browser tidak ada di HTML.
- Tag utama kosong.
- Daftar artikel tidak muncul.
- Tombol *load more* harus diklik dulu.
- Hasil pencarian muncul setelah form diisi.

Contoh HTML awal:

```
<div id="app"></div>
<script src="/assets/app.js"></script>
```

Bagi manusia, halaman terlihat lengkap. Bagi `requests`, datanya belum ada.

Jika data muncul setelah JavaScript berjalan, kita perlu alat yang bisa menjalankan halaman seperti browser.

8.2 HTTP Scraping vs Browser Automation

HTTP scraping

HTTP scraping memakai alat seperti:

- `requests`
- BeautifulSoup
- Scrapy

Cocok untuk:

- HTML statis.
- Tabel publik sederhana.
- Halaman artikel biasa.
- Link dan metadata yang sudah ada di source HTML.
- Scraping cepat dan ringan.

Kelebihan:

- Cepat.
- Ringan.
- Lebih mudah diaudit.
- Lebih hemat resource.

Kekurangan:

- Tidak menjalankan JavaScript.
- Tidak bisa klik tombol.
- Tidak bisa menunggu konten render.
- Tidak cocok untuk halaman interaktif berat.

Browser automation

Browser automation memakai alat seperti Playwright.

Cocok untuk:

- Konten muncul setelah JavaScript.
- Perlu klik tombol.
- Perlu mengisi form.
- Perlu menunggu selector.
- Perlu screenshot debugging.
- Perlu melihat hasil render final.

Kelebihan:

- Bisa membaca halaman seperti browser.
- Bisa klik, isi form, scroll, dan screenshot.
- Bisa menunggu elemen muncul.

Kekurangan:

- Lebih berat.
- Lebih lambat.
- Lebih mudah membebani komputer.
- Lebih berisiko jika dipakai sembarangan.

Strategi profesional: mulai dari HTTP scraping. Gunakan Playwright hanya jika benar-benar diperlukan.

8.3 Apa Itu Headless Browser?

Headless browser adalah browser yang berjalan tanpa tampilan grafis. Ia tetap dapat membuka halaman, menjalankan JavaScript, membaca DOM, klik tombol, mengisi form, dan mengambil screenshot.

Dalam Playwright:

```
browser = p.chromium.launch(headless=True)
```

Makna:

- **headless=True**: browser berjalan tanpa jendela terlihat.
- **headless=False**: browser terlihat, cocok untuk debugging awal.

Untuk belajar, mahasiswa bisa memakai:

```
headless=False
```

Agar dapat melihat apa yang terjadi.

Untuk pipeline otomatis, gunakan:

```
headless=True
```

Agar lebih ringan dan cocok dijalankan dari script.

Mode terlihat untuk belajar. Mode headless untuk pipeline.

8.4 Waiting Strategy: Jangan Asal Sleep

Pada website dinamis, masalah utama adalah waktu. Konten tidak selalu muncul langsung setelah halaman dibuka.

Strategi menunggu yang umum:

- Tunggu halaman selesai navigasi.
- Tunggu selector muncul.
- Tunggu elemen terlihat.
- Tunggu jaringan relatif diam.
- Tunggu setelah klik tombol.

Playwright memiliki konsep *locator* yang menjadi bagian penting dari *auto-waiting* dan *retry-ability*. Dokumentasi Playwright menjelaskan bahwa locator merepresentasikan cara menemukan elemen pada halaman, dan menjadi pusat mekanisme auto-waiting. ([Playwright](#))

Contoh lebih baik:

```
page.locator(".article-card").first.wait_for()
```

Daripada:

```
page.wait_for_timeout(5000)
```

Mengapa?

- `wait_for_timeout(5000)` hanya menunggu 5 detik.
- Jika konten muncul 1 detik, waktu terbuang.
- Jika konten muncul 8 detik, data tetap kosong.
- Menunggu selector lebih logis.

Untuk navigasi dan load state, dokumentasi Playwright menjelaskan bahwa `page.goto()` menunggu event load, dan halaman modern bisa melakukan redirect atau proses client-side. ([Playwright](#))

Jangan menunggu waktu. Tunggu kondisi.

8.5 Screenshot dan Debugging

Salah satu kekuatan Playwright adalah screenshot. Screenshot membantu mahasiswa memahami:

- Apakah halaman benar-benar terbuka.
- Apakah konten sudah muncul.
- Apakah tombol terlihat.
- Apakah selector salah.
- Apakah halaman menampilkan error.
- Apakah layout berubah.

Playwright mendukung screenshot halaman, termasuk *full page screenshot*.

Dokumentasi resminya menunjukkan penggunaan `page.screenshot()` untuk menyimpan screenshot ke file. ([Playwright](#))

Contoh:

```
page.screenshot(path="logs/bab08/debug.png",  
full_page=True)
```

Screenshot adalah bukti visual. Dalam pipeline profesional, screenshot debugging sangat membantu saat scraping gagal.

Jika data kosong, jangan menebak. Ambil screenshot. Lihat apa yang sebenarnya terjadi.

8.6 Risiko Scraping Login Page dan Data Privat

Website dinamis sering memiliki:

- Login page.
- Dashboard pribadi.
- Data akun.
- Data pelanggan.
- Data transaksi.
- Proteksi akses.
- Captcha.
- Token sesi.

Batas etika sangat jelas:

- Jangan mengambil data privat.
- Jangan bypass proteksi.
- Jangan masuk ke area login tanpa izin.
- Jangan mengambil data akun orang lain.
- Jangan mengakali pembatasan akses.
- Jangan melakukan scraping yang membebani server.
- Jangan menyimpan credential dalam kode.
- Jangan mengambil data sensitif untuk latihan.

Playwright sangat kuat. Karena itu, tanggung jawabnya juga lebih besar.

Kemampuan teknis bukan izin etis.

Bagian Praktik

8.7 Persiapan Folder Bab 8

Masuk ke folder proyek:

```
cd webscraping-llm-database
source .venv/bin/activate
```

Buat folder:

```
mkdir -p scripts/bab08
mkdir -p data/raw/bab08
mkdir -p data/clean/bab08
mkdir -p logs/bab08/screenshots
```

Penjelasan:

- `scripts/bab08` menyimpan script Playwright.
- `data/raw/bab08` menyimpan HTML atau file latihan.
- `data/clean/bab08` menyimpan data hasil scraping.
- `logs/bab08/screenshots` menyimpan screenshot debugging.

Instal paket:

```
pip install playwright beautifulsoup4 lxml pandas
python-dotenv psycopg2-binary
```

Instal browser runtime Playwright:

```
playwright install
```

Penjelasan:

- `playwright` adalah library browser automation.
- `playwright install` memasang runtime browser yang diperlukan Playwright.
- `beautifulsoup4` dan `lxml` dipakai untuk parsing HTML hasil render.
- `pandas` dipakai untuk menyimpan dan membaca CSV.
- `psycopg2-binary` dipakai untuk koneksi PostgreSQL.

8.8 Membuat Halaman Dinamis Lokal untuk Latihan

Agar praktik aman, kita membuat halaman HTML lokal yang memakai JavaScript. Ini membuat mahasiswa belajar konsep website dinamis tanpa membebani website publik.

Buat file:

```
nano data/raw/bab08/dynamic_articles.html
```

Isi:

```
<!DOCTYPE html>
<html>
<head>
  <title>Portal Artikel Dinamis</title>
</head>
<body>
  <h1>Portal Artikel Dinamis</h1>

  <input id="searchBox" type="text" placeholder="Cari
artikel">
  <button id="searchButton">Cari</button>

  <div id="articles"></div>

  <button id="loadMore">Load More</button>

  <script>
    const allArticles = [
      {
        title: "Data Engineering untuk Mahasiswa",
        url:
"https://kampus.example/artikel/data-engineering",
        date: "2026-05-27",
        author: "Admin Kampus",
        summary: "Pipeline data membantu mahasiswa
mengubah data mentah menjadi informasi."
      },
      {
        title: "Web Scraping Etis",
        url:
"https://kampus.example/artikel/scraping-etis",
        date: "2026-05-28",
        author: "Tim Data",
        summary: "Scraping harus memperhatikan
legalitas, privasi, dan beban server."
      },
      {
        title: "LLM Lokal untuk Ekstraksi Informasi",
```

```

        url: "https://kampus.example/artikel/llm-lokal",
        date: "2026-05-29",
        author: "Lab AI",
        summary: "Model lokal dapat membantu ringkasan,
klasifikasi, dan JSON."
    },
    {
        title: "Database untuk Data Scraping",
        url:
"https://kampus.example/artikel/database-scraping",
        date: "2026-05-30",
        author: "Tim Database",
        summary: "Database menjaga data scraping tetap
rapi dan mudah diaudit."
    }
];

let visibleCount = 2;

function renderArticles(keyword = "") {
    const container =
document.getElementById("articles");
    container.innerHTML = "";

    const filtered = allArticles
        .filter(item =>
item.title.toLowerCase().includes(keyword.toLowerCase())
        )
        .slice(0, visibleCount);

    filtered.forEach(item => {
        const article =
document.createElement("article");
        article.className = "article-card";

        article.innerHTML = `
            <h2 class="article-title"><a
href="${item.url}">${item.title}</a></h2>
            <p class="article-date">${item.date}</p>
            <p class="article-author">${item.author}</p>
            <p class="article-summary">${item.summary}</p>
        `;

        container.appendChild(article);
    });
}

```

```

document.getElementById("loadMore").addEventListener("click", () => {
    visibleCount = allArticles.length;

    renderArticles(document.getElementById("searchBox").value);
});

document.getElementById("searchButton").addEventListener(
    "click", () => {

    renderArticles(document.getElementById("searchBox").value);
    });

    setTimeout(() => {
        renderArticles();
    }, 1000);
</script>
</body>
</html>

```

Halaman ini mensimulasikan:

- Konten muncul setelah JavaScript berjalan.
- Tombol *load more*.
- Form pencarian.
- Artikel hasil render.

8.9 Praktik 1 — Menjalankan Browser Headless

Buat file:

```
nano scripts/bab08/01_open_dynamic_page.py
```

Isi:

```
from pathlib import Path
from playwright.sync_api import sync_playwright

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
SCREENSHOT_FILE =
Path("logs/bab08/screenshots/01_page_open.png")

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    page.goto(HTML_FILE.as_uri())
    page.screenshot(path=str(SCREENSHOT_FILE),
full_page=True)

    print("Title halaman:", page.title())
    print(f"Screenshot disimpan ke: {SCREENSHOT_FILE}")

    browser.close()
```

Jalankan:

```
python scripts/bab08/01_open_dynamic_page.py
```

Penjelasan:

- `sync_playwright()` menjalankan Playwright mode sinkron.
- `p.chromium.launch(headless=True)` membuka browser tanpa tampilan grafis.
- `browser.new_page()` membuat tab baru.
- `page.goto()` membuka file HTML lokal.
- `page.screenshot()` menyimpan screenshot.
- `browser.close()` menutup browser.

8.10 Praktik 2 — Menunggu Konten JavaScript Selesai

Buat file:

```
nano scripts/bab08/02_wait_for_articles.py
```

Isi:

```
from pathlib import Path
from playwright.sync_api import sync_playwright

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
SCREENSHOT_FILE =
Path("logs/bab08/screenshots/02_articles_loaded.png")

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    page.goto(HTML_FILE.as_uri())

    page.locator(".article-card").first.wait_for(timeout=5000)

    article_count =
page.locator(".article-card").count()

    page.screenshot(path=str(SCREENSHOT_FILE),
full_page=True)

    print(f"Jumlah artikel terlihat: {article_count}")
    print(f"Screenshot disimpan ke: {SCREENSHOT_FILE}")

    browser.close()
```

Jalankan:

```
python scripts/bab08/02_wait_for_articles.py
```

Penjelasan:

- `.article-card` adalah selector artikel.
- `.first().wait_for()` menunggu artikel pertama muncul.
- `timeout=5000` berarti maksimal menunggu 5 detik.
- `count()` menghitung jumlah artikel yang terlihat.

Ini lebih baik daripada asal menunggu 5 detik tanpa kondisi.

8.11 Praktik 3 — Mengambil Data Hasil Render

Buat file:

```
nano scripts/bab08/03_extract_rendered_articles.py
```

Isi:

```
import json
from pathlib import Path
from playwright.sync_api import sync_playwright

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
OUTPUT_FILE =
Path("data/clean/bab08/rendered_articles.json")
SCREENSHOT_FILE =
Path("logs/bab08/screenshots/03_rendered_articles.png")

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    page.goto(HTML_FILE.as_uri())

page.locator(".article-card").first.wait_for(timeout=500
0)

articles = []

cards = page.locator(".article-card")
total = cards.count()

for index in range(total):
    card = cards.nth(index)

    title_link = card.locator(".article-title a")
    title = title_link.inner_text()
    url = title_link.get_attribute("href")
    date =
card.locator(".article-date").inner_text()
    author =
card.locator(".article-author").inner_text()
    summary =
card.locator(".article-summary").inner_text()

    articles.append({
        "title": title,
        "url": url,
        "date": date,
```

```

        "author": author,
        "summary": summary,
        "source_url": HTML_FILE.as_uri()
    })

    OUTPUT_FILE.write_text(
        json.dumps(articles, indent=2,
ensure_ascii=False),
        encoding="utf-8"
    )

    page.screenshot(path=str(SCREENSHOT_FILE),
full_page=True)

    print(f"Jumlah artikel diekstrak: {len(articles)}")
    print(f"Output: {OUTPUT_FILE}")

    browser.close()

```

Jalankan:

```
python scripts/bab08/03_extract_rendered_articles.py
```

Output:

```
data/clean/bab08/rendered_articles.json
```

Pelajaran:

- Playwright membaca DOM setelah JavaScript berjalan.
- Data diambil dari elemen hasil render.
- Screenshot disimpan sebagai bukti debugging.

8.12 Praktik 4 — Klik Tombol Load More

Buat file:

```
nano scripts/bab08/04_click_load_more.py
```

Isi:

```
import json
from pathlib import Path
from playwright.sync_api import sync_playwright

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
OUTPUT_FILE =
Path("data/clean/bab08/rendered_articles_load_more.json"
)
SCREENSHOT_FILE =
Path("logs/bab08/screenshots/04_after_load_more.png")

def extract_articles(page):
    articles = []
    cards = page.locator(".article-card")
    total = cards.count()

    for index in range(total):
        card = cards.nth(index)
        title_link = card.locator(".article-title a")

        articles.append({
            "title": title_link.inner_text(),
            "url": title_link.get_attribute("href"),
            "date":
card.locator(".article-date").inner_text(),
            "author":
card.locator(".article-author").inner_text(),
            "summary":
card.locator(".article-summary").inner_text(),
            "source_url": HTML_FILE.as_uri()
        })

    return articles

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    page.goto(HTML_FILE.as_uri())
```

```

page.locator(".article-card").first.wait_for(timeout=500
0)

    before_count = page.locator(".article-card").count()

    page.locator("#loadMore").click()

page.locator(".article-card").nth(3).wait_for(timeout=50
00)

    after_count = page.locator(".article-card").count()

    articles = extract_articles(page)

    OUTPUT_FILE.write_text(
        json.dumps(articles, indent=2,
ensure_ascii=False),
        encoding="utf-8"
    )

    page.screenshot(path=str(SCREENSHOT_FILE),
full_page=True)

    print(f"Sebelum load more: {before_count}")
    print(f"Sesudah load more: {after_count}")
    print(f"Output: {OUTPUT_FILE}")

    browser.close()

```

Jalankan:

```
python scripts/bab08/04_click_load_more.py
```

Penjelasan:

- `page.locator("#loadMore").click()` klik tombol.
- `.nth(3).wait_for()` menunggu artikel keempat muncul.
- Setelah klik, jumlah artikel bertambah.

Dokumentasi Playwright Actions menjelaskan bahwa locator dapat dipakai untuk berinteraksi dengan elemen, termasuk klik dan pengisian input. ([Playwright](#))

8.13 Praktik 5 — Mengisi Form Pencarian

Buat file:

```
nano scripts/bab08/05_fill_search_form.py
```

Isi:

```
import json
from pathlib import Path
from playwright.sync_api import sync_playwright

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
OUTPUT_FILE =
Path("data/clean/bab08/search_results.json")
SCREENSHOT_FILE =
Path("logs/bab08/screenshots/05_search_results.png")

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    page.goto(HTML_FILE.as_uri())

page.locator(".article-card").first.wait_for(timeout=500
0)

page.locator("#searchBox").fill("LLM")
page.locator("#searchButton").click()

page.locator(".article-card").first.wait_for(timeout=500
0)

results = []
cards = page.locator(".article-card")

for index in range(cards.count()):
    card = cards.nth(index)
    title_link = card.locator(".article-title a")

    results.append({
        "title": title_link.inner_text(),
        "url": title_link.get_attribute("href"),
        "date":
card.locator(".article-date").inner_text(),
        "author":
card.locator(".article-author").inner_text(),
```

```

        "summary":
card.locator(".article-summary").inner_text(),
        "query": "LLM",
        "source_url": HTML_FILE.as_uri()
    })

    OUTPUT_FILE.write_text(
        json.dumps(results, indent=2,
ensure_ascii=False),
        encoding="utf-8"
    )

    page.screenshot(path=str(SCREENSHOT_FILE),
full_page=True)

    print(f"Jumlah hasil pencarian: {len(results)}")
    print(f"Output: {OUTPUT_FILE}")

    browser.close()

```

Jalankan:

```
python scripts/bab08/05_fill_search_form.py
```

Penjelasan:

- `fill("LLM")` mengisi input pencarian.
- `click()` menekan tombol cari.
- Playwright membaca hasil yang sudah dirender ulang.

Dokumentasi Playwright menjelaskan bahwa `locator.fill()` adalah cara mudah mengisi field seperti `<input>`, `<textarea>`, dan elemen `contenteditable`. ([Playwright](#))

8.14 Praktik 6 — Menyimpan Data ke CSV

Buat file:

```
nano scripts/bab08/06_json_to_csv.py
```

Isi:

```
from pathlib import Path
import pandas as pd

INPUT_FILE =
Path("data/clean/bab08/rendered_articles_load_more.json"
)
OUTPUT_FILE =
Path("data/clean/bab08/rendered_articles.csv")

df = pd.read_json(INPUT_FILE)

df["title"] =
df["title"].fillna("").astype(str).str.split().str.join(
" ")
df["summary"] =
df["summary"].fillna("").astype(str).str.split().str.joi
n(" ")
df["author"] =
df["author"].fillna("").astype(str).str.split().str.join
(" ")

df = df.drop_duplicates(subset=["url"])

df.to_csv(OUTPUT_FILE, index=False)

print(f"CSV disimpan ke: {OUTPUT_FILE}")
print(df)
```

Jalankan:

```
python scripts/bab08/06_json_to_csv.py
```

Fungsi script:

- Membaca JSON hasil Playwright.
- Membersihkan teks.
- Menghapus duplikasi URL.
- Menyimpan CSV.

8.15 Praktik 7 — Membuat Content Hash dan Timestamp

Buat file:

```
nano scripts/bab08/07_prepare_for_database.py
```

Isi:

```
from datetime import datetime
from pathlib import Path
import hashlib
import pandas as pd

INPUT_FILE =
Path("data/clean/bab08/rendered_articles.csv")
OUTPUT_FILE =
Path("data/clean/bab08/rendered_articles_ready.csv")

def make_hash(*values):
    text = "|".join(str(value or "") for value in
values)
    return
hashlib.sha256(text.encode("utf-8")).hexdigest()

df = pd.read_csv(INPUT_FILE).fillna("")

df["scraped_at"] = datetime.utcnow().isoformat()
df["content_hash"] = df.apply(
    lambda row: make_hash(row["title"], row["url"],
row["summary"]),
    axis=1
)
df["validation_status"] = "valid"

df.to_csv(OUTPUT_FILE, index=False)

print(f"File siap database: {OUTPUT_FILE}")
print(df[["title", "url", "content_hash"]])
```

Jalankan:

```
python scripts/bab08/07_prepare_for_database.py
```

Field penting:

- **scraped_at**
- **content_hash**
- **validation_status**

Ini menyambungkan Bab 8 dengan Bab 5 dan Bab 6.

8.16 Praktik 8 — Simpan Data ke SQLite

Gunakan database SQLite dari Bab 5. Jika belum ada, buat kembali schema Bab 5.

Buat file:

```
nano scripts/bab08/08_insert_to_sqlite.py
```

Isi:

```
from pathlib import Path
import sqlite3
import pandas as pd

DB_FILE = Path("database/bab05/scraping.db")
INPUT_FILE =
Path("data/clean/bab08/rendered_articles_ready.csv")

df = pd.read_csv(INPUT_FILE).fillna("")

connection = sqlite3.connect(DB_FILE)
cursor = connection.cursor()

sql = """
INSERT OR IGNORE INTO scraped_items
(title, url, date, author, summary, source_url,
scraped_at, content_hash, validation_status)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
"""

for _, row in df.iterrows():
    cursor.execute(
        sql,
        (
            row["title"],
            row["url"],
            row["date"],
            row["author"],
            row["summary"],
            row["source_url"],
            row["scraped_at"],
            row["content_hash"],
            row["validation_status"],
        )
    )

connection.commit()
connection.close()
```

```
print(f"Data Playwright dicoba masuk SQLite: {len(df)}  
baris")
```

Jalankan:

```
python scripts/bab08/08_insert_to_sqlite.py
```

Cek isi database:

```
sqlite3 database/bab05/scraping.db "SELECT title, url  
FROM scraped_items;"
```

Output menunjukkan data hasil render sudah masuk database.

8.17 Praktik 9 — Logging Screenshot Jika Gagal

Dalam scraping dinamis, error sering terjadi. Selector berubah, JavaScript lambat, tombol tidak muncul, atau halaman kosong. Maka screenshot saat gagal sangat penting.

Buat file:

```
nano scripts/bab08/09_debug_screenshot_on_error.py
```

Isi:

```
from pathlib import Path
from playwright.sync_api import sync_playwright,
TimeoutError as PlaywrightTimeoutError

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
SUCCESS_SCREENSHOT =
Path("logs/bab08/screenshots/09_success.png")
ERROR_SCREENSHOT =
Path("logs/bab08/screenshots/09_error.png")
LOG_FILE = Path("logs/bab08/playwright_error.log")

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    try:
        page.goto(HTML_FILE.as_uri())

        page.locator(".selector-yang-salah").first.wait_for(time
out=3000)

        page.screenshot(path=str(SUCCESS_SCREENSHOT),
full_page=True)
        print("Berhasil.")

    except PlaywrightTimeoutError as error:
        page.screenshot(path=str(ERROR_SCREENSHOT),
full_page=True)

        LOG_FILE.write_text(
            f"Timeout error: {error}\nScreenshot:
{ERROR_SCREENSHOT}\n",
            encoding="utf-8"
        )

        print("Terjadi timeout.")
```

```
print(f"Screenshot error: {ERROR_SCREENSHOT}")
print(f"Log: {LOG_FILE}")

finally:
    browser.close()
```

Jalankan:

```
python scripts/bab08/09_debug_screenshot_on_error.py
```

Script ini sengaja memakai selector salah agar mahasiswa melihat mekanisme debugging.

Pelajaran:

- Error harus dicatat.
- Screenshot membantu memahami kondisi halaman.
- Jangan hanya membaca stack trace.
- Lihat bukti visual.

8.18 Praktik 10 — Pipeline Playwright → Clean → Database

Sekarang gabungkan proses menjadi satu script pipeline.

Buat file:

```
nano
scripts/bab08/10_playwright_clean_database_pipeline.py
```

Isi:

```
from datetime import datetime
from pathlib import Path
import hashlib
import sqlite3
import pandas as pd
from playwright.sync_api import sync_playwright

HTML_FILE =
Path("data/raw/bab08/dynamic_articles.html").resolve()
SCREENSHOT_FILE = Path("logs/bab08/screenshots/10_pipeline.png")
OUTPUT_CSV = Path("data/clean/bab08/pipeline_ready.csv")
DB_FILE = Path("database/bab05/scraping.db")

def clean_text(value):
    if value is None:
        return ""

    return " ".join(str(value).split())

def make_hash(*values):
    text = "|".join(clean_text(value) for value in values)
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

def extract_articles(page):
    articles = []
    cards = page.locator(".article-card")

    for index in range(cards.count()):
        card = cards.nth(index)
        title_link = card.locator(".article-title a")

        title = clean_text(title_link.inner_text())
        url = clean_text(title_link.get_attribute("href"))
        date =
clean_text(card.locator(".article-date").inner_text())
        author =
clean_text(card.locator(".article-author").inner_text())
        summary =
clean_text(card.locator(".article-summary").inner_text())

        articles.append({
            "title": title,
            "url": url,
```

```

        "date": date,
        "author": author,
        "summary": summary,
        "source_url": HTML_FILE.as_uri(),
        "scraped_at": datetime.utcnow().isoformat(),
        "content_hash": make_hash(title, url, summary),
        "validation_status": "valid"
    })

    return articles

def save_to_sqlite(items):
    connection = sqlite3.connect(DB_FILE)
    cursor = connection.cursor()

    sql = """
    INSERT OR IGNORE INTO scraped_items
    (title, url, date, author, summary, source_url, scraped_at,
    content_hash, validation_status)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """

    for item in items:
        cursor.execute(
            sql,
            (
                item["title"],
                item["url"],
                item["date"],
                item["author"],
                item["summary"],
                item["source_url"],
                item["scraped_at"],
                item["content_hash"],
                item["validation_status"],
            )
        )

    connection.commit()
    connection.close()

def main():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()

        page.goto(HTML_FILE.as_uri())

        page.locator(".article-card").first.wait_for(timeout=5000)

        page.locator("#loadMore").click()

        page.locator(".article-card").nth(3).wait_for(timeout=5000)

```

```

        page.screenshot(path=str(SCREENSHOT_FILE),
full_page=True)

        items = extract_articles(page)

        browser.close()

df = pd.DataFrame(items)
df = df.drop_duplicates(subset=["url"])
df.to_csv(OUTPUT_CSV, index=False)

save_to_sqlite(df.to_dict(orient="records"))

print(f"Total item: {len(df)}")
print(f"CSV          : {OUTPUT_CSV}")
print(f"Screenshot: {SCREENSHOT_FILE}")
print("Data masuk database SQLite.")

if __name__ == "__main__":
    main()

```

Jalankan:

```

python
scripts/bab08/10_playwright_clean_database_pipeline.py

```

Output:

```

Total item: 4
CSV          : data/clean/bab08/pipeline_ready.csv
Screenshot: logs/bab08/screenshots/10_pipeline.png
Data masuk database SQLite.

```

Pipeline ini adalah bentuk ringkas:

```

Playwright → Render → Extract → Clean → Hash → CSV →
SQLite

```

8.19 Struktur Akhir Folder Bab 8

Struktur akhir:

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   │   └── bab08/
│   │       └── dynamic_articles.html
│   └── clean/
│       └── bab08/
│           ├── rendered_articles.json
│           ├── rendered_articles_load_more.json
│           ├── search_results.json
│           ├── rendered_articles.csv
│           ├── rendered_articles_ready.csv
│           └── pipeline_ready.csv
├── logs/
│   └── bab08/
│       ├── playwright_error.log
│       └── screenshots/
│           ├── 01_page_open.png
│           ├── 02_articles_loaded.png
│           ├── 03_rendered_articles.png
│           ├── 04_after_load_more.png
│           ├── 05_search_results.png
│           ├── 09_error.png
│           └── 10_pipeline.png
└── scripts/
    └── bab08/
        ├── 01_open_dynamic_page.py
        ├── 02_wait_for_articles.py
        ├── 03_extract_rendered_articles.py
        ├── 04_click_load_more.py
        ├── 05_fill_search_form.py
        ├── 06_json_to_csv.py
        ├── 07_prepare_for_database.py
        ├── 08_insert_to_sqlite.py
        ├── 09_debug_screenshot_on_error.py
        └── 10_playwright_clean_database_pipeline.py
```

8.20 Checklist Praktik Bab 8

Mahasiswa dianggap selesai Bab 8 jika sudah mampu:

- ☐ Memahami perbedaan HTTP scraping dan browser automation
- ☐ Menginstal Playwright Python
- ☐ Menjalankan browser headless
- ☐ Membuka halaman HTML dinamis lokal
- ☐ Menunggu selector muncul
- ☐ Mengambil screenshot debugging
- ☐ Mengekstrak data hasil render JavaScript
- ☐ Klik tombol load more
- ☐ Mengisi form pencarian sederhana
- ☐ Menyimpan hasil ke JSON
- ☐ Membersihkan data ke CSV
- ☐ Membuat content_hash
- ☐ Menyimpan data hasil render ke database
- ☐ Membuat screenshot saat error
- ☐ Membuat pipeline Playwright → Clean → Database

8.21 Kesalahan Umum Bab 8

1. Memakai Playwright untuk semua scraping

Masalah:

- Pipeline lambat.
- Resource boros.
- Debugging lebih kompleks.

Solusi:

Gunakan requests/Scrapy jika HTML statis sudah cukup.
Gunakan Playwright hanya untuk halaman dinamis.

2. Mengambil data sebelum render selesai

Masalah:

- Data kosong.
- Artikel tidak muncul.
- Selector tidak ditemukan.

Solusi:

```
page.locator(".article-card").first.wait_for(timeout=5000)
```

3. Terlalu banyak memakai fixed timeout

Masalah:

```
page.wait_for_timeout(10000)
```

Solusi:

```
page.locator(".target").wait_for()
```

4. Tidak menyimpan screenshot

Masalah:

- Sulit tahu halaman gagal di bagian mana.

Solusi:

```
page.screenshot(path="logs/bab08/screenshots/debug.png",  
full_page=True)
```


5. Scraping login page tanpa izin

Masalah:

- Berisiko melanggar etika dan aturan akses.
- Berisiko mengambil data privat.

Solusi:

Jangan scraping area login, data akun, atau data sensitif tanpa izin resmi.

6. Tidak menyimpan source_url

Masalah:

- Data sulit diaudit.

Solusi:

```
"source_url": HTML_FILE.as_uri()
```

7. Tidak membersihkan data sebelum database

Masalah:

- Database menjadi kotor.

Solusi:

Clean → Hash → Validate → Insert

8.22 Ringkasan Bab

Bab ini memperkenalkan Playwright untuk scraping website dinamis. Mahasiswa belajar bahwa tidak semua halaman dapat diambil dengan `requests`, karena banyak website modern memuat data setelah JavaScript berjalan. Playwright digunakan ketika data baru muncul setelah render, setelah klik tombol, atau setelah form pencarian dijalankan.

Pada bagian praktik, mahasiswa membuat halaman dinamis lokal, menjalankan Playwright, menunggu selector, mengambil screenshot, mengekstrak artikel hasil render, klik tombol *load more*, mengisi form pencarian, membersihkan data, membuat hash, dan menyimpan hasil ke database. Mahasiswa juga belajar membuat screenshot saat error agar debugging tidak hanya berdasarkan tebakan.

Output akhir Bab 8:

- Script Playwright.
- Data dari halaman dinamis.
- Screenshot debugging.
- CSV hasil render.
- Data masuk database.
- Pipeline **Playwright** → **Clean** → **Database**.

Pesan strategis Bab 8:

Website dinamis membutuhkan pendekatan dinamis. Tetapi semakin kuat alatnya, semakin besar tanggung jawabnya: tunggu konten dengan benar, simpan bukti debugging, hormati batas etika, dan jangan pernah memakai browser automation untuk mengambil data privat atau membypass proteksi.

Bab 9 — Pipeline End-to-End: Web → Scraper → LLM → Database → Query

Pipeline yang baik harus bisa dijalankan ulang, diaudit, diperbaiki, dan dijelaskan oleh orang lain tanpa bergantung pada ingatan pembuatnya.

Bab 9 menggabungkan seluruh komponen: scraping data legal, cleaning, deduplication, ekstraksi LLM, penyimpanan ke PostgreSQL/MongoDB, query, scheduling, logging, audit trail, dan dokumentasi portofolio.

Docker Compose dipakai karena dapat mendefinisikan dan menjalankan aplikasi multi-container melalui satu file YAML, termasuk service database, network, dan volume. ([Docker Documentation](#)) Ollama dipakai karena API lokalnya tersedia secara default di <http://localhost:11434/api> dan dapat dipanggil dari Python untuk proses LLM. ([Ollama Documentation](#))

9.1 Mengapa Bab Ini Menjadi Jantung Buku?

Bab sebelumnya membahas komponen secara bertahap:

- Bab 2: HTTP, HTML, dan parsing.
- Bab 3: scraping terstruktur.
- Bab 4: Scrapy untuk crawling.
- Bab 5: database open source.
- Bab 6: cleaning, validation, dan data quality.
- Bab 7: LLM untuk ekstraksi dan ringkasan.
- Bab 8: Playwright untuk website dinamis.

Bab 9 menyatukan semuanya menjadi satu proyek nyata.

Targetnya bukan hanya “script berjalan”. Targetnya adalah **project portofolio end-to-end**.

Pipeline akhir:

```
Web / HTML Legal
↓
Scraper
↓
Raw Data
↓
Cleaner + Deduplication
↓
LLM Extraction + Summary
↓
PostgreSQL / MongoDB
↓
Query
↓
Report / README Portofolio
```

Mahasiswa tidak hanya belajar mengambil data. Mahasiswa belajar membangun sistem data kecil yang profesional.

9.2 Arsitektur Pipeline End-to-End

Pipeline Bab 9 memiliki dua jalur penyimpanan.

Jalur 1 — Raw Data

Raw data disimpan ke MongoDB.

Tujuannya:

- Menyimpan data mentah.
- Menyimpan sumber URL.
- Menyimpan timestamp.
- Menyimpan status scraping.
- Menyimpan HTML/text sebelum diproses.
- Menyediakan bahan audit.

Jalur 2 — Processed Data

Processed data disimpan ke PostgreSQL.

Tujuannya:

- Menyimpan data bersih.
- Menyimpan hasil ringkasan LLM.
- Menyimpan entitas LLM.
- Menyimpan topik.
- Menyimpan metadata model.
- Memudahkan query analitik.

Arsitektur sederhana:

```
[Scraper]
  ↓
[Raw JSON / Raw Text]
  ↓
[MongoDB: raw_pages]
  ↓
[Cleaner + Validator]
  ↓
[Ollama LLM]
  ↓
[PostgreSQL: processed_items + llm_extractions]
  ↓
[Query / Analysis]
```

Prinsip desain:

- **Raw data jangan dibuang.**
- **Data bersih harus divalidasi.**
- **Output LLM harus disimpan bersama nama model.**
- **Setiap record harus punya audit trail.**
- **Pipeline harus bisa dijalankan ulang.**

9.3 Folder Proyek Produksi Sederhana

Struktur folder akhir:

```
webscraping-llm-database/  
├── data/  
│   ├── raw/  
│   ├── clean/  
│   └── rejected/  
├── database/  
│   ├── docker-compose.yml  
│   ├── schema_postgres.sql  
│   └── schema_mongo_notes.md  
├── logs/  
├── scripts/  
│   ├── run_pipeline.py  
│   ├── query_postgres.py  
│   └── query_mongodb.py  
├── .env.example  
├── requirements.txt  
├── README.md  
└── .gitignore
```

Fungsi folder:

- **data/raw/** menyimpan hasil scraping awal.
- **data/clean/** menyimpan data siap proses.
- **data/rejected/** menyimpan data gagal validasi.
- **database/** menyimpan konfigurasi database.
- **logs/** menyimpan log pipeline.
- **scripts/** menyimpan script utama.
- **.env.example** menjadi template konfigurasi.
- **README.md** menjelaskan cara menjalankan proyek.

Struktur folder adalah dokumentasi pertama dari cara berpikir engineer.

9.4 Konfigurasi dengan .env

File `.env` menyimpan konfigurasi lokal. Jangan tulis konfigurasi penting langsung di kode.

Buat file contoh:

```
nano .env.example
```

Isi:

```
PROJECT_NAME=webscraping-llm-database
```

```
POSTGRES_HOST=localhost
```

```
POSTGRES_PORT=5432
```

```
POSTGRES_DB=scraping_db
```

```
POSTGRES_USER=scraping_user
```

```
POSTGRES_PASSWORD=change_me
```

```
MONGO_URI=mongodb://localhost:27017
```

```
MONGO_DB=scraping_db
```

```
OLLAMA_HOST=http://localhost:11434
```

```
OLLAMA_MODEL=llama3.2:3b
```

Lalu buat `.env` lokal:

```
cp .env.example .env
```

`.env.example` boleh dibagikan. `.env` jangan dipublikasikan.

9.5 Docker Compose untuk Database

Buat file:

```
nano database/docker-compose.yml
```

Isi:

```
services:
  postgres:
    image: postgres:16
    container_name: pipeline_postgres
    environment:
      POSTGRES_DB: scraping_db
      POSTGRES_USER: scraping_user
      POSTGRES_PASSWORD: scraping_password
    ports:
      - "5432:5432"
    volumes:
      - pipeline_postgres_data:/var/lib/postgresql/data

  mongo:
    image:
      mongodb/mongodb-community-server:7.0-ubuntu2204
    container_name: pipeline_mongo
    ports:
      - "27017:27017"
    volumes:
      - pipeline_mongo_data:/data/db

volumes:
  pipeline_postgres_data:
  pipeline_mongo_data:
```

Jalankan:

```
cd database
docker compose up -d
cd ..
```

Cek:

```
docker ps
```

Docker Compose memudahkan reproducibility karena database, port, environment, dan volume didefinisikan dalam satu file konfigurasi. ([Docker Documentation](#))

9.6 Schema PostgreSQL

Buat file:

nano database/schema_postgres.sql

Isi:

```
CREATE TABLE IF NOT EXISTS processed_items (  
  id SERIAL PRIMARY KEY,  
  title TEXT NOT NULL,  
  url TEXT NOT NULL UNIQUE,  
  date TEXT,  
  author TEXT,  
  summary TEXT,  
  clean_text TEXT,  
  source_url TEXT,  
  scraped_at TIMESTAMP,  
  content_hash TEXT,  
  validation_status TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS llm_extractions (  
  id SERIAL PRIMARY KEY,  
  item_url TEXT NOT NULL,  
  model_name TEXT NOT NULL,  
  llm_summary TEXT,  
  topics_json JSONB,  
  entities_json JSONB,  
  raw_llm_output JSONB,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(item_url, model_name)  
);
```

```
CREATE TABLE IF NOT EXISTS pipeline_runs (  
  id SERIAL PRIMARY KEY,  
  run_id TEXT UNIQUE,  
  started_at TIMESTAMP,  
  finished_at TIMESTAMP,  
  status TEXT,  
  raw_count INTEGER,  
  clean_count INTEGER,  
  rejected_count INTEGER,  
  llm_count INTEGER,  
  error_message TEXT  
);
```

```
CREATE INDEX IF NOT EXISTS idx_processed_items_url  
ON processed_items(url);
```

```
CREATE INDEX IF NOT EXISTS idx_processed_items_hash  
ON processed_items(content_hash);
```

```
CREATE INDEX IF NOT EXISTS idx_llm_item_url  
ON llm_extractions(item_url);
```

Catatan:

- `processed_items` menyimpan data bersih.
- `llm_extractions` menyimpan output LLM.
- `pipeline_runs` menyimpan histori eksekusi pipeline.
- `UNIQUE` mencegah duplikasi.

PostgreSQL mendukung `INSERT ... ON CONFLICT` untuk menentukan aksi alternatif saat terjadi konflik unique constraint, seperti `DO NOTHING` atau `DO UPDATE`. ([PostgreSQL](#))

9.7 Instalasi Paket Python

Masuk ke root proyek:

```
cd webscraping-llm-database  
source .venv/bin/activate
```

Instal paket:

```
pip install requests beautifulsoup4 lxml pandas pydantic  
psycopg2-binary pymongo python-dotenv
```

Simpan dependency:

```
pip freeze > requirements.txt
```

Paket yang dipakai:

- **requests**: mengambil halaman.
- **beautifulsoup4**: parsing HTML.
- **lxml**: parser HTML.
- **pandas**: olah dataset.
- **pydantic**: validasi schema.
- **psycopg2-binary**: koneksi PostgreSQL.
- **pymongo**: koneksi MongoDB.
- **python-dotenv**: membaca **.env**.

Bagian Praktik Proyek Utama

9.8 Membuat Halaman Artikel Legal untuk Latihan

Agar aman dan reproducible, proyek utama memakai HTML lokal. Konsepnya sama dengan web publik, tetapi tidak membebani server luar.

Buat folder:

```
mkdir -p data/raw/bab09/site
```

Buat halaman:

```
nano data/raw/bab09/site/articles.html
```

Isi:

```
<!DOCTYPE html>
<html>
<head>
  <title>Portal Data Mahasiswa</title>
</head>
<body>
  <h1>Portal Data Mahasiswa</h1>

  <article class="article-item">
    <h2 class="article-title">
      <a
href="https://kampus.example/artikel/data-engineering">D
ata Engineering untuk Mahasiswa</a>
    </h2>
    <p class="article-date">2026-05-27</p>
    <p class="article-author">Admin Kampus</p>
    <p class="article-summary">Pipeline data membantu
mahasiswa mengubah data mentah menjadi informasi.</p>
    <div class="article-body">
      Data engineering adalah kemampuan membangun
pipeline dari pengambilan data,
      pembersihan, validasi, penyimpanan, sampai
      analisis. Mahasiswa perlu memahami
      Python, database, dan LLM sebagai satu ekosistem
      kerja data modern.
    </div>
  </article>

  <article class="article-item">
    <h2 class="article-title">
      <a
href="https://kampus.example/artikel/scraping-etis">Web
Scraping yang Bertanggung Jawab</a>
```

```

</h2>
<p class="article-date">2026-05-28</p>
<p class="article-author">Tim Data</p>
<p class="article-summary">Scraping harus
memperhatikan legalitas, privasi, dan beban server.</p>
<div class="article-body">
    Web scraping harus dilakukan secara etis. Praktisi
    perlu memperhatikan legalitas,
    privasi, terms of service, robots.txt, beban
    server, dan kualitas data.
    Scraping bukan sekadar mengambil data, tetapi
    membangun pipeline yang dapat diaudit.
</div>
</article>

<article class="article-item">
    <h2 class="article-title">
        <a
href="https://kampus.example/artikel/llm-lokal">LLM
Lokal untuk Ekstraksi Informasi</a>
    </h2>
    <p class="article-date">2026-05-29</p>
    <p class="article-author">Lab AI</p>
    <p class="article-summary">Model lokal membantu
ringkasan, klasifikasi, dan JSON.</p>
    <div class="article-body">
        LLM lokal dapat membantu ekstraksi entitas,
        ringkasan, klasifikasi topik,
        dan konversi teks menjadi JSON. Namun output LLM
        harus divalidasi dengan schema
        agar tidak menghasilkan informasi palsu atau
        format rusak.
    </div>
</article>
</body>
</html>

```

9.9 Membuat Script Pipeline Utama

Buat file:

```
nano scripts/run_pipeline.py
```

Isi script berikut. Script ini menggabungkan:

- scraping HTML,
- simpan raw data ke MongoDB,
- cleaning,
- deduplication,
- validasi Pydantic,
- LLM extraction,
- simpan processed data ke PostgreSQL,
- simpan log pipeline.

```
import hashlib
import json
import logging
import os
import uuid
from datetime import datetime
from pathlib import Path

import psycopg2
import requests
from bs4 import BeautifulSoup
from dotenv import load_dotenv
from pydantic import BaseModel, Field, HttpUrl,
ValidationError
from pymongo import MongoClient
from psycopg2.extras import Json

load_dotenv()

SITE_FILE =
Path("data/raw/bab09/site/articles.html").resolve()
LOG_FILE = Path("logs/pipeline.log")
POSTGRES_SCHEMA = Path("database/schema_postgres.sql")

OLLAMA_HOST = os.getenv("OLLAMA_HOST",
"http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:3b")

logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="% (asctime)s | % (levelname)s | % (message)s"
)
```

```

class CleanItem(BaseModel):
    title: str = Field(min_length=1)
    url: HttpUrl
    date: str = ""
    author: str = ""
    summary: str = ""
    clean_text: str = Field(min_length=1)
    source_url: str
    scraped_at: datetime
    content_hash: str
    validation_status: str = "valid"

def clean_text(value):
    if value is None:
        return ""

    return " ".join(str(value).split())

def make_hash(*values):
    text = "|".join(clean_text(value) for value in
values)
    return
hashlib.sha256(text.encode("utf-8")).hexdigest()

def get_postgres_connection():
    return psycopg2.connect(
        host=os.getenv("POSTGRES_HOST"),
        port=os.getenv("POSTGRES_PORT"),
        dbname=os.getenv("POSTGRES_DB"),
        user=os.getenv("POSTGRES_USER"),
        password=os.getenv("POSTGRES_PASSWORD"),
    )

def get_mongo_collection():
    client = MongoClient(os.getenv("MONGO_URI"))
    db = client[os.getenv("MONGO_DB")]
    return client, db["raw_pages"]

def create_postgres_schema():
    connection = get_postgres_connection()
    cursor = connection.cursor()

    schema_sql =
POSTGRES_SCHEMA.read_text(encoding="utf-8")
    cursor.execute(schema_sql)

    connection.commit()

```

```

        cursor.close()
        connection.close()

def fetch_html():
    html = SITE_FILE.read_text(encoding="utf-8")
    return html

def parse_articles(html):
    soup = BeautifulSoup(html, "lxml")
    articles = []

    for node in soup.select("article.article-item"):
        title_node = node.select_one(".article-title a")

        title = clean_text(title_node.get_text()) if
title_node else ""
        url = clean_text(title_node.get("href")) if
title_node else ""
        date =
clean_text(node.select_one(".article-date").get_text())
if node.select_one(".article-date") else ""
        author =
clean_text(node.select_one(".article-author").get_text()
) if node.select_one(".article-author") else ""
        summary =
clean_text(node.select_one(".article-summary").get_text(
)) if node.select_one(".article-summary") else ""
        body =
clean_text(node.select_one(".article-body").get_text())
if node.select_one(".article-body") else ""

        clean_body = clean_text(summary + " " + body)

        item = {
            "title": title,
            "url": url,
            "date": date,
            "author": author,
            "summary": summary,
            "clean_text": clean_body,
            "source_url": SITE_FILE.as_uri(),
            "scraped_at": datetime.utcnow(),
            "content_hash": make_hash(title, url,
clean_body),
            "validation_status": "pending"
        }

        articles.append(item)

```



```

        return articles

def save_raw_to_mongo(items, run_id):
    client, collection = get_mongo_collection()

    documents = []

    for item in items:
        doc = dict(item)
        doc["run_id"] = run_id
        doc["scraped_at"] =
doc["scraped_at"].isoformat()
        documents.append(doc)

    if documents:
        collection.insert_many(documents)

    client.close()

def validate_and_deduplicate(items):
    seen_urls = set()
    seen_hashes = set()
    valid_items = []
    rejected_items = []

    for item in items:
        if item["url"] in seen_urls:
            rejected_items.append({"reason":
"duplicate_url", "item": item})
            continue

        if item["content_hash"] in seen_hashes:
            rejected_items.append({"reason":
"duplicate_hash", "item": item})
            continue

        seen_urls.add(item["url"])
        seen_hashes.add(item["content_hash"])

    try:
        item["validation_status"] = "valid"
        validated = CleanItem(**item)
        valid_items.append(validated.model_dump())
    except ValidationError as error:
        rejected_items.append({
            "reason": "validation_error",
            "error": error.errors(),

```

```

        "item": item
    })

    return valid_items, rejected_items

def call_ollama(prompt):
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "format": "json"
    }

    response = requests.post(
        f"{OLLAMA_HOST}/api/generate",
        json=payload,
        timeout=180
    )

    response.raise_for_status()
    return response.json()["response"]

def extract_with_llm(item):
    prompt = f"""
Anda adalah mesin ekstraksi informasi.

Gunakan hanya informasi dari teks.
Jangan menambah fakta dari luar teks.
Jika tidak ada informasi, gunakan list kosong.
Output harus JSON valid.

Format:
{{
  "llm_summary": "ringkasan pendek",
  "topics": ["topik1", "topik2"],
  "entities": [
    {{ "name": "nama", "type":
"TECHNOLOGY|CONCEPT|DATABASE|RISK|OTHER" }}
  ]
}}

Judul:
{item["title"]}

Teks:
{item["clean_text"]}
"""

```

```

raw_output = call_ollama(prompt)

try:
    parsed = json.loads(raw_output)
except json.JSONDecodeError:
    parsed = {
        "llm_summary": "",
        "topics": [],
        "entities": [],
        "raw_parse_error": raw_output
    }

return parsed, raw_output

def save_processed_to_postgres(items, llm_outputs):
    connection = get_postgres_connection()
    cursor = connection.cursor()

    item_sql = """
    INSERT INTO processed_items
    (title, url, date, author, summary, clean_text,
    source_url, scraped_at, content_hash, validation_status)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    ON CONFLICT (url) DO UPDATE SET
    title = EXCLUDED.title,
    date = EXCLUDED.date,
    author = EXCLUDED.author,
    summary = EXCLUDED.summary,
    clean_text = EXCLUDED.clean_text,
    source_url = EXCLUDED.source_url,
    scraped_at = EXCLUDED.scraped_at,
    content_hash = EXCLUDED.content_hash,
    validation_status = EXCLUDED.validation_status
    """

    llm_sql = """
    INSERT INTO llm_extractions
    (item_url, model_name, llm_summary, topics_json,
    entities_json, raw_llm_output)
    VALUES (%s, %s, %s, %s, %s, %s)
    ON CONFLICT (item_url, model_name) DO UPDATE SET
    llm_summary = EXCLUDED.llm_summary,
    topics_json = EXCLUDED.topics_json,
    entities_json = EXCLUDED.entities_json,
    raw_llm_output = EXCLUDED.raw_llm_output,
    created_at = CURRENT_TIMESTAMP
    """

```

```

for item in items:
    cursor.execute(
        item_sql,
        (
            item["title"],
            str(item["url"]),
            item["date"],
            item["author"],
            item["summary"],
            item["clean_text"],
            item["source_url"],
            item["scraped_at"],
            item["content_hash"],
            item["validation_status"],
        )
    )

for item, llm_output in zip(items, llm_outputs):
    parsed = llm_output["parsed"]
    cursor.execute(
        llm_sql,
        (
            str(item["url"]),
            OLLAMA_MODEL,
            parsed.get("llm_summary", ""),
            Json(parsed.get("topics", [])),
            Json(parsed.get("entities", [])),
            Json(parsed),
        )
    )

connection.commit()
cursor.close()
connection.close()

def save_pipeline_run(run_id, started_at, finished_at,
status, raw_count, clean_count, rejected_count,
llm_count, error_message=""):
    connection = get_postgres_connection()
    cursor = connection.cursor()

    sql = """
INSERT INTO pipeline_runs
(run_id, started_at, finished_at, status, raw_count,
clean_count, rejected_count, llm_count, error_message)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (run_id) DO NOTHING
"""

```

```

        cursor.execute(
            sql,
            (
                run_id,
                started_at,
                finished_at,
                status,
                raw_count,
                clean_count,
                rejected_count,
                llm_count,
                error_message,
            )
        )

        connection.commit()
        cursor.close()
        connection.close()

def save_rejected_rows(rejected_items, run_id):
    output_file =
    Path(f"data/rejected/rejected_{run_id}.json")
    output_file.write_text(
        json.dumps(rejected_items, indent=2,
            ensure_ascii=False, default=str),
        encoding="utf-8"
    )

def main():
    run_id = str(uuid.uuid4())
    started_at = datetime.utcnow()

    logging.info(f"Pipeline started | run_id={run_id}")

    try:
        create_postgres_schema()

        html = fetch_html()
        raw_items = parse_articles(html)

        save_raw_to_mongo(raw_items, run_id)

        valid_items, rejected_items =
        validate_and_deduplicate(raw_items)
        save_rejected_rows(rejected_items, run_id)

        llm_outputs = []

```

```

        for item in valid_items:
            parsed, raw_output = extract_with_llm(item)
            llm_outputs.append({
                "parsed": parsed,
                "raw_output": raw_output
            })

        save_processed_to_postgres(valid_items,
                                   llm_outputs)

        finished_at = datetime.utcnow()

        save_pipeline_run(
            run_id=run_id,
            started_at=started_at,
            finished_at=finished_at,
            status="success",
            raw_count=len(raw_items),
            clean_count=len(valid_items),
            rejected_count=len(rejected_items),
            llm_count=len(llm_outputs),
        )

        logging.info(
            f"Pipeline success | run_id={run_id} | "
            f"raw={len(raw_items)} | clean={len(valid_items)} | "
            f"rejected={len(rejected_items)}"
        )

        print("Pipeline selesai.")
        print(f"Run ID      : {run_id}")
        print(f"Raw items    : {len(raw_items)}")
        print(f"Clean items   : {len(valid_items)}")
        print(f"Rejected     : {len(rejected_items)}")
        print(f"LLM outputs  : {len(llm_outputs)}")

    except Exception as error:
        finished_at = datetime.utcnow()
        error_message = str(error)

        logging.exception(f"Pipeline failed | "
                           f"run_id={run_id}")

    try:
        save_pipeline_run(
            run_id=run_id,
            started_at=started_at,

```

```

        finished_at=finished_at,
        status="failed",
        raw_count=0,
        clean_count=0,
        rejected_count=0,
        llm_count=0,
        error_message=error_message,
    )
except Exception:
    logging.exception("Failed to save pipeline
error status")

    raise

if __name__ == "__main__":
    main()

```

Jalankan:

```
python scripts/run_pipeline.py
```

Output:

```

Pipeline selesai.
Run ID          : ...
Raw items       : 3
Clean items     : 3
Rejected        : 0
LLM outputs     : 3

```

9.10 Penjelasan Script run_pipeline.py

Script ini dibagi menjadi fungsi kecil agar mudah dipahami.

fetch_html()

Membaca sumber HTML.

```
html = SITE_FILE.read_text(encoding="utf-8")
```

Pada proyek nyata, bagian ini bisa diganti menjadi requests.get() atau Scrapy.

parse_articles()

Mengubah HTML menjadi list of dict.

Field yang diambil:

- title
- url
- date
- author
- summary
- clean_text
- source_url
- scraped_at
- content_hash

save_raw_to_mongo()

Menyimpan data mentah ke MongoDB.

Tujuannya:

- Audit.
- Backup raw data.
- Jejak asal data.
- Bahan debugging.

`validate_and_deduplicate()`

Melakukan:

- Deduplication URL.
- Deduplication hash.
- Validasi Pydantic.
- Pemisahan valid dan rejected data.

`extract_with_llm()`

Mengirim teks ke Ollama.

Ollama `/api/generate` menerima `model` dan `prompt`; endpoint ini juga mendukung `stream: false` agar respons dikembalikan lengkap. ([Ollama](#))

`save_processed_to_postgres()`

Menyimpan:

- Data bersih ke `processed_items`.
- Output LLM ke `llm_extractions`.

`save_pipeline_run()`

Menyimpan histori pipeline.

Isi audit:

- `run_id`
- `started_at`
- `finished_at`
- `status`
- `raw_count`
- `clean_count`
- `rejected_count`
- `llm_count`
- `error_message`

9.11 Error Handling dan Retry

Pipeline harus siap gagal. Penyebab umum:

- Database belum aktif.
- Ollama belum berjalan.
- Model belum diunduh.
- JSON LLM rusak.
- Data kosong.
- Koneksi timeout.
- Schema belum dibuat.
- File `.env` salah.

Strategi error handling:

- Gunakan `try-except`.
- Simpan log error.
- Simpan status `failed`.
- Jangan hilangkan rejected data.
- Jangan menimpa raw data.
- Gunakan `raise` setelah log agar error terlihat.

Untuk retry, mulai sederhana:

```
def retry_call(function, attempts=3):
    last_error = None

    for attempt in range(1, attempts + 1):
        try:
            return function()
        except Exception as error:
            last_error = error
            logging.warning(f"Retry {attempt}/{attempts}
gagal: {error}")

    raise last_error
```

Gunakan retry untuk:

- Request web.
- Panggilan LLM.
- Koneksi sementara.

Jangan gunakan retry untuk:

- Data invalid.
- Schema salah.
- Credential salah.
- Prompt buruk.

Retry bukan obat untuk desain yang salah. Retry hanya untuk kegagalan sementara.

9.12 Logging Pipeline

Python memiliki modul `logging` bawaan untuk mencatat event aplikasi. Dokumentasi Python menjelaskan logging sebagai cara melacak kejadian yang terjadi saat software berjalan, lengkap dengan level seperti info, warning, error, dan lainnya. ([Python documentation](#))

Contoh konfigurasi:

```
logging.basicConfig(
    filename="logs/pipeline.log",
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)
```

Level penting:

- **INFO**: proses normal.
- **WARNING**: ada masalah tetapi pipeline masih berjalan.
- **ERROR**: terjadi error.
- **EXCEPTION**: error lengkap dengan traceback.

Contoh log:

```
2026-05-27 10:00:00 | INFO | Pipeline started
2026-05-27 10:00:05 | INFO | Raw items: 3
2026-05-27 10:00:20 | INFO | Pipeline success
```

Log adalah ingatan pipeline. Tanpa log, debugging berubah menjadi tebak-tebakan.

9.13 Query PostgreSQL untuk Analisis

Buat file:

```
nano scripts/query_postgres.py
```

Isi:

```
import os
import psycopg2
from dotenv import load_dotenv

load_dotenv()

connection = psycopg2.connect(
    host=os.getenv("POSTGRES_HOST"),
    port=os.getenv("POSTGRES_PORT"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

cursor = connection.cursor()

print("Total processed items:")
cursor.execute("SELECT COUNT(*) FROM processed_items")
print(cursor.fetchone()[0])

print("\nDaftar artikel:")
cursor.execute("""
SELECT title, author, date
FROM processed_items
ORDER BY date ASC
""")

for row in cursor.fetchall():
    print(row)

print("\nRingkasan LLM:")
cursor.execute("""
SELECT p.title, l.llm_summary
FROM processed_items p
JOIN llm_extractions l
ON p.url = l.item_url
ORDER BY p.date ASC
""")

for row in cursor.fetchall():
    print("-" * 60)
    print("Title   :", row[0])
```

```

        print("Summary:", row[1])

    print("\nTopik JSON:")
    cursor.execute("""
SELECT item_url, topics_json
FROM llm_extractions
""")

    for row in cursor.fetchall():
        print(row)

    cursor.close()
    connection.close()

```

Jalankan:

```
python scripts/query_postgres.py
```

Query yang dipelajari:

- COUNT
- SELECT
- ORDER BY
- JOIN
- membaca JSONB

9.14 Query MongoDB untuk Dokumen Mentah

Buat file:

```
nano scripts/query_mongodb.py
```

Isi:

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()

client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("MONGO_DB")]
collection = db["raw_pages"]

print("Total raw documents:")
print(collection.count_documents({}))

print("\nContoh dokumen mentah:")
for doc in collection.find({}, {"_id": 0}).limit(3):
    print("-" * 60)
    print(doc)

print("\nFilter author = Tim Data:")
for doc in collection.find({"author": "Tim Data"},
{"_id": 0, "title": 1, "url": 1, "run_id": 1}):
    print(doc)

client.close()
```

Jalankan:

```
python scripts/query_mongodb.py
```

MongoDB dipakai untuk menjawab:

- Data mentah apa yang masuk?
- Run ID mana yang menghasilkan data?
- Apakah raw data masih tersedia?
- Apa perbedaan raw dan processed?

9.15 Scheduling dengan Cron

Pipeline yang baik bisa dijalankan terjadwal.

Cek lokasi Python:

```
which python
```

Cek lokasi proyek:

```
pwd
```

Buka crontab:

```
crontab -e
```

Contoh menjalankan pipeline setiap hari pukul 07:00:

```
0 7 * * * cd /path/ke/webscraping-llm-database &&  
/path/ke/.venv/bin/python scripts/run_pipeline.py >>  
logs/cron.log 2>&1
```

Penjelasan:

- `0 7 * * *` berarti setiap hari pukul 07:00.
- `cd ...` masuk ke folder proyek.
- `/path/ke/.venv/bin/python` memakai Python dari virtual environment.
- `scripts/run_pipeline.py` menjalankan pipeline.
- `>> logs/cron.log` menambahkan output ke log.
- `2>&1` menggabungkan error ke log yang sama.

Prinsip scheduling:

- Jalankan pelan.
- Jangan membanjiri sumber.
- Simpan log.
- Simpan run ID.
- Pastikan pipeline aman dijalankan ulang.

9.16 Audit Trail

Audit trail menjawab pertanyaan:

- Data diambil kapan?
- Dari sumber mana?
- Diproses oleh script apa?
- Diproses oleh model apa?
- Masuk database kapan?
- Berapa data valid?
- Berapa data ditolak?
- Error apa yang muncul?

Field wajib audit:

```
run_id
source_url
scraped_at
content_hash
model_name
created_at
validation_status
```

Tabel penting:

- `pipeline_runs`
- `processed_items`
- `llm_extractions`

Koleksi penting:

- `raw_pages`

Tanpa audit trail, pipeline sulit dipercaya.

9.17 Reproducibility

Project harus bisa dijalankan ulang oleh orang lain.

Agar reproducible, siapkan:

- `requirements.txt`
- `.env.example`
- `docker-compose.yml`
- `schema_postgres.sql`
- `README.md`
- struktur folder jelas
- script utama `run_pipeline.py`
- data latihan legal
- instruksi menjalankan pipeline
- instruksi query hasil

Checklist reproducibility:

- ☐ Bisa install dependency
- ☐ Bisa start database
- ☐ Bisa copy `.env.example` menjadi `.env`
- ☐ Bisa membuat schema
- ☐ Bisa menjalankan pipeline
- ☐ Bisa melihat log
- ☐ Bisa query PostgreSQL
- ☐ Bisa query MongoDB

Project portofolio yang bagus bukan hanya terlihat keren, tetapi bisa dijalankan ulang.

9.18 Membuat README Proyek

Buat file:

```
nano README.md
```

Isi:

```
# Web Scraping + Database + Open Source LLM Pipeline
```

```
Project ini adalah pipeline data end-to-end untuk  
latihan mahasiswa.
```

```
## Tujuan
```

```
Pipeline ini mengambil data artikel legal, membersihkan  
data, menyimpan raw data ke MongoDB, mengekstrak  
ringkasan dan entitas dengan LLM lokal, menyimpan data  
bersih ke PostgreSQL, lalu menjalankan query analisis  
sederhana.
```

```
## Arsitektur
```

```
Web / HTML Legal → Scraper → Cleaner → Validator →  
MongoDB Raw → Ollama LLM → PostgreSQL Processed → Query
```

```
## Tools
```

```
- Python  
- BeautifulSoup  
- lxml  
- Pydantic  
- PostgreSQL  
- MongoDB Community  
- Docker Compose  
- Ollama
```

```
## Struktur Folder
```

```
```text  
data/
database/
logs/
scripts/
.env.example
requirements.txt
README.md
```

## Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

## Jalankan Database

```
cd database
docker compose up -d
cd ..
```

## Jalankan Pipeline

```
python scripts/run_pipeline.py
```

## Query PostgreSQL

```
python scripts/query_postgres.py
```

## Query MongoDB

```
python scripts/query_mongodb.py
```

## Output

- Raw data masuk ke MongoDB collection `raw_pages`
- Processed data masuk ke PostgreSQL table `processed_items`
- Hasil LLM masuk ke PostgreSQL table `llm_extractions`
- Log pipeline tersimpan di `logs/pipeline.log`

## Etika

Project ini memakai data latihan legal. Untuk website publik, pastikan mematuhi aturan situs, tidak mengambil data privat, tidak bypass proteksi, dan tidak membebani server.

README adalah bagian penting portofolio. Ia menjelaskan bukan hanya kode, tetapi **cara berpikir proyek**.

---

## 9.19 Membuat `.gitignore`

Buat file:

```
```bash
nano .gitignore
```

Isi:

```
.venv/
.env
__pycache__/
*.pyc

logs/*.log
data/rejected/*.json

database/*.db
```

Tujuan:

- Jangan unggah virtual environment.
- Jangan unggah credential lokal.
- Jangan unggah cache.
- Jangan unggah database lokal besar.
- Jangan unggah file sensitif.

9.20 Struktur Akhir Proyek Bab 9

Struktur akhir:

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   │   └── bab09/
│   │       └── site/
│   │           └── articles.html
│   ├── clean/
│   └── rejected/
├── database/
│   ├── docker-compose.yml
│   └── schema_postgres.sql
├── logs/
│   ├── pipeline.log
│   └── cron.log
├── scripts/
│   ├── run_pipeline.py
│   ├── query_postgres.py
│   └── query_mongodb.py
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

Output utama:

- Project end-to-end.
- Docker Compose database.
- Script pipeline utama.
- Tabel raw dan processed.
- Hasil ekstraksi LLM.
- README portofolio.

9.21 Checklist Praktik Bab 9

Mahasiswa dianggap selesai Bab 9 jika mampu:

- ☐ Menjelaskan arsitektur pipeline end-to-end
- ☐ Membuat struktur folder proyek
- ☐ Membuat .env.example
- ☐ Menjalankan PostgreSQL dan MongoDB dengan Docker Compose
- ☐ Membuat schema PostgreSQL
- ☐ Membuat script run_pipeline.py
- ☐ Scraping data artikel legal
- ☐ Menyimpan raw data ke MongoDB
- ☐ Cleaning dan deduplication
- ☐ Validasi data dengan Pydantic
- ☐ Mengirim teks ke Ollama
- ☐ Membuat ringkasan artikel dengan LLM
- ☐ Mengekstrak entitas dengan LLM
- ☐ Menyimpan processed data ke PostgreSQL
- ☐ Query PostgreSQL
- ☐ Query MongoDB
- ☐ Membuat pipeline log
- ☐ Membuat audit trail
- ☐ Menjadwalkan pipeline dengan cron
- ☐ Membuat README proyek

9.22 Kesalahan Umum Bab 9

1. Semua kode ditaruh di satu blok tanpa fungsi

Masalah:

- Sulit debug.
- Sulit diuji.
- Sulit diperbaiki.

Solusi:

Pisahkan fetch, parse, clean, validate, LLM, save, query.

2. Tidak menyimpan raw data

Masalah:

- Tidak bisa audit.
- Tidak bisa reprocess.
- Tidak bisa cek kesalahan parser.

Solusi:

Simpan raw data ke MongoDB atau folder raw.

3. Tidak mencatat run ID

Masalah:

- Tidak tahu eksekusi mana yang menghasilkan data.

Solusi:

Gunakan UUID sebagai run_id.

4. Tidak menyimpan nama model

Masalah:

- Hasil LLM tidak reproducible.

Solusi:

Simpan model_name di tabel llm_extractions.

5. Tidak ada logging

Masalah:

- Pipeline gagal tanpa jejak.

Solusi:

Gunakan logging ke logs/pipeline.log.

6. Tidak menangani duplikasi

Masalah:

- Data membengkok palsu.
- Analisis salah.

Solusi:

Gunakan UNIQUE(url), content_hash, dan ON CONFLICT.

7. README tidak jelas

Masalah:

- Project sulit dinilai sebagai portofolio.

Solusi:

README harus menjelaskan tujuan, tools, arsitektur, setup, cara run, output, dan etika.

9.23 Ringkasan Bab

Bab ini menggabungkan seluruh komponen buku menjadi satu pipeline end-to-end.

Mahasiswa membangun proyek lengkap: scraping data artikel legal, menyimpan raw data, membersihkan dan memvalidasi data, mengekstrak ringkasan serta entitas dengan LLM lokal, menyimpan data ke PostgreSQL dan MongoDB, melakukan query, membuat logging, mencatat audit trail, menjadwalkan pipeline dengan cron, dan menulis README portofolio.

Pipeline ini memperlihatkan bahwa skill data modern bukan hanya kemampuan menulis script kecil. Skill yang dicari di dunia nyata adalah kemampuan membangun alur data yang **legal, bersih, terstruktur, reproducible, dan bisa diaudit**.

Output akhir Bab 9:

- Project end-to-end.
- Docker Compose database.
- Script `run_pipeline.py`.
- Raw data di MongoDB.
- Processed data di PostgreSQL.
- Hasil ekstraksi LLM.
- Query analisis.
- Logging pipeline.
- README portofolio.

Pesan strategis Bab 9:

Portofolio terbaik bukan sekadar demo yang berjalan sekali, tetapi pipeline yang bisa dijalankan ulang, diperiksa kualitasnya, ditelusuri sumber datanya, dan dijelaskan secara profesional.

Bab 10 — Etika, Keamanan, Portofolio, dan Masa Depan Pipeline Data AI

Pipeline data yang hebat bukan yang mengambil data paling banyak, tetapi yang paling etis, aman, hemat sumber daya, terdokumentasi, dan dapat dipertanggungjawabkan.

Bab ini menutup buku *Web Scraping + Database dengan Open Source LLM: Pipeline Data Modern, 2026*. Dalam outline, Bab 10 diarahkan untuk menutup buku dengan sikap profesional: *responsible scraping*, keamanan credential, dokumentasi, portofolio, dan tren pipeline data AI.

10.1 Mengapa Etika Menjadi Bab Penutup?

Mahasiswa yang sudah mengikuti Bab 1 sampai Bab 9 telah mampu membangun pipeline:

`Web → Scraper → Cleaner → Validator → Database → LLM → Query`

Kemampuan ini kuat. Tetapi kemampuan yang kuat juga berisiko disalahgunakan.

Scraping dapat membantu:

- Riset akademik.
- Jurnalisme data.
- Analisis pasar.
- Monitoring informasi publik.
- Dataset AI.
- Otomasi laporan.
- Portofolio data engineering.

Namun scraping juga dapat merugikan jika dipakai untuk:

- Mengambil data pribadi.
- Membebani server.
- Mengabaikan aturan situs.
- Mengumpulkan data tanpa tujuan jelas.
- Membypass proteksi.
- Menyebarkan data sensitif.
- Membuat dataset yang tidak bisa diaudit.

Bab terakhir ini menegaskan bahwa kompetensi teknis harus berjalan bersama etika, keamanan, dan tanggung jawab.

10.2 Prinsip Responsible Scraping

Responsible scraping adalah praktik mengambil data secara bertanggung jawab.

Prinsip dasarnya:

- Ambil hanya data yang memang perlu.
- Gunakan sumber legal dan aman.
- Jangan mengambil data pribadi tanpa dasar yang sah.
- Jangan membanjiri server.
- Hormati **robots.txt** dan aturan situs.
- Gunakan delay dan rate limit.
- Simpan sumber URL dan timestamp.
- Jelaskan tujuan scraping.
- Validasi data sebelum dipakai.
- Dokumentasikan batasan proyek.

RFC 9309 mendefinisikan *Robots Exclusion Protocol* sebagai mekanisme yang dapat digunakan pemilik layanan untuk menyatakan aturan akses crawler terhadap URI tertentu. Namun RFC tersebut juga menegaskan bahwa aturan ini bukan mekanisme otorisasi akses. Artinya, **robots.txt** adalah sinyal etika dan operasional, bukan izin mutlak untuk mengambil semua data. ([Datatracker](#))

Etika scraping dimulai dari pertanyaan sederhana: apakah data ini memang pantas diambil, disimpan, dan dipakai?

10.3 Menghormati robots.txt, Rate Limit, dan Terms of Service

Sebelum scraping website publik, lakukan pemeriksaan minimum.

1. Cek robots.txt

Contoh:

`https://example.org/robots.txt`

Perhatikan:

- Path yang boleh diakses.
- Path yang dilarang.
- Aturan untuk crawler.
- Crawl delay jika tersedia.

2. Cek *terms of service*

Baca ketentuan layanan jika tersedia.

Perhatikan:

- Apakah scraping dilarang.
- Apakah data boleh dipakai untuk riset.
- Apakah data boleh disimpan.
- Apakah data boleh dipublikasikan ulang.
- Apakah ada batas penggunaan.

3. Gunakan rate limit

Jangan mengirim request terlalu cepat.

Contoh prinsip:

1 request per detik lebih aman daripada 100 request per detik.

4. Jangan scraping area sensitif

Hindari:

- Halaman login.
- Data akun.
- Data transaksi.
- Data pribadi.
- Dashboard internal.
- Data yang tidak dimaksudkan untuk publik.

Bisa diakses secara teknis bukan berarti boleh diambil secara etis.

10.4 Jangan Mengambil Data Pribadi Tanpa Dasar yang Sah

Data pribadi harus diperlakukan sangat hati-hati.

Contoh data pribadi:

- Nama lengkap individu.
- Alamat rumah.
- Nomor telepon.
- Email pribadi.
- Nomor identitas.
- Riwayat kesehatan.
- Lokasi presisi.
- Data transaksi personal.
- Foto wajah.
- Data akun.

Dalam proyek mahasiswa, sebaiknya:

- Gunakan data latihan.
- Gunakan data publik yang aman.
- Hindari data individu.
- Anonimkan jika perlu.
- Jangan menyimpan data sensitif.
- Jangan memakai scraping untuk profiling orang tanpa dasar jelas.
- Jangan memasukkan data sensitif ke LLM.

Jika ragu, jangan ambil.

Data pribadi bukan bahan latihan sembarangan.

10.5 Keamanan Credential Database dan API

Credential adalah rahasia sistem. Contohnya:

- Password database.
- API key.
- Token.
- Private key.
- Connection string.
- Secret untuk aplikasi.

OWASP menekankan pentingnya pengelolaan rahasia secara terpusat, aman, dapat diaudit, dapat dirotasi, dan tidak bocor ke kode atau repository. ([OWASP Cheat Sheet Series](#))

Prinsip aman:

- Jangan menulis password langsung di script.
- Simpan konfigurasi lokal di `.env`.
- Sediakan `.env.example`.
- Masukkan `.env` ke `.gitignore`.
- Gunakan password kuat untuk proyek nyata.
- Rotasi credential jika pernah bocor.
- Jangan unggah log yang berisi credential.
- Jangan menampilkan credential di screenshot.
- Batasi akses database.

Contoh buruk:

```
password = "scraping_password"
```

Contoh lebih baik:

```
password = os.getenv("POSTGRES_PASSWORD")
```

Credential yang bocor dapat membuat seluruh pipeline runtuh.

10.6 Struktur Repository Portofolio yang Profesional

Repository portofolio harus mudah dibaca dan dijalankan.

Struktur minimal:

```
webscraping-llm-database/
├── data/
│   ├── sample/
│   └── rejected/
├── database/
│   ├── docker-compose.yml
│   └── schema_postgres.sql
├── docs/
│   └── architecture.md
├── logs/
├── scripts/
│   ├── run_pipeline.py
│   ├── query_postgres.py
│   └── query_mongodb.py
├── .env.example
├── .gitignore
├── requirements.txt
├── README.md
└── PROJECT_REPORT.md
```

Repository yang baik menjawab:

- Proyek ini untuk apa?
- Data berasal dari mana?
- Tools apa yang dipakai?
- Bagaimana cara instalasi?
- Bagaimana cara menjalankan?
- Output-nya apa?
- Bagaimana cara query?
- Apa batas etika proyek?
- Apa yang tidak dilakukan proyek ini?

README adalah komponen penting repository karena menjelaskan mengapa proyek berguna, apa yang bisa dilakukan pengguna, dan bagaimana cara menggunakannya. ([GitHub Docs](#))

Portofolio yang kuat bukan hanya kode, tetapi cerita teknis yang rapi.

10.7 Dokumentasi Proyek

Dokumentasi minimal:

- `README.md`
- `.env.example`
- `requirements.txt`
- `database/schema_postgres.sql`
- `database/docker-compose.yml`
- `docs/architecture.md`
- `PROJECT_REPORT.md`

Isi dokumentasi harus pendek tetapi jelas.

README

Berisi:

- Tujuan proyek.
- Arsitektur.
- Tools.
- Cara setup.
- Cara menjalankan pipeline.
- Cara query.
- Output.
- Batas etika.

Architecture

Berisi diagram alur:

```
Web Legal → Scraper → Cleaner → Validator →  
MongoDB Raw → LLM → PostgreSQL → Query
```

Project Report

Berisi:

- Latar belakang.
- Metode.
- Hasil.
- Kualitas data.
- Risiko.
- Keterbatasan.
- Rekomendasi lanjutan.

Dokumentasi adalah cara membuat proyek bisa dipahami tanpa kehadiran pembuatnya.

10.8 Cara Menjelaskan Proyek Saat Wawancara

Saat wawancara kerja atau magang, jangan hanya berkata:

“Saya membuat scraper.”

Itu terlalu sempit.

Jelaskan seperti ini:

“Saya membangun pipeline data end-to-end menggunakan Python dan tool open source. Pipeline mengambil data legal, membersihkan dan memvalidasi data, menyimpan raw data ke MongoDB, menyimpan processed data ke PostgreSQL, lalu memakai LLM lokal untuk ringkasan dan ekstraksi entitas. Saya juga menambahkan audit trail, logging, deduplication, dan README agar proyek bisa dijalankan ulang.”

Poin yang perlu ditekankan:

- **Masalah yang diselesaikan.**
- **Arsitektur pipeline.**
- **Data source legal.**
- **Cleaning dan validation.**
- **Database design.**
- **LLM sebagai lapisan interpretasi.**
- **Anti-halusinasi dan validasi JSON.**
- **Logging dan audit trail.**
- **Reproducibility.**
- **Etika scraping.**

Kalimat kuat:

“Saya tidak hanya mengambil data. Saya membangun pipeline yang menjaga data tetap bersih, terlacak, dan siap dianalisis.”

10.9 Tren 2026: Masa Depan Pipeline Data AI

Pipeline data AI bergerak cepat. Beberapa tren penting:

1. AI-ready crawling

Crawler tidak hanya mengambil HTML, tetapi menghasilkan Markdown, metadata, dan chunk yang siap dipakai LLM.

2. RAG pipeline

Data hasil scraping dibersihkan, dipecah menjadi chunk, diberi metadata, lalu dimasukkan ke sistem pencarian berbasis embedding.

3. Agentic data pipeline

Agen AI dapat membantu menjalankan tugas seperti mengambil data, mengecek kualitas, membuat ringkasan, dan membuat laporan. Namun tetap perlu guardrail, logging, dan validasi.

4. Local LLM

Model lokal makin penting untuk privasi, biaya, eksperimen kampus, dan kontrol data.

5. Data governance

Data harus punya aturan: asal data, izin pakai, kualitas, keamanan, retensi, dan audit.

NIST merilis *Generative AI Profile* pada 26 Juli 2024 untuk membantu organisasi mengidentifikasi risiko unik AI generatif dan tindakan pengelolaan risikonya. Ini relevan untuk pipeline LLM karena output model perlu dikelola dari sisi risiko, tata kelola, dan kepercayaan. ([NIST](#))

Masa depan bukan hanya scraping lebih cepat, tetapi pipeline yang lebih bertanggung jawab, auditable, dan AI-ready.

Bagian Praktik

10.10 Praktik 1 — Checklist Legal dan Etika Scraping

Buat folder:

```
mkdir -p docs
```

Buat file:

```
nano docs/ethical_scraping_checklist.md
```

Isi:

```
# Checklist Legal dan Etika Scraping

## Identitas Proyek

- Nama proyek:
- Pembuat:
- Tujuan:
- Tanggal:
- Sumber data:

## Legal dan Aturan Situs

- [ ] Sumber data bersifat publik.
- [ ] Tujuan scraping jelas.
- [ ] robots.txt sudah dicek.
- [ ] Terms of service sudah dicek jika tersedia.
- [ ] Tidak mengambil data login.
- [ ] Tidak mengambil data privat.
- [ ] Tidak bypass proteksi.
- [ ] Tidak mengambil data pribadi tanpa dasar sah.

## Teknis dan Beban Server

- [ ] Request dibatasi.
- [ ] Delay diterapkan.
- [ ] Retry tidak agresif.
- [ ] Timeout diterapkan.
- [ ] Error handling tersedia.
- [ ] Log pipeline tersedia.

## Data Quality

- [ ] source_url disimpan.
- [ ] scraped_at disimpan.
- [ ] content_hash dibuat.
- [ ] Data duplikat dicek.
```

- [] Data gagal validasi disimpan.
- [] Laporan kualitas data dibuat.

LLM

- [] Prompt melarang halusinasi.
- [] Output LLM disimpan mentah.
- [] Output JSON divalidasi.
- [] Nama model disimpan.
- [] Hasil LLM tidak dianggap fakta tanpa sumber.

Publikasi

- [] Credential tidak ikut terunggah.
- [] .env masuk .gitignore.
- [] README menjelaskan batas etika.
- [] Dataset contoh aman untuk publik.

Checklist ini menjadi pagar sebelum proyek dipublikasikan.

10.11 Praktik 2 — Menambahkan .gitignore

Buat file:

```
nano .gitignore
```

Isi:

```
# Virtual environment
.venv/

# Local secrets
.env

# Python cache
__pycache__/
*.pyc

# Logs
logs/*.log
logs/**/*.log

# Local database files
*.db
*.sqlite
*.sqlite3

# Temporary data
data/tmp/

# Rejected sensitive data
data/rejected/*.json
data/rejected/**/*.json
```

Penjelasan:

- `.env` tidak boleh dipublikasikan.
- `.venv/` tidak perlu diunggah.
- Cache Python tidak perlu diunggah.
- Database lokal besar tidak perlu diunggah.
- Rejected data bisa berisi data bermasalah, jadi jangan asal publikasikan.

Repository publik harus bersih dari rahasia dan data sensitif.

10.12 Praktik 3 — Menyimpan Credential di .env

Buat file contoh:

```
nano .env.example
```

Isi:

```
PROJECT_NAME=webscraping-llm-database

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=scraping_db
POSTGRES_USER=scraping_user
POSTGRES_PASSWORD=change_me

MONGO_URI=mongodb://localhost:27017
MONGO_DB=scraping_db

OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b
```

Buat file lokal:

```
cp .env.example .env
```

Edit .env:

```
nano .env
```

Aturan:

- `.env.example` boleh dibagikan.
- `.env` tidak boleh dibagikan.
- Password nyata jangan ditaruh di README.
- Jangan menampilkan `.env` dalam screenshot.

Di Python:

```
import os
from dotenv import load_dotenv

load_dotenv()

password = os.getenv("POSTGRES_PASSWORD")
```

Credential harus dipanggil dari environment, bukan ditanam di kode.

10.13 Praktik 4 — Membuat README Final

Buat file:

```
nano README.md
```

Isi:

```
# Web Scraping + Database + Open Source LLM Pipeline
```

```
## Ringkasan
```

Project ini adalah pipeline data end-to-end untuk mahasiswa. Pipeline mengambil data artikel legal, membersihkan data, menyimpan raw data ke MongoDB, memproses data bersih ke PostgreSQL, lalu memakai LLM lokal untuk ringkasan dan ekstraksi entitas.

```
## Tujuan
```

- Membangun pipeline data modern.
- Melatih scraping yang bertanggung jawab.
- Menerapkan cleaning dan validation.
- Menyimpan data ke database open source.
- Menggunakan LLM lokal untuk ekstraksi informasi.
- Membuat proyek portofolio yang reproducible.

```
## Arsitektur
```

```
```text
Web Legal
 ↓
Scraper
 ↓
Cleaner + Validator
 ↓
MongoDB Raw Data
 ↓
Ollama LLM
 ↓
PostgreSQL Processed Data
 ↓
Query + Report
```



## Tools

- Python
- BeautifulSoup
- lxml
- Pydantic
- PostgreSQL
- MongoDB Community
- Docker Compose
- Ollama

## Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

## Jalankan Database

```
cd database
docker compose up -d
cd ..
```

## Jalankan Pipeline

```
python scripts/run_pipeline.py
```

## Query

```
python scripts/query_postgres.py
python scripts/query_mongodb.py
```

## Output

- Raw data tersimpan di MongoDB.
- Processed data tersimpan di PostgreSQL.
- Hasil LLM tersimpan di tabel ekstraksi.
- Log pipeline tersimpan di folder logs.

## Etika

Project ini menggunakan data latihan legal. Untuk website publik, pengguna wajib memeriksa robots.txt, terms of service, rate limit, dan tidak mengambil data pribadi atau data privat.

## Batasan

- Project ini untuk pembelajaran.
- Data contoh terbatas.
- Output LLM harus divalidasi.
- Tidak digunakan untuk bypass proteksi atau mengambil data sensitif.

## Lisensi

Gunakan sesuai kebutuhan pembelajaran dan riset dengan tetap menjaga etika data.

README yang baik harus menjawab **apa**, **mengapa**, **bagaimana**, **output**, dan **batasan**.

---

## 10.14 Praktik 5 – Membuat Diagram Arsitektur Pipeline

Buat file:

```
```bash
nano docs/architecture.md
```

Isi:

```
# Diagram Arsitektur Pipeline
```

```
```text
```

```
+-----+
| Web / HTML Legal |
+-----+
```

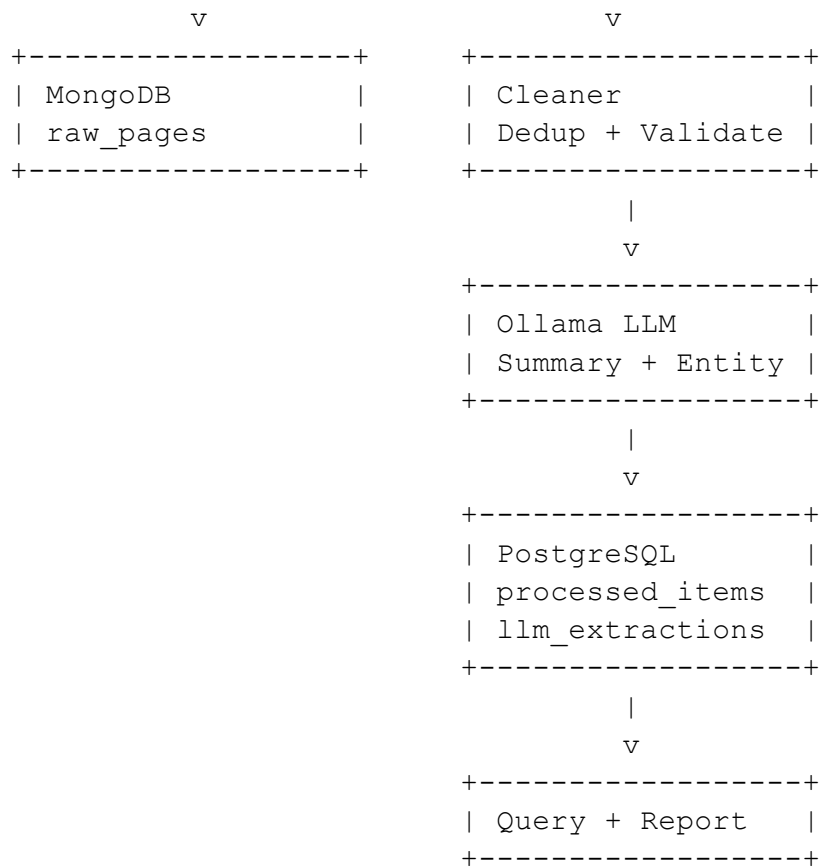
```
 |
 v
```

```
+-----+
| Scraper Python |
+-----+
```

```
 |
 v
```

```
+-----+
| Raw Data |
| source_url |
| scraped_at |
+-----+
```

```
 |
 +-----+
 |
```



## Audit Trail

Setiap record menyimpan:

- source\_url
- scraped\_at
- content\_hash
- validation\_status
- model\_name
- created\_at
- run\_id

Diagram ini membantu pembaca memahami proyek tanpa membaca seluruh kode.

---

## 10.15 Praktik 6 – Final Checklist Portofolio

Buat file:

```
```bash
```

```
nano docs/final_portfolio_checklist.md
```

Isi:

Final Checklist Portofolio

Kode

- [] Script utama tersedia.
- [] Fungsi dipisah dengan rapi.
- [] Error handling tersedia.
- [] Logging tersedia.
- [] Tidak ada credential di kode.

Data

- [] Sumber data jelas.
- [] Raw data disimpan.
- [] Data bersih tersedia.
- [] Data rejected disimpan jika perlu.
- [] content_hash dibuat.
- [] source_url dan scraped_at tersedia.

Database

- [] PostgreSQL berjalan.
- [] MongoDB berjalan.
- [] Schema tersedia.
- [] Query contoh tersedia.
- [] Index dasar dibuat.

LLM

- [] Model lokal berjalan.
- [] Prompt tersedia.
- [] Output mentah disimpan.
- [] Output JSON divalidasi.
- [] Nama model disimpan.

Dokumentasi

- [] README lengkap.
- [] .env.example tersedia.
- [] requirements.txt tersedia.
- [] docker-compose.yml tersedia.
- [] Diagram arsitektur tersedia.
- [] Laporan proyek tersedia.

Etika dan Keamanan

- [] robots.txt dicek jika memakai situs publik.
- [] Terms of service dicek jika tersedia.
- [] Tidak mengambil data pribadi.
- [] Tidak bypass proteksi.
- [] Rate limit diterapkan.
- [] .env masuk .gitignore.

Checklist ini adalah alat kontrol sebelum repository dipublikasikan.

10.16 Praktik 7 — Menulis Laporan Singkat Proyek Akhir

Buat file:

```
nano PROJECT_REPORT.md
```

Isi:

```
# Laporan Singkat Proyek Akhir
```

```
## Judul
```

```
Pipeline Data Artikel dengan Web Scraping, Database, dan  
Open Source LLM
```

```
## Ringkasan
```

```
Proyek ini membangun pipeline data end-to-end untuk  
mengambil data artikel legal, membersihkan dan  
memvalidasi data, menyimpan raw data ke MongoDB,  
menyimpan data bersih ke PostgreSQL, dan memakai LLM  
lokal untuk membuat ringkasan serta ekstraksi entitas.  
Proyek ini dirancang sebagai portofolio mahasiswa di  
bidang data engineering dan AI engineering.
```

```
## Latar Belakang
```

```
Data web sering menjadi bahan penting untuk riset,  
jurnalisme data, analisis bisnis, dan AI. Namun data  
hasil scraping sering kotor, duplikat, tidak konsisten,  
dan berisiko jika tidak ditangani secara etis. Karena  
itu, pipeline ini menekankan scraping bertanggung jawab,  
data quality, audit trail, dan validasi output LLM.
```

```
## Arsitektur
```

```
Web Legal → Scraper → Cleaner → Validator → MongoDB Raw  
→ Ollama LLM → PostgreSQL Processed → Query
```

```
## Metode
```

1. Mengambil data artikel legal.
2. Membersihkan teks.
3. Menghapus duplikasi.
4. Membuat content_hash.
5. Menyimpan raw data.
6. Memvalidasi data.
7. Mengirim teks ke LLM lokal.
8. Memvalidasi output JSON.
9. Menyimpan hasil ke database.

10. Menjalankan query analisis.

Output

- Raw data di MongoDB.
- Processed data di PostgreSQL.
- Hasil ringkasan LLM.
- Hasil ekstraksi entitas.
- Log pipeline.
- Dokumentasi dan checklist etika.

Risiko dan Mitigasi

Risiko	Mitigasi
Data kotor	Cleaning dan validation
Data duplikat	URL unique dan content_hash
Output LLM salah	Prompt anti-halusinasi dan validasi schema
Credential bocor	.env dan .gitignore
Scraping tidak etis	Checklist legal dan etika
Pipeline sulit diulang	README, requirements, docker-compose

Batasan

- Dataset latihan masih kecil.
- Pipeline memakai contoh data legal.
- Output LLM tetap perlu audit manusia.
- Belum mencakup dashboard visual.
- Belum mencakup pipeline RAG penuh.

Kesimpulan

Proyek ini menunjukkan bahwa web scraping modern tidak boleh berhenti pada pengambilan data. Pipeline yang baik harus legal, bersih, tervalidasi, tersimpan rapi, dapat diaudit, dan mampu menggunakan LLM secara aman. Proyek ini layak menjadi portofolio awal untuk mahasiswa yang ingin masuk ke data engineering, AI engineering, riset digital, atau automation pipeline.

10.17 Praktik 8 — Cek Repository Sebelum Dipublikasikan

Jalankan perintah berikut:

```
find . -name ".env" -print
```

Pastikan .env ada lokal, tetapi tidak akan ikut dipublikasikan karena ada di .gitignore.

Cek file penting:

```
ls -la
```

Cek struktur docs:

```
ls docs
```

Cek dependency:

```
cat requirements.txt
```

Cek apakah credential tertulis di kode:

```
grep -R "PASSWORD" scripts database docs README.md
```

Jika muncul password nyata di README atau kode, hapus dan pindahkan ke .env. Sebelum publikasi, lakukan audit kecil. Jangan menyesal setelah credential bocor.

10.18 Praktik 9 — Cara Menjelaskan Project dalam 60 Detik

Tuliskan pitch singkat di PROJECT_REPORT.md.

Contoh:

Pitch 60 Detik

Saya membangun pipeline data end-to-end menggunakan Python dan tool open source. Pipeline ini mengambil data artikel legal, membersihkan dan memvalidasi data, menyimpan raw data ke MongoDB, menyimpan processed data ke PostgreSQL, lalu memakai LLM lokal untuk membuat ringkasan dan ekstraksi entitas. Saya menambahkan deduplication, content_hash, audit trail, logging, .env untuk credential, README, diagram arsitektur, dan checklist etika. Fokus proyek ini bukan hanya scraping, tetapi membangun pipeline data yang reproducible, aman, dan bisa dipertanggungjawabkan.

Pitch ini bisa dipakai untuk:

- Wawancara magang.
- Presentasi kelas.
- Portofolio Git.
- Diskusi dengan dosen.
- Demo proyek akhir.

10.19 Praktik 10 — Final Audit Command

Buat file:

```
nano scripts/final_audit.py
```

Isi:

```
from pathlib import Path

required_files = [
    "README.md",
    ".env.example",
    ".gitignore",
    "requirements.txt",
    "PROJECT_REPORT.md",
    "docs/architecture.md",
    "docs/ethical_scraping_checklist.md",
    "docs/final_portfolio_checklist.md",
    "database/docker-compose.yml",
    "database/schema_postgres.sql",
    "scripts/run_pipeline.py",
    "scripts/query_postgres.py",
    "scripts/query_mongodb.py",
]

print("Final Audit Repository")
print("=" * 40)

missing = []

for file_path in required_files:
    path = Path(file_path)

    if path.exists():
        print(f"[OK]          {file_path}")
    else:
        print(f"[MISSING] {file_path}")
        missing.append(file_path)

print("=" * 40)

if missing:
    print("Status: belum siap publikasi.")
    print("File yang belum ada:")
    for item in missing:
        print(f"- {item}")
else:
    print("Status: struktur repository siap publikasi.")
```

Jalankan:

```
python scripts/final_audit.py
```

Output ideal:

```
Status: struktur repository siap publikasi.
```

10.20 Struktur Akhir Repository Siap Publik

```
webscraping-llm-database/  
├── data/  
│   ├── sample/  
│   └── rejected/  
├── database/  
│   ├── docker-compose.yml  
│   └── schema_postgres.sql  
├── docs/  
│   ├── architecture.md  
│   ├── ethical_scraping_checklist.md  
│   └── final_portfolio_checklist.md  
├── logs/  
├── scripts/  
│   ├── run_pipeline.py  
│   ├── query_postgres.py  
│   ├── query_mongodb.py  
│   └── final_audit.py  
├── .env.example  
├── .gitignore  
├── requirements.txt  
├── README.md  
└── PROJECT_REPORT.md
```

Output akhir mahasiswa:

- **Checklist etika.**
- **Repository siap publik.**
- **Dokumentasi pipeline.**
- **Diagram arsitektur.**
- **Laporan proyek akhir.**

10.21 Checklist Bab 10

Mahasiswa dianggap selesai Bab 10 jika mampu:

- ☐ Menjelaskan prinsip responsible scraping
- ☐ Memahami robots.txt dan batasannya
- ☐ Memahami pentingnya rate limit
- ☐ Tidak mengambil data pribadi tanpa dasar sah
- ☐ Mengamankan credential dengan .env
- ☐ Menambahkan .gitignore
- ☐ Membuat README
- ☐ Membuat diagram arsitektur
- ☐ Membuat checklist etika
- ☐ Membuat checklist portofolio
- ☐ Membuat PROJECT_REPORT.md
- ☐ Menjelaskan proyek dalam 60 detik
- ☐ Melakukan final audit repository
- ☐ Menjelaskan tren AI-ready crawling, RAG, agentic pipeline, local LLM, dan data governance

10.22 Kesalahan Umum Bab 10

1. Menganggap data publik selalu bebas diambil

Masalah:

- Data bisa publik tetapi tidak bebas untuk scraping massal.
- Ada aturan situs.
- Ada risiko privasi.

Solusi:

Cek robots.txt, terms of service, tujuan, dan risiko.

2. Mengunggah .env

Masalah:

- Credential bocor.

Solusi:

.env masuk .gitignore.
Gunakan .env.example untuk template.

3. README terlalu pendek

Masalah:

- Orang lain tidak tahu cara menjalankan proyek.

Solusi:

README harus berisi tujuan, arsitektur, setup, run, query, output, dan etika.

4. Tidak punya laporan proyek

Masalah:

- Portofolio terlihat seperti kumpulan script, bukan project engineering.

Solusi:

Buat PROJECT_REPORT.md.

5. Tidak menyebut batasan proyek

Masalah:

- Terlihat berlebihan dan tidak kritis.

Solusi:

Tulis keterbatasan, risiko, dan rencana pengembangan.

6. Tidak bisa menjelaskan proyek

Masalah:

- Kode ada, tetapi konsep tidak dikuasai.

Solusi:

Latih pitch 60 detik.

10.23 Ringkasan Bab

Bab terakhir ini menutup buku dengan sikap profesional. Mahasiswa belajar bahwa web scraping, database, dan LLM bukan sekadar keterampilan teknis, tetapi bagian dari tanggung jawab data. Pipeline yang baik harus etis, aman, legal, hemat resource, terdokumentasi, bisa diaudit, dan bermanfaat.

Bab ini menekankan *responsible scraping*, penghormatan terhadap `robots.txt`, rate limit, dan aturan situs, serta larangan mengambil data pribadi tanpa dasar yang sah. Mahasiswa juga belajar menjaga credential, membuat `.gitignore`, menyusun README, membuat diagram arsitektur, menyusun checklist portofolio, menulis laporan proyek akhir, dan menjelaskan proyek secara profesional saat wawancara.

Output akhir Bab 10:

- Checklist etika.
- Repository siap publik.
- Dokumentasi pipeline.
- Diagram arsitektur.
- Laporan proyek akhir.
- Pitch proyek 60 detik.
- Final audit repository.

Pesan strategis Bab 10:

Masa depan pipeline data AI bukan hanya tentang mengambil data lebih cepat, tetapi tentang membangun sistem yang legal, aman, transparan, reproducible, dan layak dipercaya.

Lampiran A — Instalasi Tools Open Source

Lampiran ini berisi panduan instalasi ringkas untuk tools utama buku **Web Scraping + Database dengan Open Source LLM: Pipeline Data Modern, 2026**. Lampiran ini selaras dengan struktur lampiran yang disarankan dalam dokumen outline buku, yaitu instalasi Python, *virtual environment*, Docker, database, Ollama, Playwright, Scrapy, dan Crawl4AI.

A.1 Python, pip, dan Virtual Environment

Python dipakai sebagai bahasa utama pipeline. Modul `venv` pada Python digunakan untuk membuat *virtual environment* yang ringan dan terisolasi dari paket Python global. ([Python documentation](#))

```
python3 --version
python3 -m pip --version
```

Instal paket dasar:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Buat virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

Perbarui pip:

```
python -m pip install --upgrade pip
```

Buat file dependency:

```
pip freeze > requirements.txt
```

A.2 Docker dan Docker Compose

Docker dipakai untuk menjalankan database secara terisolasi. Docker Compose dipakai untuk mendefinisikan dan menjalankan beberapa service dalam satu file YAML, termasuk PostgreSQL dan MongoDB. ([Docker Documentation](#))

Cek instalasi:

```
docker --version
docker compose version
```

Contoh menjalankan service:

```
docker compose up -d
docker ps
```

Menghentikan service:

```
docker compose down
```

A.3 PostgreSQL

PostgreSQL adalah sistem database objek-relasional open source yang kuat dan menggunakan serta memperluas SQL. ([PostgreSQL](#))

Contoh service PostgreSQL di docker-compose.yml:

```
services:
  postgres:
    image: postgres:16
    container_name: scraping_postgres
    environment:
      POSTGRES_DB: scraping_db
      POSTGRES_USER: scraping_user
      POSTGRES_PASSWORD: scraping_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

volumes:

```
postgres_data:
```

Jalankan:

```
docker compose up -d
```

A.4 MongoDB

MongoDB dipakai untuk menyimpan dokumen mentah dan JSON semi-terstruktur. MongoDB menyediakan image komunitas resmi yang dapat dijalankan melalui Docker untuk kebutuhan belajar dan pengembangan. ([MongoDB](#))

Contoh service MongoDB:

```
services:
  mongo:
    image:
      mongodb/mongodb-community-server:7.0-ubuntu2204
    container_name: scraping_mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
```

volumes:

```
mongo_data:
```

Cek container:

```
docker ps
```

A.5 SQLite

SQLite cocok untuk latihan lokal karena ringan, tidak membutuhkan server, dan tersedia sebagai file database tunggal. SQLite juga memiliki modul bawaan di Python melalui sqlite3. ([Python documentation](#))

Instal:

```
sudo apt update
sudo apt install sqlite3
```

Cek versi:

```
sqlite3 --version
```

Contoh membuat database:

```
sqlite3 database/scraping.db
```

A.6 Ollama

Ollama dipakai untuk menjalankan model LLM lokal. API lokal Ollama tersedia secara default pada `http://localhost:11434/api`, dan endpoint `/api/generate` dapat digunakan untuk menghasilkan respons dari model. ([Ollama Documentation](#))

Instalasi umum:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Cek versi:

```
ollama --version
```

Unduh model ringan:

```
ollama pull llama3.2:3b
```

Tes model:

```
ollama run llama3.2:3b
```

A.7 Playwright

Playwright dipakai untuk scraping website dinamis yang memuat data melalui JavaScript. Playwright Python menyediakan API untuk membuka halaman, klik tombol, mengisi form, mengambil screenshot, dan membaca konten hasil render. ([Playwright](#))

Instal:

```
pip install playwright
playwright install
```

Tes singkat:

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://example.org")
    print(page.title())
    browser.close()
```

A.8 Scrapy

Scrapy adalah framework tingkat tinggi untuk web crawling dan web scraping yang dipakai untuk menjelajah website dan mengekstrak data terstruktur. ([Scrapy](#))

Instal:

```
pip install scrapy
```

Cek versi:

```
scrapy version
```

Buat project:

```
scrapy startproject scrape_scrapy
```

Jalankan spider:

```
scrapy crawl nama_spider -O items.json
```

Scrapy mendukung feed exports untuk menyimpan hasil ke format seperti JSON, JSON Lines, dan CSV. ([Scrapy](#))

A.9 Crawl4AI

Crawl4AI dipakai untuk crawling yang ramah LLM, termasuk menghasilkan Markdown dan melakukan ekstraksi yang lebih siap untuk pipeline AI. Dokumentasi resminya menyediakan instalasi via pip, perintah setup, dan diagnosa dengan crawl4ai-doctor. ([Crawl4AI Documentation](#))

Instal:

```
pip install crawl4ai
crawl4ai-setup
crawl4ai-doctor
```

Contoh penggunaan awal:

```
import asyncio
from crawl4ai import AsyncWebCrawler

async def main():
    async with AsyncWebCrawler() as crawler:
        result = await
crawler.arun(url="https://example.org")
        print(result.markdown)

asyncio.run(main())
```

Lampiran B — Template Struktur Folder Proyek

Struktur folder ini mengikuti arah proyek utama buku: pipeline data artikel publik dengan alur **Web → Scraper → Cleaner → Validator → Database → LLM → Query/API**.

```
webscraping-llm-database/
├── data/
│   ├── raw/
│   ├── clean/
│   └── rejected/
├── scripts/
│   ├── scrape_basic.py
│   ├── scrape_scrapy/
│   ├── scrape_playwright.py
│   ├── clean_data.py
│   ├── validate_data.py
│   ├── llm_extract.py
│   └── run_pipeline.py
├── database/
│   ├── schema.sql
│   └── seed.sql
├── logs/
├── docker-compose.yml
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

B.1 Fungsi Folder

- **data/raw/**
Menyimpan data mentah dari scraping: HTML, JSON Lines, raw text, atau response awal.
- **data/clean/**
Menyimpan data hasil cleaning, deduplication, dan validasi.
- **data/rejected/**
Menyimpan data yang gagal validasi beserta alasan penolakan.
- **scripts/**
Menyimpan seluruh script Python pipeline.
- **database/**
Menyimpan schema, seed data, dan script SQL.
- **logs/**
Menyimpan log pipeline, error, dan hasil debugging.
- **requirements.txt**
Menyimpan daftar paket Python.

- **.env.example**
Template konfigurasi tanpa credential asli.
- **.gitignore**
Daftar file yang tidak boleh dipublikasikan.
- **README.md**
Dokumentasi utama proyek.

B.2 Template .env.example

```
PROJECT_NAME=webscraping-llm-database

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=scraping_db
POSTGRES_USER=scraping_user
POSTGRES_PASSWORD=change_me

MONGO_URI=mongodb://localhost:27017
MONGO_DB=scraping_db

OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b
```

B.3 Template .gitignore

```
.venv/
.env
__pycache__/*
*.pyc

logs/*.log
logs/**/*.log

*.db
*.sqlite
*.sqlite3

data/rejected/*.json
data/rejected/**/*.json
```

Lampiran C — Template Schema Database

Schema ini memakai prinsip sederhana: **raw data dipisah dari clean data, hasil LLM dipisah dari data utama, dan setiap proses pipeline dicatat dalam log.**

C.1 Tabel raw_pages

```
CREATE TABLE IF NOT EXISTS raw_pages (  
    id SERIAL PRIMARY KEY,  
    source_url TEXT NOT NULL,  
    title TEXT,  
    raw_text TEXT,  
    scraped_at TIMESTAMP,  
    content_hash TEXT,  
    validation_status TEXT DEFAULT 'raw',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX IF NOT EXISTS idx_raw_pages_source_url  
ON raw_pages(source_url);  
  
CREATE INDEX IF NOT EXISTS idx_raw_pages_hash  
ON raw_pages(content_hash);
```

Fungsi:

- Menyimpan data mentah.
- Menyediakan bahan audit.
- Menyimpan sumber URL dan waktu pengambilan.

C.2 Tabel scraped_items

```
CREATE TABLE IF NOT EXISTS scraped_items (  
    id SERIAL PRIMARY KEY,  
    source_url TEXT,  
    title TEXT NOT NULL,  
    url TEXT NOT NULL UNIQUE,  
    raw_text TEXT,  
    scraped_at TIMESTAMP,  
    content_hash TEXT,  
    validation_status TEXT DEFAULT 'scraped',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_url  
ON scraped_items(url);  
  
CREATE INDEX IF NOT EXISTS idx_scraped_items_hash  
ON scraped_items(content_hash);
```

Fungsi:

- Menyimpan hasil ekstraksi awal.
- Menjaga URL tetap unik.
- Menyimpan hash untuk deteksi duplikasi.

C.3 Tabel clean_items

```
CREATE TABLE IF NOT EXISTS clean_items (  
    id SERIAL PRIMARY KEY,  
    source_url TEXT,  
    title TEXT NOT NULL,  
    url TEXT NOT NULL UNIQUE,  
    clean_text TEXT NOT NULL,  
    content_hash TEXT,  
    scraped_at TIMESTAMP,  
    validation_status TEXT DEFAULT 'valid',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX IF NOT EXISTS idx_clean_items_url  
ON clean_items(url);  
  
CREATE INDEX IF NOT EXISTS idx_clean_items_hash  
ON clean_items(content_hash);
```

Fungsi:

- Menyimpan data bersih.
- Menjadi input utama LLM.
- Menjadi basis analisis dan query.

C.4 Tabel llm_extractions

```
CREATE TABLE IF NOT EXISTS llm_extractions (  
    id SERIAL PRIMARY KEY,  
    item_url TEXT NOT NULL,  
    model_name TEXT NOT NULL,  
    llm_output_json JSONB,  
    summary TEXT,  
    entities_json JSONB,  
    topics_json JSONB,  
    validation_status TEXT DEFAULT 'valid',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE(item_url, model_name)  
);  
  
CREATE INDEX IF NOT EXISTS idx_llm_extractions_item_url  
ON llm_extractions(item_url);  
  
CREATE INDEX IF NOT EXISTS idx_llm_extractions_model  
ON llm_extractions(model_name);
```

Fungsi:

- Menyimpan hasil LLM.
- Menyimpan nama model.
- Menyimpan JSON hasil ekstraksi.
- Mendukung audit output AI.

C.5 Tabel pipeline_logs

```
CREATE TABLE IF NOT EXISTS pipeline_logs (  
    id SERIAL PRIMARY KEY,  
    run_id TEXT UNIQUE,  
    started_at TIMESTAMP,  
    finished_at TIMESTAMP,  
    status TEXT,  
    raw_count INTEGER DEFAULT 0,  
    clean_count INTEGER DEFAULT 0,  
    rejected_count INTEGER DEFAULT 0,  
    llm_count INTEGER DEFAULT 0,  
    error_message TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Fungsi:

- Mencatat setiap eksekusi pipeline.
- Menyimpan status sukses/gagal.
- Menyimpan jumlah data yang diproses.
- Menyimpan pesan error jika pipeline gagal.

C.6 Field Penting

Field	Fungsi
<code>id</code>	Identitas unik record
<code>source_url</code>	URL asal data
<code>title</code>	Judul item/artikel
<code>raw_text</code>	Teks mentah
<code>clean_text</code>	Teks bersih
<code>content_hash</code>	Sidik konten untuk deduplication
<code>scraped_at</code>	Waktu data diambil
<code>model_name</code>	Nama model LLM
<code>llm_output_json</code>	Output LLM dalam format JSON
<code>validation_status</code>	Status validasi data

Lampiran D — 30 Prompt LLM untuk Data Extraction

Gunakan prompt ini untuk memproses teks hasil scraping. Prinsip utama: **output harus JSON valid, tidak boleh menambah fakta dari luar teks, dan wajib menyatakan kosong jika informasi tidak tersedia.**

D.1 Ekstraksi Judul dan Ringkasan

Prompt 1

Ekstrak judul utama dan ringkasan pendek dari teks berikut.

Aturan:

- Gunakan hanya informasi dari teks.
- Jangan menambah fakta dari luar teks.
- Output hanya JSON valid.

Format:

```
{
  "title": "",
  "summary": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 2

Buat ringkasan maksimal 2 kalimat dari teks berikut.

Aturan:

- Jangan menebak.
- Jangan menambah informasi baru.
- Output JSON valid.

Format:

```
{
  "summary_short": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 3

Buat ringkasan eksekutif dari teks berikut untuk pembaca non-teknis.

Format JSON:

```
{
  "executive_summary": "",
  "key_message": ""
}
```

Teks:

```
{{clean_text}}
```

D.2 Ekstraksi Entitas

Prompt 4

Ekstrak entitas dari teks berikut.

Kategori entitas:

PERSON, ORGANIZATION, LOCATION, TECHNOLOGY, DATABASE,
CONCEPT, RISK, OTHER

Output hanya JSON valid.

Format:

```
{
  "entities": [
    {
      "name": "",
      "type": ""
    }
  ]
}
```

Teks:

```
{{clean_text}}
```

Prompt 5

Identifikasi nama teknologi, database, framework, dan bahasa pemrograman yang muncul eksplisit dalam teks.

Jangan menambah tools yang tidak disebutkan.

Format:

```
{  
  "technologies": []  
}
```

Teks:

```
{{clean_text}}
```

Prompt 6

Ekstrak organisasi atau institusi yang disebutkan dalam teks.

Jika tidak ada, gunakan list kosong.

Format:

```
{  
  "organizations": []  
}
```

Teks:

```
{{clean_text}}
```

D.3 Ekstraksi Topik

Prompt 7

Ekstrak maksimal 5 topik utama dari teks.

Aturan:

- Topik harus berasal dari isi teks.
- Jangan terlalu umum.
- Output JSON valid.

Format:

```
{  
  "topics": []  
}
```

Teks:

```
{{clean_text}}
```


Prompt 8

Kelompokkan topik teks ke dalam kategori berikut:
web_scraping, database, llm, data_quality, ethics, security, other

Format:

```
{  
  "topic_categories": []  
}
```

Teks:

```
{{clean_text}}
```

Prompt 9

Tentukan satu topik paling dominan dari teks.

Format:

```
{  
  "primary_topic": "",  
  "reason": ""  
}
```

Teks:

```
{{clean_text}}
```

D.4 Klasifikasi Artikel

Prompt 10

Klasifikasikan artikel berikut.

Kategori:

tutorial, berita, opini, dokumentasi, riset, pengumuman,
other

Format:

```
{  
  "category": "",  
  "confidence_reason": ""  
}
```

Teks:

```
{{clean_text}}
```


Prompt 11

Tentukan tingkat teknis artikel berikut.

Pilihan:

beginner, intermediate, advanced

Format:

```
{  
  "technical_level": "",  
  "reason": ""  
}
```

Teks:

{{clean_text}}

Prompt 12

Tentukan apakah artikel ini cocok untuk mahasiswa.

Format:

```
{  
  "suitable_for_students": true,  
  "reason": ""  
}
```

Teks:

{{clean_text}}

D.5 Konversi Teks ke JSON

Prompt 13

Ubah teks berikut menjadi JSON terstruktur.

Format:

```
{  
  "title": "",  
  "summary": "",  
  "topics": [],  
  "entities": [],  
  "risks": []  
}
```

Teks:

{{clean_text}}

Prompt 14

Konversi teks berikut menjadi metadata artikel.

Format:

```
{
  "metadata": {
    "title": "",
    "main_topic": "",
    "keywords": [],
    "audience": "",
    "difficulty": ""
  }
}
```

Teks:

```
{{clean_text}}
```

Prompt 15

Buat output JSON untuk pipeline database.

Format:

```
{
  "clean_summary": "",
  "keywords": [],
  "named_entities": [],
  "data_quality_notes": []
}
```

Teks:

```
{{clean_text}}
```

D.6 Validasi JSON

Prompt 16

Periksa apakah JSON berikut valid dan sesuai schema.

Schema:

```
{
  "title": "string",
  "summary": "string",
  "topics": "array",
  "entities": "array"
}
```

Output:

```
{
  "is_valid": true,
  "errors": []
}
```

JSON:

```
{{llm_output_json}}
```

Prompt 17

Cek apakah JSON berikut mengandung field kosong yang penting.

Field wajib:

title, summary, topics

Output:

```
{
  "has_empty_required_fields": false,
  "empty_fields": []
}
```

JSON:

```
{{llm_output_json}}
```

Prompt 18

Perbaiki JSON berikut agar menjadi JSON valid.
Jangan menambah informasi baru.

Output hanya JSON valid.

JSON:

```
{{broken_json}}
```

D.7 Deteksi Data Kosong

Prompt 19

Periksa apakah teks berikut cukup informatif untuk diproses.

Format:

```
{
  "is_empty_or_useless": false,
  "reason": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 20

Deteksi apakah teks berikut hanya berisi navigasi, menu, atau footer.

Format:

```
{
  "is_boilerplate": false,
  "reason": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 21

Tentukan apakah teks berikut layak masuk database clean_items.

Format:

```
{
  "is_database_ready": true,
  "issues": []
}
```

Teks:

```
{{clean_text}}
```

D.8 Deteksi Informasi Sensitif

Prompt 22

Deteksi apakah teks berikut mengandung data pribadi atau informasi sensitif.

Jangan menyalin data sensitif ke output.

Format:

```
{
  "contains_sensitive_data": false,
  "sensitive_categories": [],
  "recommendation": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 23

Periksa apakah teks berikut aman untuk dipakai sebagai dataset latihan mahasiswa.

Format:

```
{
  "safe_for_student_dataset": true,
  "risks": []
}
```

Teks:

```
{{clean_text}}
```

Prompt 24

Identifikasi bagian teks yang perlu disamarkan sebelum publikasi dataset.

Jangan tampilkan ulang data sensitif secara lengkap.

Format:

```
{
  "needs_redaction": false,
  "redaction_categories": [],
  "notes": ""
}
```

Teks:

```
{{clean_text}}
```

D.9 Ringkasan Eksekutif

Prompt 25

Buat ringkasan eksekutif 1 paragraf untuk pimpinan non-teknis.

Aturan:

- Maksimal 120 kata.
- Tajam.
- Tidak menambah fakta.
- Output JSON valid.

Format:

```
{
  "executive_summary": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 26

Buat 3 butir insight strategis dari teks.

Format:

```
{
  "strategic_insights": []
}
```

Teks:

```
{{clean_text}}
```

Prompt 27

Buat rekomendasi tindakan berdasarkan teks.

Aturan:

- Rekomendasi harus langsung terkait isi teks.
- Jangan menambah asumsi eksternal.

Format:

```
{
  "recommendations": []
}
```

Teks:


```
{{clean_text}}
```

D.10 Pembuatan Metadata

Prompt 28

Buat metadata dokumen dari teks.

Format:

```
{
  "document_type": "",
  "main_topic": "",
  "keywords": [],
  "audience": "",
  "language": "id"
}
```

Teks:

```
{{clean_text}}
```

Prompt 29

Buat metadata untuk kebutuhan RAG.

Format:

```
{
  "chunk_title": "",
  "summary": "",
  "keywords": [],
  "entities": [],
  "source_quality": ""
}
```

Teks:

```
{{clean_text}}
```

Prompt 30

Buat metadata audit untuk hasil ekstraksi.

Format:

```
{
  "source_url_required": true,
  "timestamp_required": true,
  "validation_required": true,
  "llm_review_required": true,
  "notes": ""
}
```

```
Teks:
{{clean_text}}
```

Lampiran E — Checklist Responsible Scraping

Checklist ini wajib digunakan sebelum menjalankan scraping pada website publik.

E.1 Legal dan Aturan Situs

- ☐ Cek robots.txt
- ☐ Cek terms of service
- ☐ Pastikan sumber data bersifat publik
- ☐ Pastikan tujuan scraping jelas
- ☐ Jangan mengambil data pribadi tanpa dasar sah
- ☐ Jangan mengambil data login
- ☐ Jangan bypass proteksi
- ☐ Jangan mengambil data yang dibatasi aksesnya

E.2 Etika Teknis

- ☐ Gunakan delay
- ☐ Batasi request
- ☐ Gunakan retry secara wajar
- ☐ Gunakan timeout
- ☐ Jangan membanjiri server
- ☐ Hentikan proses jika muncul banyak 403 atau 429
- ☐ Jangan melakukan scraping agresif pada jam sibuk

E.3 Audit Data

- ☐ Simpan source_url
- ☐ Simpan scraped_at
- ☐ Simpan content_hash
- ☐ Simpan raw data jika perlu
- ☐ Simpan data rejected
- ☐ Simpan log pipeline
- ☐ Dokumentasikan tujuan scraping

E.4 Keamanan

- ☐ Credential disimpan di .env
- ☐ .env masuk .gitignore
- ☐ Tidak ada password di kode

- ☐ Tidak ada token di README
- ☐ Tidak ada data sensitif di log
- ☐ Tidak ada screenshot yang membocorkan credential

E.5 LLM

- ☐ Prompt melarang halusinasi
- ☐ Output mentah LLM disimpan
- ☐ Output JSON divalidasi
- ☐ Nama model disimpan
- ☐ Output LLM tidak dianggap fakta tanpa sumber
- ☐ Data sensitif tidak dikirim ke LLM

E.6 Keputusan Akhir

- ☐ Aman untuk dijalankan
- ☐ Aman untuk disimpan
- ☐ Aman untuk dianalisis
- ☐ Aman untuk dipublikasikan
- ☐ Aman untuk dijadikan portofolio

Lampiran F — Troubleshooting Praktis

Lampiran ini berisi masalah umum dan solusi cepat.

F.1 Error 403 — Forbidden

Gejala:

```
403 Forbidden
```

Penyebab umum:

- Server menolak request.
- Halaman tidak boleh diakses otomatis.
- Header kurang sesuai.
- Akses dibatasi.

Solusi:

- [] Cek apakah halaman boleh diakses
- [] Cek robots.txt dan aturan situs
- [] Jangan bypass proteksi
- [] Coba gunakan sumber latihan yang legal
- [] Hentikan scraping jika akses memang ditolak

F.2 Error 429 — Too Many Requests

Gejala:

429 Too Many Requests

Penyebab umum:

- Request terlalu cepat.
- Tidak ada delay.
- Terlalu banyak halaman diambil.

Solusi:

- [] Tambahkan delay
- [] Kurangi jumlah request
- [] Aktifkan throttling jika memakai Scrapy
- [] Tunggu sebelum mencoba lagi
- [] Jangan lanjutkan scraping agresif

Contoh Scrapy:

```
DOWNLOAD_DELAY = 2
AUTOTHROTTLE_ENABLED = True
```

F.3 Selector Berubah

Gejala:

- Data tiba-tiba kosong.
- Field tertentu hilang.
- Jumlah item turun drastis.

Penyebab umum:

- Struktur HTML berubah.
- Class berubah.
- Elemen berpindah.
- Website memakai render dinamis.

Solusi:

- [] Simpan raw HTML
- [] Cek ulang struktur HTML
- [] Gunakan selector yang lebih stabil
- [] Ambil container item dulu
- [] Buat fallback selector

Contoh:

```
title_node = node.select_one(".article-title a") or  
node.select_one("h2 a")
```

F.4 Data Kosong

Gejala:

- Output JSON kosong.
- CSV hanya header.
- Jumlah item nol.

Penyebab umum:

- Selector salah.
- Response bukan HTML target.
- Data dimuat JavaScript.
- Status code bukan 200.
- Halaman menampilkan error.

Solusi:

- [] Cek status code
- [] Cek response.text awal
- [] Simpan raw HTML
- [] Ambil screenshot jika memakai Playwright
- [] Gunakan Playwright jika data muncul setelah JavaScript

F.5 Encoding Rusak

Gejala:

```
Pendidikan â€™ digital
```

Penyebab umum:

- Encoding salah baca.
- Response tidak UTF-8.
- File disimpan dengan encoding berbeda.

Solusi:

- [] Cek response.encoding
- [] Simpan file dengan UTF-8
- [] Gunakan encoding eksplisit saat read/write

Contoh:

```
text = response.text
output_file.write_text(text, encoding="utf-8")
```


F.6 Docker Database Gagal Start

Gejala:

```
container exited  
port already allocated
```

Penyebab umum:

- Port sudah dipakai.
- Volume rusak.
- Konfigurasi salah.
- Password atau environment salah.

Solusi:

```
docker ps  
docker ps -a  
docker logs scraping_postgres  
docker logs scraping_mongo
```

Jika port bentrok, ubah port di `docker-compose.yml`.

Contoh:

```
ports:  
  - "5433:5432"
```

F.7 Koneksi PostgreSQL Gagal

Gejala:

```
connection refused
authentication failed
database does not exist
```

Penyebab umum:

- Container belum berjalan.
- Host/port salah.
- User/password salah.
- Database belum dibuat.

Solusi:

```
[ ] Cek docker ps
[ ] Cek .env
[ ] Cek POSTGRES_HOST
[ ] Cek POSTGRES_PORT
[ ] Cek POSTGRES_DB
[ ] Cek POSTGRES_USER
[ ] Cek POSTGRES_PASSWORD
```

Tes log:

```
docker logs scraping_postgres
```

F.8 MongoDB Tidak Bisa Connect

Gejala:

```
ServerSelectionTimeoutError
```

Penyebab umum:

- Container belum aktif.
- URI salah.
- Port salah.
- Service belum siap.

Solusi:

```
docker ps
docker logs scraping_mongo
```

Cek `.env`:

```
MONGO_URI=mongodb://localhost:27017
MONGO_DB=scraping_db
```

F.9 Output LLM Bukan JSON

Gejala:

- LLM menjawab dengan paragraf.
- JSON rusak.
- Ada teks sebelum atau sesudah JSON.

Penyebab umum:

- Prompt kurang tegas.
- Tidak meminta output JSON valid.
- Model tidak konsisten.
- Teks input terlalu panjang.

Solusi:

- [] Minta output hanya JSON valid
- [] Gunakan schema sederhana
- [] Simpan raw_llm_output
- [] Parse JSON secara eksplisit
- [] Validasi dengan Pydantic
- [] Tolak output yang tidak valid

Prompt aman:

Output hanya JSON valid.
Jangan tulis penjelasan di luar JSON.
Jika data tidak ada, gunakan null atau list kosong.
Jangan menebak.

F.10 Playwright Timeout

Gejala:

`TimeoutError`

Penyebab umum:

- Selector tidak muncul.
- Halaman lambat.
- JavaScript error.
- Data tidak ada.
- Selector salah.

Solusi:

- [] Ambil screenshot
- [] Jalankan `headless=False` untuk debugging
- [] Cek selector
- [] Tunggu selector yang benar
- [] Jangan hanya memakai `fixed sleep`

Contoh:

```
page.locator(".article-card").first.wait_for(timeout=10000)
page.screenshot(path="logs/debug.png", full_page=True)
```

F.11 Ringkasan Troubleshooting Cepat

Masalah	Cek Cepat	Solusi Utama
403	Akses ditolak	Jangan bypass, cek aturan situs
429	Terlalu banyak request	Tambah delay, kurangi request
Selector berubah	Data kosong	Cek raw HTML, update selector
Data kosong	Status/HTML/JS	Cek status code, pakai screenshot
Encoding rusak	Teks aneh	Pakai UTF-8 eksplisit
Database gagal start	<code>docker logs</code>	Perbaiki port/config
PostgreSQL gagal connect	<code>.env</code> dan container	Cek host, port, user, password
MongoDB gagal connect	URI dan container	Cek <code>MONGO_URI</code>
LLM bukan JSON	Prompt/output	Validasi Pydantic
Playwright timeout	Screenshot	Tunggu selector yang benar

Prinsip akhir troubleshooting: jangan menebak. Simpan raw data, simpan log, ambil screenshot, baca error, lalu perbaiki dari sumber masalah.

Glossary

Istilah	Pengertian
API Access	Cara mengambil data melalui <i>Application Programming Interface</i> yang disediakan secara resmi oleh sistem atau layanan tertentu. Biasanya lebih stabil daripada scraping HTML.
API Key	Kunci rahasia untuk mengakses API. Harus disimpan aman di <code>.env</code> , bukan ditulis langsung di kode.
Audit Trail	Jejak proses data dari sumber awal sampai hasil akhir, termasuk <code>source_url</code> , <code>scraped_at</code> , <code>content_hash</code> , dan status validasi.
BeautifulSoup	Library Python untuk membaca dan mengekstrak data dari HTML atau XML. Cocok untuk scraping sederhana dan parsing halaman statis.
Browser Automation	Teknik mengendalikan browser secara otomatis untuk membuka halaman, klik tombol, mengisi form, dan membaca konten hasil render.
Clean Data	Data yang sudah dibersihkan, dinormalisasi, divalidasi, dan siap disimpan ke database atau diproses lebih lanjut.
Cleaner	Komponen pipeline yang membersihkan teks, menghapus spasi berlebihan, newline, HTML tersisa, dan format yang tidak konsisten.
Cleaning	Proses membersihkan data kotor agar lebih rapi, konsisten, dan siap dianalisis.
Content Hash	Sidik digital dari isi data, biasanya dibuat dengan SHA256, untuk membantu mendeteksi duplikasi atau perubahan konten.
Crawler	Program yang menjelajahi banyak halaman web secara otomatis dengan mengikuti link atau pagination.
Crawling	Proses menjelajahi halaman web secara otomatis untuk menemukan dan mengambil banyak halaman.
Crawl4AI	Tool open source untuk crawling yang ramah LLM, menghasilkan Markdown bersih, dan mendukung pipeline RAG atau ekstraksi berbasis AI.
CSV	Format data tabular sederhana berbasis teks, sering dipakai untuk pertukaran data dan analisis awal dengan Pandas.
Data Governance	Tata kelola data yang mengatur asal-usul, kualitas, keamanan, izin penggunaan, audit, dan retensi data.

Data Ingestion	Proses memasukkan data dari berbagai sumber ke sistem penyimpanan atau pipeline.
Data Lineage	Informasi asal-usul dan perjalanan data dari sumber sampai masuk database atau hasil analisis.
Data Quality	Ukuran kualitas data, mencakup kelengkapan, konsistensi, validitas, akurasi, dan bebas duplikasi.
Database	Sistem penyimpanan data yang memungkinkan data dicari, difilter, dianalisis, dan digunakan aplikasi.
Deduplication	Proses menghapus data duplikat berdasarkan URL, hash, atau kombinasi field tertentu.
Docker	Platform untuk menjalankan aplikasi dan database dalam container agar lebih mudah direplikasi dan dipindahkan.
Docker Compose	Tool untuk menjalankan beberapa container melalui satu file konfigurasi YAML, misalnya PostgreSQL dan MongoDB.
Document Database	Jenis database yang menyimpan data dalam bentuk dokumen mirip JSON, cocok untuk data semi-terstruktur.
Encoding	Cara merepresentasikan teks dalam bentuk byte, misalnya UTF-8. Encoding salah dapat membuat teks rusak.
Entity Extraction	Proses mengekstrak entitas dari teks, seperti nama orang, organisasi, lokasi, teknologi, atau konsep.
Error Handling	Teknik menangani error agar pipeline tidak berhenti tanpa catatan dan tetap menghasilkan log yang jelas.
Feed Export	Fitur Scrapy untuk mengekspor hasil scraping ke format seperti JSON, JSON Lines, atau CSV.
Field	Kolom atau atribut data, misalnya <code>title</code> , <code>url</code> , <code>author</code> , <code>summary</code> , atau <code>scraped_at</code> .
Gitignore	File <code>.gitignore</code> berisi daftar file yang tidak boleh ikut dipublikasikan, seperti <code>.env</code> , cache, log sensitif, dan database lokal.
HTML	Bahasa markup untuk membangun struktur halaman web, berisi tag seperti <code><h1></code> , <code><p></code> , <code><a></code> , <code><table></code> , dan <code><form></code> .
HTTP Request	Permintaan yang dikirim client atau script ke server, misalnya untuk mengambil halaman web.
HTTP Response	Balasan dari server terhadap request, berisi status code, header, dan body seperti HTML atau JSON.

Index	Struktur database untuk mempercepat pencarian pada kolom tertentu, seperti <code>url</code> , <code>date</code> , atau <code>content_hash</code> .
Item	Struktur data hasil scraping pada Scrapy, biasanya berisi field seperti <code>title</code> , <code>url</code> , <code>date</code> , dan <code>summary</code> .
JSON	Format data terstruktur berbasis pasangan key-value yang mudah dibaca Python, API, database, dan LLM.
JSON Lines	Format data di mana setiap baris adalah satu JSON valid. Cocok untuk pipeline dan dataset besar.
LLM	<i>Large Language Model</i> , model AI bahasa yang dapat membantu ringkasan, ekstraksi entitas, klasifikasi, dan konversi teks ke JSON.
LLM-Assisted Extraction	Teknik memakai LLM untuk membantu mengubah teks hasil scraping menjadi informasi terstruktur.
Log	Catatan proses pipeline, error, status, dan hasil eksekusi. Penting untuk debugging dan audit.
MongoDB	Database dokumen open source yang cocok untuk menyimpan data JSON, raw pages, metadata, dan data semi-terstruktur.
Ollama	Tool untuk menjalankan model LLM secara lokal dan menyediakan API lokal untuk integrasi dengan Python.
Pandas	Library Python untuk membaca, membersihkan, menganalisis, dan menyimpan data tabular seperti CSV atau JSON Lines.
Pagination	Mekanisme halaman bertahap, misalnya page 1, page 2, atau tombol <i>next</i> , yang perlu diikuti saat crawling.
Parser	Komponen yang membaca HTML atau XML agar elemen tertentu dapat dipilih dan diekstrak.
Pipeline Data	Alur kerja data dari pengambilan, pembersihan, validasi, penyimpanan, pemrosesan LLM, sampai query atau laporan.
Playwright	Tool browser automation untuk scraping website dinamis yang memuat data melalui JavaScript.
PostgreSQL	Database relasional open source yang kuat untuk menyimpan data bersih, hasil ekstraksi, audit trail, dan query analitik.
Primary Key	Kolom unik utama pada tabel database untuk mengidentifikasi setiap baris data.
Prompt	Instruksi yang diberikan kepada LLM agar menghasilkan output tertentu, misalnya JSON, ringkasan, atau entitas.

Pydantic	Library Python untuk validasi data berbasis <i>type hints</i> , berguna untuk memastikan output scraping dan LLM sesuai schema.
Query	Perintah untuk membaca, memfilter, mengurutkan, menghitung, atau menganalisis data dari database.
RAG	<i>Retrieval-Augmented Generation</i> , pendekatan AI yang menggabungkan pencarian dokumen dengan generasi jawaban oleh LLM.
Rate Limit	Pembatasan jumlah request dalam periode tertentu agar tidak membebani server.
Raw Data	Data mentah hasil scraping sebelum dibersihkan dan divalidasi.
Rejected Data	Data yang gagal validasi dan tidak dimasukkan ke dataset bersih atau database utama.
Reproducibility	Kemampuan proyek untuk dijalankan ulang oleh orang lain dengan hasil dan alur yang jelas.
Request Header	Metadata dalam HTTP request, seperti <i>User-Agent</i> , <i>Content-Type</i> , dan <i>Accept-Language</i> .
Responsible Scraping	Praktik scraping yang legal, etis, hemat resource, tidak mengambil data pribadi, dan menghormati aturan situs.
Retry	Mekanisme mencoba ulang proses yang gagal karena error sementara, misalnya timeout atau koneksi terputus.
robots.txt	File yang memberi arahan kepada crawler tentang area website yang boleh atau tidak boleh diakses.
Schema	Struktur data yang mendefinisikan field, tipe data, aturan validasi, dan batas kualitas.
Scraped Item	Satu record hasil scraping, misalnya satu artikel, satu produk, atau satu baris tabel.
Scraper	Script atau program yang mengambil dan mengekstrak data dari halaman web.
Scrapy	Framework open source untuk crawling dan scraping banyak halaman secara lebih terstruktur.
Screenshot Debugging	Teknik mengambil gambar halaman saat scraping untuk melihat kondisi visual ketika data kosong atau error.
Selector	Pola untuk memilih elemen HTML, misalnya CSS selector atau XPath.

SQLite	Database ringan berbasis file tunggal, cocok untuk latihan lokal dan prototipe kecil.
Status Code	Kode balasan HTTP, misalnya 200, 301, 403, 404, 429, dan 500.
Structured Data	Data yang sudah memiliki field jelas dan konsisten, sehingga mudah disimpan, dicari, dan dianalisis.
Terms of Service	Ketentuan penggunaan sebuah website atau layanan yang harus diperhatikan sebelum scraping.
Timeout	Batas waktu tunggu sebelum request atau proses dianggap gagal.
User-Agent	Header HTTP yang memberi informasi tentang client yang mengakses server.
Validation	Proses memastikan data sesuai schema, memiliki field wajib, dan tidak melanggar aturan kualitas.
Virtual Environment	Lingkungan Python terisolasi untuk menyimpan dependency proyek tanpa mengganggu sistem utama.
Web Scraping	Proses mengambil data dari halaman web dan mengubahnya menjadi format yang dapat diproses.
XPath	Bahasa selector untuk memilih elemen berdasarkan struktur dokumen HTML/XML.

Referensi

Beautiful Soup. (n.d.). *Beautiful Soup documentation*.

<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Crawl4AI. (n.d.). *Crawl4AI documentation*. <https://docs.crawl4ai.com/>

Docker. (n.d.). *Docker Compose overview*. <https://docs.docker.com/compose/>

Fielding, R., Nottingham, M., & Koster, M. (2022). *RFC 9309: Robots Exclusion Protocol*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9309.html>

lxml. (n.d.). *lxml: XML and HTML with Python*. <https://lxml.de/>

MongoDB. (n.d.). *Install MongoDB Community with Docker*.

<https://www.mongodb.com/docs/v7.0/tutorial/install-mongodb-community-with-docker/>

MongoDB. (n.d.). *MongoDB manual*. <https://www.mongodb.com/docs/manual/>

National Institute of Standards and Technology. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*.

<https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>

Ollama. (n.d.). *Ollama API documentation*. <https://docs.ollama.com/api/introduction>

OWASP Foundation. (n.d.). *Secrets Management Cheat Sheet*.

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Pandas. (n.d.). *Pandas documentation: Input/output*.

https://pandas.pydata.org/docs/user_guide/io.html

Pandas. (n.d.). *pandas.DataFrame.drop_duplicates*.

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

Pandas. (n.d.). *pandas.read_csv*.

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

Pandas. (n.d.). *pandas.read_html*.

https://pandas.pydata.org/docs/reference/api/pandas.read_html.html

Playwright. (n.d.). *Playwright for Python*. <https://playwright.dev/python/docs/intro>

Playwright. (n.d.). *Playwright locator API*. <https://playwright.dev/docs/api/class-locator>

Playwright. (n.d.). *Screenshots*. <https://playwright.dev/docs/screenshots>

PostgreSQL Global Development Group. (n.d.). *About PostgreSQL*.

<https://www.postgresql.org/about/>

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation: INSERT*. <https://www.postgresql.org/docs/current/sql-insert.html>

Purbo, O. W. (2026). *Web Scraping + Database dengan Open Source LLM: Pipeline Data Modern*. Institut Teknologi Tangerang Selatan, Dinas KOMINFO Tangerang Selatan, OnnoCenter. Dokumen terlampir.

Pydantic. (n.d.). *Pydantic documentation*. <https://docs.pydantic.dev/>

Pydantic. (n.d.). *Pydantic validation*. <https://pydantic.dev/docs/validation/latest/get-started/>

Python Packaging Authority. (n.d.). *Installing packages using pip and virtual environments*. <https://packaging.python.org/guides/installing-using-pip-and-virtual-environments/>

Python Software Foundation. (n.d.). *hashlib — Secure hashes and message digests*. <https://docs.python.org/3/library/hashlib.html>

Python Software Foundation. (n.d.). *logging — Logging facility for Python*. <https://docs.python.org/3/library/logging.html>

Python Software Foundation. (n.d.). *sqlite3 — DB-API 2.0 interface for SQLite databases*. <https://docs.python.org/3/library/sqlite3.html>

Python Software Foundation. (n.d.). *venv — Creation of virtual environments*. <https://docs.python.org/3/library/venv.html>

Requests. (n.d.). *Requests: HTTP for Humans*. <https://requests.readthedocs.io/en/latest/>

Scrapy. (n.d.). *Scrapy documentation*. <https://docs.scrapy.org/>

Scrapy. (n.d.). *Feed exports*. <https://docs.scrapy.org/en/latest/topics/feed-exports.html>

Scrapy. (n.d.). *Item pipeline*. <https://docs.scrapy.org/en/latest/topics/item-pipeline.html>

SQLite. (n.d.). *About SQLite*. <https://sqlite.org/about.html>

SQLite. (n.d.). *SQLite documentation*. <https://sqlite.org/docs.html>

VSCodium. (n.d.). *VSCodium: Open source binaries of code editor*. <https://vscodium.com/>

WHATWG. (n.d.). *HTML Living Standard*. <https://html.spec.whatwg.org/>

World Wide Web Consortium. (n.d.). *HTML and CSS*. <https://www.w3.org/standards/webdesign/htmlcss.html>

Tentang Penulis



Onno W. Purbo, saat ini bertugas sebagai rektor di Institut Teknologi Tangerang Selatan (ITTS). Onno memperoleh gelar Ph.D bidang Electrical Engineering dari University of Waterloo, Canada, adalah seorang copyleft, educator dan ICT evangelist. Dia sudah mempublikasikan 50++ buku, termasuk free ICT ebook untuk sekolah tahun 2008. Beberapa buku terakhirnya adalah "Internet-TCP/IP: Konsep Dan Implementasi", 2018; "Sistem Operasi, Konsep Dan Membuat Linux OpenWRT Dan ROM Android", 2019; "IPv6 Untuk Mendukung Operasi Jaringan Dan Domain Name System", 2019; "Kubernetes untuk Pemula", 2024 dan "Membuat Operator Seluler 5G Sendiri", 2024.

Dia memimpin sambungan pertama Internet di Institut Teknologi Bandung, tahun 1993-2000, dan menggunakannya untuk membuat jaringan Internet pendidikan yang pertama di Indonesia. Dia membebaskan frekuensi WiFi, memperkenalkan RT/RW-net, antenna Wajanbolic dan jaringan selular OpenBTS dan private 5G sendiri. Dia memimpin jaringan telepon pertama di atas Internet, VoIP Merdeka, yang kemudian hari dikenal sebagai VoIP Rakyat berbasis SIP dan menggunakan kode area +62520 dan +62521. Dia saat ini aktif memperkenalkan e-Learning, dan menjalankan server e-Learning di <http://lms.onnocenter.or.id/moodle/> 66,000++ siswa / mahasiswa dan <https://opencourse.itts.ac.id> 39.000+ mahasiswa secara gratis. Email: onno@indo.net.id